

Introduction to Numerical Methods and MATLAB Programming for Engineers

Todd Young and Martin J. Mohlenkamp
Department of Mathematics
Ohio University
Athens, OH 45701
`youngt@ohio.edu`

May 13, 2026

Copyright © 2008, 2009, 2011, 2014, 2016, 2017, 2018, 2020, 2021, 2023, 2025 Todd R. Young and Martin J. Mohlenkamp.
Original edition 2004, by Todd R. Young.

Thanks go to many students who have pointed out typos and mistakes. Thank you to Dr. Yaqin Feng for suggestions to improve the exercises.

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-sa/4.0/>.



Preface

These notes were developed by the first author in the process of teaching a course on applied numerical methods for Civil Engineering majors during 2002-2004 and was modified to include Mechanical Engineering in 2005. The materials have been periodically updated since then and underwent a major revision by the second author in 2006-2007.

The main goals of these lectures are to introduce concepts of numerical methods and introduce MATLAB in an Engineering framework. By this we do not mean that every problem is a “real life” engineering application, but more that the engineering way of thinking is emphasized throughout the discussion.

The philosophy of this book was formed over the course of many years. My father was a Civil Engineer and surveyor, and he introduced me to engineering ideas from an early age. At the University of Kentucky I took most of the basic Engineering courses while getting a Bachelor’s degree in Mathematics. Immediately afterward, I completed a M.S. degree in Engineering Mechanics at Kentucky.

While working on my Ph.D. in Mathematics at Georgia Tech I taught all of the introductory math courses for engineers. During my education, I observed that incorporation of computation in coursework had been extremely unfocused and poor. For instance during my college career I had to learn 8 different programming and markup languages on 4 different platforms plus numerous other software applications. There was almost no technical help provided in the courses and I wasted innumerable hours figuring out software on my own. A typical, but useless, inclusion of software has been (and still is in most calculus books) to set up a difficult ‘applied’ problem and then add the line “write a program to solve” or “use a computer algebra system to solve”.

At Ohio University we have tried to take a much more disciplined and focused approach. The Russ College of Engineering and Technology decided that MATLAB should be the primary computational software for undergraduates. At about the same time members of the Department of Mathematics proposed an 1804 project to bring MATLAB into the calculus sequence and provide access to the program at nearly all computers on campus, including in the dorm rooms. The stated goal of this project was to make MATLAB the universal language for computation on campus. That project was approved and implemented in the 2001-2002 academic year.

In these lecture notes, instruction on using MATLAB is dispersed through the material on numerical methods. In these lectures details about how to use MATLAB are detailed (but not verbose) and explicit. To teach programming, students are usually given examples of working programs and are asked to make modifications.

The lectures are designed to be used in a computer classroom with students working MATLAB

examples during the lecture or with students reading the notes and working the examples after a brief introduction. At Ohio University we have had good success with this Lecture/Lab format.

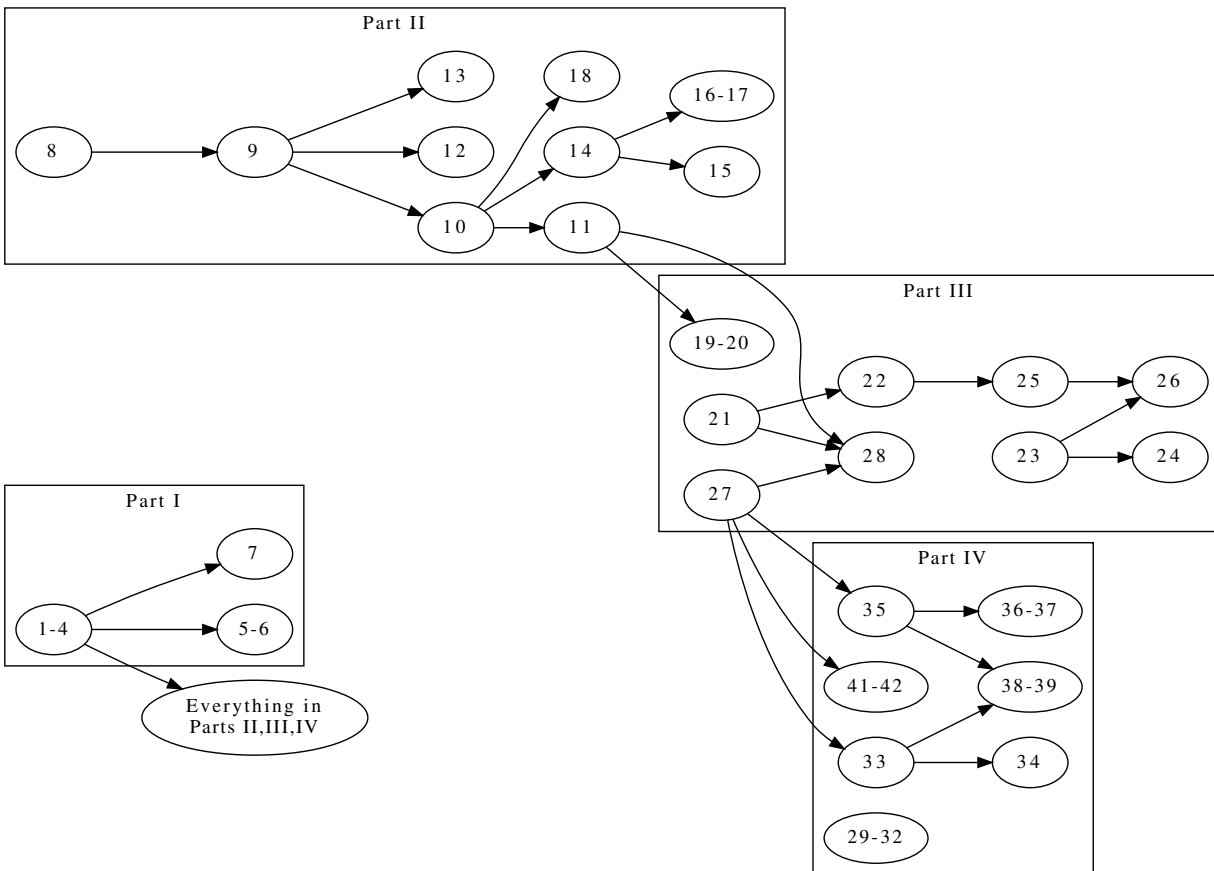
The lectures are divided into four Parts with a summary provided at the end of each Part. Typically we will give an exam covering each of Parts I, II, and III and a comprehensive final exam.

The exercises grow in complexity as the students build their programming skills. At Ohio University we ask students to complete the exercises in groups of 2-3 students and this accounts for a significant portion of the grade (e.g. 30%).

Todd Young

Dependencies

Below we give the dependencies between Lectures. Almost everything depends on Lectures 1–4, so those links are omitted to reduce clutter. Some lectures, marked with * in the table of contents, have not yet been developed.



Contents

Preface	iii
I Matlab and Solving Equations	1
Lecture 1. Vectors, Functions, and Plots in MATLAB	2
Lecture 2. MATLAB Programs	6
Lecture 3. Newton's Method and Loops	10
Lecture 4. Controlling Error and Conditional Statements	14
Lecture 5. The Bisection Method and Locating Roots	18
Lecture 6. Secant Methods	22
Lecture 7. Symbolic Computations	25
Review of Part I	30
II Linear Algebra	35
Lecture 8. Matrices and Matrix Operations in Matlab	36
Lecture 9. Introduction to Linear Systems	41
Lecture 10. Some Facts About Linear Systems	45
Lecture 11. Accuracy, Condition Numbers and Pivoting	48
Lecture 12. LU Decomposition	52
Lecture 13. Nonlinear Systems - Newton's Method	55
Lecture 14. Eigenvalues and Eigenvectors	59
Lecture 15. Vibrational Modes and Frequencies	62
Lecture 16. Numerical Methods for Eigenvalues	64
Lecture 17. The QR Method*	68

Lecture 18. Iterative solution of linear systems*	70
Review of Part II	71
III Functions and Data	75
Lecture 19. Polynomial and Spline Interpolation	76
Lecture 20. Least Squares Fitting: Noisy Data	80
Lecture 21. Integration: Left, Right and Trapezoid Rules	84
Lecture 22. Integration: Midpoint and Simpson's Rules	89
Lecture 23. Plotting Functions of Two Variables	93
Lecture 24. The Gradient and Max-Min Problems	96
Lecture 25. Double Integrals for Rectangles	99
Lecture 26. Double Integrals for Non-rectangles	103
Lecture 27. Numerical Differentiation	106
Lecture 28. The Main Sources of Error	110
Review of Part III	114
IV Differential Equations	121
Lecture 29. Reduction of Higher Order Equations to Systems	122
Lecture 30. Euler Methods	126
Lecture 31. Higher Order Methods	130
Lecture 32. Multi-step Methods*	133
Lecture 33. ODE Boundary Value Problems and Finite Differences	134
Lecture 34. Finite Difference Method – Nonlinear ODE	138
Lecture 35. Parabolic PDEs - Explicit Method	141
Lecture 36. Solution Instability for the Explicit Method	146
Lecture 37. Implicit Methods	149

Lecture 38. Insulated Boundary Conditions	153
Lecture 39. Finite Difference Method for Elliptic PDEs	158
Lecture 40. Convection-Diffusion Equations*	161
Lecture 41. Finite Elements	162
Lecture 42. Determining Internal Node Values	166
Review of Part IV	170
 V Appendices	 173
Lecture A. Glossary of Matlab Commands	174

Part I

Matlab and Solving Equations

©Copyright, Todd Young and Martin Mohlenkamp, Department of Mathematics, Ohio University, 2026

Lecture 1

Vectors, Functions, and Plots in MATLAB

In these notes

>>
>>

will indicate commands to be entered at the MATLAB prompt `>>` in the command window. You do not type the symbol `>>`.

Entering vectors

In MATLAB, the basic objects are matrices, i.e. arrays of numbers. Vectors can be thought of as special matrices. A row vector is recorded as a $1 \times n$ matrix and a column vector is recorded as a $m \times 1$ matrix. To enter a row vector in Matlab, type the following in the command window:

>> `v = [0 1 2 3]`

and press enter. MATLAB will print out the row vector. To enter a column vector type

>> `u = [9; 10; 11; 12; 13]`

You can access an entry in a vector with

>> `u(2)`

and change the value of that entry with

>> `u(2)=47`

You can extract a *slice* out of a vector with

>> `u(2:4)`

You can change a row vector into a column vector, and vice versa, easily in Matlab using

>> `w = v'`

(This is called *transposing* the vector and we call `'` the transpose operator.) There are also useful shortcuts to make vectors such as

>> `x = -1:.1:1`

>> `y = linspace(0,1,11)`

Basic Formatting

To make MATLAB put fewer blank lines in its output, enter

```
>> format compact
>> pi
>> x
```

To make MATLAB display more digits, enter

```
>> format long
>> pi
```

Note that this does not change the number of digits MATLAB is using in its calculations; it only changes what is displayed.

Plotting Data

Consider the data in Table 1.1.¹ We can enter this data into MATLAB with the following commands entered

T (C°)	5	20	30	50	55
μ	0.08	0.015	0.009	0.006	0.0055

Table 1.1: Viscosity of a liquid as a function of temperature.

in the command window:

```
>> x = [ 5 20 30 50 55 ]
>> y = [ 0.08 0.015 0.009 0.006 0.0055]
```

Entering the name of the variable retrieves its current values. For instance

```
>> x
>> y
```

We can plot data in the form of vectors using the plot command:

```
>> plot(x,y)
```

This will produce a graph with the data points connected by lines. If you would prefer that the data points be represented by symbols you can do so. For instance

```
>> plot(x,y,'*')
>> plot(x,y,'o')
>> plot(x,y,'.')
```

¹Adapted from Ayyup & McCuen 1996, p.174.

Data as a Representation of a Function

A major theme in this course is that often we are interested in a certain function $y = f(x)$, but the only information we have about this function is a discrete set of data $\{(x_i, y_i)\}$. Plotting the data, as we did above, can be thought of envisioning the function using just the data. We will find later that we can also do other things with the function, like differentiating and integrating, just using the available data. Numerical methods, the topic of this course, means doing mathematics by computer. Since a computer can only store a finite amount of information, we will almost always be working with a finite, discrete set of values of the function (data), rather than a formula for the function.

Built-in Functions

If we wish to deal with formulas for functions, MATLAB contains a number of built-in functions, including all the usual functions, such as `sin( )`, `exp( )`, etc.. The meaning of most of these is clear. The dependent variable (input) always goes in parentheses in MATLAB. For instance

```
>> sin(pi)
```

should return the value of $\sin \pi$, which is of course 0 and

```
>> exp(0)
```

will return e^0 , which is 1. More importantly, the built-in functions can operate not only on single numbers but on vectors. For example

```
>> x = linspace(0,2*pi,41)
>> y = sin(x)
>> plot(x,y)
```

will return a plot of $\sin x$ on the interval $[0, 2\pi]$

Some of the built-in functions in MATLAB include: `cos( )`, `tan( )`, `sinh( )`, `cosh( )`, `log( )` (natural logarithm), `log10( )` (log base 10), `asin( )` (inverse sine), `acos( )`, `atan( )`. To find out more about a function, use the `help` command; try

```
>> help plot
```

User-Defined Anonymous Functions

If we wish to deal with a function that is a combination of the built-in functions, MATLAB has a couple of ways for the user to define functions. One that we will use a lot is the anonymous function, which is a way to define a function in the command window. The following is a typical anonymous function:

```
>> f = @(x) 2*x.^2 - 3*x + 1
```

This produces the function $f(x) = 2x^2 - 3x + 1$. To obtain a single value of this function enter

```
>> y = f(2.23572)
```

Just as for built-in functions, the function f as we defined it can operate not only on single numbers but on vectors. Try the following:

```
>> x = -2:.2:2
>> y = f(x)
```

This is an example of *vectorization*, i.e. putting several numbers into a vector and treating the vector all at once, rather than one component at a time, and is one of the strengths of MATLAB. The reason $f(x)$ works when x is a vector is because we represented x^2 by $x.^2$. The $.$ turns the exponent operator $^$ into entry-wise exponentiation, so that $[-2 \ -1.8 \ -1.6].^2$ means $[(-2)^2, (-1.8)^2, (-1.6)^2]$ and yields $[4 \ 3.24 \ 2.56]$. In contrast, $[-2 \ -1.8 \ -1.6]^2$ means the matrix product $[-2, -1.8, -1.6][-2, -1.8, -1.6]$ and yields only an error. The $.$ is needed in $.^$, $.*$, and $./$. It is not needed when you $*$ or $/$ by a scalar or for $+$.

The results can be plotted using the `plot` command, just as for data:

```
>> plot(x,y)
```

Notice that before plotting the function, we in effect converted it into data. Plotting on any machine always requires this step.

Exercises

- 1.1 Recall that $.*$, $./$, $.^$ are component-wise operations. Make row vectors $\mathbf{a} = 0 : 1 : 3$ and $\mathbf{b} = [-1 \ 0 \ 1 \ 2]$. Try the following commands and report the answer (or error) they produce. Are any of the results surprising?

- (a) $\mathbf{a}.*\mathbf{b}$, (b) $\mathbf{a}*\mathbf{b}$, (c) $\mathbf{a}*\mathbf{b}'$, (d) $\mathbf{a}+3*\mathbf{b}$,
 (e) $\mathbf{a}./\mathbf{b}$, (f) $2*\mathbf{b}./\mathbf{a}$, (g) $\mathbf{a}.^3$, (h) $\mathbf{a}.^{\mathbf{b}}$,

- 1.2 Find a table of data in an engineering or science textbook or website. Input it as vectors and plot it, *using symbols at the data points*.

Use the insert icon to label the axes and add a title to your graph. Turn in the graph. Indicate what the data is and properly reference where it came from.

- 1.3 Find a *non-linear* function formula in an engineering or science textbook or website. Make an anonymous function that produces that function. Plot it with a *smooth curve* on a physically relevant domain.

Label the axes and add a title to your graph. Turn in the graph and include the Matlab command for the anonymous function. Indicate what the function means and properly reference where it came from.

Lecture 2

MATLAB Programs

In MATLAB, programs may be written and saved in files with a suffix `.m` called *M-files*. There are two types of M-file programs: *functions* and *scripts*.

Function Programs

Begin by clicking on the new document icon in the top left of the MATLAB window (it looks like an empty sheet of paper).

In the document window type the following:

```
function y = myfunc(x)
    y = 2*x.^2 - 3*x + 1;
end
```

Save this file as: `myfunc.m` in your working directory. This file can now be used in the command window just like any predefined Matlab function; in the command window enter:

```
>> x = -2:1:2;      % Produces a vector of x values
>> y = myfunc(x);   % Produces a vector of y values
>> plot(x,y)
```

Note that the fact we used `x` and `y` in both the function program and in the command window was just a coincidence. In fact, it is the name of the file `myfunc.m` that actually mattered, not what anything in it was called. We could just as well have made the function

```
function nonsense = yourfunc(inputvector)
    nonsense = 2*inputvector.^2 - 3*inputvector + 1;
end
```

Look back at the program. All function programs are like this one, the essential elements are:

- Begin with the word `function`.
- There is an input and an output.
- The output, name of the function and the input must appear in the first line.
- The body of the program must assign a value to the output variable(s).
- The program cannot access variables in the current workspace unless they are input.

- Internal variables inside a function do not appear in the current workspace.

Functions can have multiple inputs, which are separated by commas. For example:

```
function y = myfunc2d(x,p)
    y = 2*x.^p - 3*x + 1;
end
```

Functions can have multiple outputs, which are collected into a vector. Open a new document and type:

```
function [x2 x3 x4] = mypowers(x)
    x2 = x.^2;
    x3 = x.^3;
    x4 = x.^4;
end
```

Save this file as `mypowers.m`. In the command window, we can use the results of the program to make graphs:

```
>> x = -1:.1:1
>> [x2 x3 x4] = mypowers(x);
>> plot(x,x,'black',x,x2,'blue',x,x3,'green',x,x4,'red')
```

Printing, Returning, Capturing, and Printing

Notice that in the examples above, lines ending with a semicolon “;” did not print their results.

Try the following:

```
>> myfunc(3)
>> ans^2
```

Although `myfunc` returned a value, we did not capture it. By default MATLAB captured it as `ans` so we can use it in our next computation. However, MATLAB always uses `ans` (for answer), so the result is likely to get overwritten.

Then try:

```
>> z = 0
>> z = myfunc(2)
>> z^2
```

`myfunc` returned a value that it internally called `y` and we captured the result in `z`. We can now use `z` for other calculations.

Now make a program

```
function myfuncnoreturn(x)
    y = 2*x.^2 - 3*x + 1
end
```

and try:

```
>> myfuncnoreturn(4)
>> ans^2
>> y^2
```

Although the value of `y` was printed within the function, it was not returned, so neither the value of `y` nor the value of `ans` was changed. Thus we cannot use the result from the function.

In general, the best way to use a function is to capture the result it returns and then use or print this result. Printing within functions is bad form; however, for understanding what is happening within a function it is useful to print, so many functions in this book do print.

Script Programs

MATLAB uses a second type of program that differs from a function program in several ways, namely:

- There are no inputs and outputs.
- A script program may use, create and change variables in the current workspace (the variables used by the command window).

Below is a script program that accomplishes the same thing as the function program plus the commands in the previous section:

```
x2 = x.^2;
x3 = x.^3;
x4 = x.^4;
plot(x,x,'black',x,x2,'blue',x,x3,'green',x,x4,'red')
```

Type this program into a new document and save it as `mygraphs.m`. In the command window enter:

```
>> x = -1:.1:1;
>> mygraphs
```

Note that the program used the variable `x` in its calculations, even though `x` was defined in the command window, not in the program.

Many people use script programs for routine calculations that would require typing more than one command in the command window. They do this because correcting mistakes is easier in a program than in the command window.

Program Comments

For programs that have more than a couple of lines it is important to include comments. Comments allow other people to know what your program does and they also remind yourself what your program does if you set it aside and come back to it later. It is best to include comments not only at the top of a program, but also with each section. In MATLAB anything that comes in a line after a `%` is a comment.

For a function program, the comments should at least give the purpose, inputs, and outputs. A properly commented version of the function with which we started this section is:

```
function y = myfunc(x)
    % Computes the function 2x^2 -3x +1
    % Input: x -- a number or vector;
    %           for a vector the computation is elementwise
    % Output: y -- a number or vector of the same size as x
    y = 2*x.^2 - 3*x + 1;
end
```

For a script program, there should be an initial comment stating the purpose of the script. It is also helpful to include the name of the program at the beginning. For example:

```
% mygraphs
% plots the graphs of x, x^2, x^3, and x^4
% on the interval [-1,1]

% fix the domain and evaluation points
x = -1:.1:1;

% calculate powers
% x1 is just x
x2 = x.^2;
x3 = x.^3;
x4 = x.^4;

% plot each of the graphs
plot(x,x,'+- ',x,x2,'x- ',x,x3,'o- ',x,x4,'--')
```

The MATLAB command `help` prints the first block of comments from a file. If we save the above as `mygraphs.m` and then do

```
>> help mygraphs
```

it will print into the command window:

```
>> mygraphs
>> plots the graphs of x, x^2, x^3, and x^4
>> on the interval [-1,1]
```

Exercises

- 2.1 Write a well-commented **function** program for the function $x^2 e^{-x^2}$, using component-wise operations (such as `.*` and `.^`). To get e^x use `exp(x)`. Plot the function on $[-5, 5]$ using enough points to make the graph smooth. Turn in the program and the graph.
- 2.2 Write a well-commented **script** program that graphs the functions $\sin x$, $\sin 2x$, $\sin 3x$, $\sin 4x$, $\sin 5x$ and $\sin 6x$ on the interval $[0, 2\pi]$ on one plot. (π is `pi` in MATLAB.) Use a sufficiently small step size to make all the graphs smooth. Turn in the program and the graph.

Lecture 3

Newton's Method and Loops

Solving equations numerically

For the next few lectures we will focus on the problem of solving an equation:

$$f(x) = 0. \tag{3.1}$$

As you learned in calculus, the final step in many optimization problems is to solve an equation of this form where f is the derivative of a function, F , that you want to maximize or minimize. In real engineering problems the functions, f , you wish to find roots for can come from a large variety of sources, including formulas, solutions of differential equations, experiments, or simulations.

Newton iterations

We will denote an actual solution of equation (3.1) by x^* . There are three methods that you may have discussed in Calculus: the bisection method, the secant method and Newton's method. All three depend on beginning close (in some sense) to an actual solution x^* .

Recall Newton's method. You should know that the basis for Newton's method is approximation of a function by its linearization at a point, i.e.

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0). \tag{3.2}$$

Since we wish to find x so that $f(x) = 0$, set the left hand side ($f(x)$) of this approximation equal to 0 and solve for x to obtain:

$$x \approx x_0 - \frac{f(x_0)}{f'(x_0)}. \tag{3.3}$$

We begin the method with the initial guess x_0 , which we hope is fairly close to x^* . Then we define a sequence of points $\{x_0, x_1, x_2, x_3, \dots\}$ from the formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \tag{3.4}$$

which comes from (3.3). If $f(x)$ is reasonably well-behaved near x^* and x_0 is close enough to x^* , then it is a fact that the sequence will converge to x^* and will do it very quickly.

The loop: `for ... end`

In order to do Newton's method, we need to repeat the calculation in (3.4) a number of times. This is accomplished in a program using a *loop*, which means a section of a program which is repeated. The

simplest way to accomplish this is to count the number of times through. In MATLAB, a `for ... end` statement makes a loop as in the following simple function program:

```
% mysum
% Gives the sum of the first n integers
%
n = 100
S = 0;           % start at zero
% The loop:
for i = 1:n      % do n times
    S = S + i;   % add the current integer
end             % end of the loop
S
```

Save this program as a **script** named `mysum` and run it by typing `mysum` in the command window. The result will be the sum of the first 100 integers. Next change n to 1,000,000 and run the program again.

All `for ... end` loops have the same format, it begins with `for`, followed by an index (i) and a range of numbers (`1:n`). Then come the commands that are to be repeated. Last comes the `end` command.

Loops are one of the main ways that computers are made to do calculations that humans cannot. Any calculation that involves a repeated process is easily done by a loop.

Now let's do a program that does n steps (iterations) of Newton's method. We will need to input the function, its derivative, the initial guess, and the number of steps. The output will be the final value of x , i.e. x_n . If we are only interested in the final approximation, not the intermediate steps, which is usually the case in the real world, then we can use a single variable x in the program `mynewton.m` and change it at each step:

```
function x = mynewton(f,f1,x0,n)
% Solves f(x) = 0 by doing n steps of Newton's method starting at x0.
% Inputs: f -- the function
%         f1 -- it's derivative
%         x0 -- starting guess, a number
%         n -- the number of steps to do
% Output: x -- the approximate solution
x = x0;           % set x equal to the initial guess x0
for i = 1:n       % Do n times
    x = x - f(x)/f1(x) % Newton's formula, prints x too
end
end
```

In the command window set to print more digits via

```
>> format long
```

and to not print blank lines via

```
>> format compact
```

Then define a function: $f(x) = x^3 - 5$ i.e.

```
>> f = @(x) x^3 - 5
```

and define $f1$ to be its derivative, i.e.

```
>> f1 = @(x) 3*x^2
```

Then run `mynewton` on this function. By trial and error, what is the lowest value of `n` for which the program converges (stops changing). By simple algebra, the true root of this function is $\sqrt[3]{5}$. How close is the program's answer to the true value?

Convergence

Newton's method converges rapidly when $f'(x^*)$ is nonzero and finite, and x_0 is close enough to x^* that the linear approximation (3.2) is valid. Let us take a look at what can go wrong.

For $f(x) = x^{1/3}$ we have $x^* = 0$ but $f'(x^*) = \infty$. If you try

```
>> f = @(x) x^(1/3)
>> f1 = @(x) (1/3)*x^(-2/3)
>> x = mynewton(f,f1,0.1,10)
```

then x explodes.

For $f(x) = x^2$ we have $x^* = 0$ but $f'(x^*) = 0$. If you try

```
>> f = @(x) x^2
>> f1 = @(x) 2*x
>> x = mynewton(f,f1,1,10)
```

then x does converge to 0, but not that rapidly.

If x_0 is not close enough to x^* that the linear approximation (3.2) is valid, then the iteration (3.4) gives some x_1 that may or may not be any better than x_0 . If we keep iterating, then either

- x_n will eventually get close to x^* and the method will then converge (rapidly), or
- the iterations will not approach x^* .

Exercises

- 3.1 Enter: `format long`. Use `mynewton.m` on the function $f(x) = x^5 - 2$, with $x_0 = 1$. By trial and error, what is the lowest value of `n` for which the program converges (stops changing). Compute the error, which is how close the program's answer is to the true value. Compute the residual, which is the program's answer plugged into f . (See the next section for discussion.) Are the error and residual zero?
- 3.2 For $f(x) = x^2 - 5$, perform 3 iterations of Newton's method with starting point $x_0 = 2$. (By hand, but use a calculator.) Show all steps. Calculate the solution ($x^* = \sqrt{5}$) on a calculator and find the errors and percentage errors of x_0 , x_1 , x_2 and x_3 . Use enough digits so that you do not falsely conclude the error is zero.
- 3.3 Suppose a ball is dropped from a height of 3 meters onto a hard surface and the coefficient of restitution of the collision is .91 (see Wikipedia for an explanation). Write a well-commented **script** program to calculate the total distance the ball has traveled when it hits the surface for the `n`-th time. Enter: `format long`. By trial and error approximate how large `n` must be so that total distance stops changing. Turn in the program and a brief summary of the results. (This program does not use Newton's method. It should be modeled on `mysum.m`.)

Lecture 4

Controlling Error and Conditional Statements

Measuring error and the Residual

If we are trying to find a numerical solution of an equation $f(x) = 0$, then there are a few different ways we can measure the error of our approximation. The most direct way to measure the error would be as

$$\{\text{Error at step } n\} = e_n = x_n - x^*$$

where x_n is the n -th approximation and x^* is the true value. However, we usually do not know the value of x^* , or we wouldn't be trying to approximate it. This makes it impossible to know the error directly, and so we must be more clever.

One possible strategy, which often works, is to run a program until the approximation x_n stops changing. The problem with this is that it sometimes doesn't work. Just because the program stop changing does not necessarily mean that x_n is close to the true solution.

For Newton's method we have the following principle: **At each step the number of significant digits roughly doubles.** While this is an important statement about the error (since it means Newton's method converges really quickly), it is somewhat hard to use in a program.

Rather than measure how close x_n is to x^* , in this and many other situations it is much more practical to measure how close the equation is to being satisfied, in other words, how close $y_n = f(x_n)$ is to 0. We will use the quantity $r_n = f(x_n) - 0$, called the *residual*, in many different situations. Most of the time we only care about the size of r_n , so we use the absolute value of the residual as a measure of how close the solution is to solving the problem:

$$|r_n| = |f(x_n)|.$$

The if ... end statement

If we have a certain tolerance for $|r_n| = |f(x_n)|$, then we can incorporate that into our Newton method program using an if ... end statement:

```
function x = mynewton(f,f1,x0,n,tol)
% Solves f(x) = 0 by doing n steps of Newton's method starting at x0.
% Inputs: f -- the function
%         f1 -- it's derivative
%         x0 -- starting guess, a number
%         tol -- desired tolerance, prints a warning if |f(x)|>tol
% Output: x -- the approximate solution
x = x0; % set x equal to the initial guess x0
```

```

for i = 1:n                % Do n times
    x = x - f(x)/f1(x)    % Newton's formula
end
r = abs(f(x))
if r > tol
    warning('The desired accuracy was not attained')
end
end

```

In this program `if` checks if `abs(y) > tol` is true or not. If it is true then it does everything between there and `end`. If not true, then it skips ahead to `end`.

In the command window define a function and its derivative:

```

>> f = @(x) x^3-5
>> f1 = @(x) 3*x^2

```

Then use the program with $n = 3$, $tol = .01$, and $x_0 = 2$. Next, change tol to 10^{-10} and repeat.

The loop: while ... end

While the previous program will tell us if it worked or not, we still have to input `n`, the number of steps to take. Even for a well-behaved problem, if we make `n` too small then the tolerance will not be attained and we will have to go back and increase it, or, if we make `n` too big, then the program will take more steps than necessary.

One way to control the number of steps taken is to iterate until the residual $|r_n| = |f(x)| = |y|$ is small enough. In MATLAB this is easily accomplished with a `while ... end` loop.

```

function x = mynewtontol(f,f1,x0,tol)
% Solves f(x) = 0 using Newton's method until |f(x)| < tol.
% Inputs: f -- the function
%         f1 -- it's derivative
%         x0 -- starting guess, a number
%         tol -- desired tolerance, runs until |f(x)|<tol
% Output: x -- the approximate solution
x = x0;                % set x equal to the initial guess x0
y = f(x);
while abs(y) > tol      % Do until the tolerance is reached.
    x = x - y/f1(x)    % Newton's formula
    y = f(x)
end
end

```

The statement `while ... end` is a loop, similar to `for ... end`, but instead of going through the loop a fixed number of times it keeps going as long as the statement `abs(y) > tol` is true.

One obvious drawback of the program is that `abs(y)` might never get smaller than `tol`. If this happens, the program would continue to run over and over until we stop it. Try this by setting the tolerance to a really

small number:

```
>> tol = 10^(-100)
```

then run the program again for $f(x) = x^3 - 5$. (You can use **Ctrl-c** to stop a program when it is stuck.)

One way to avoid an infinite loop is add a counter variable, say `i` and a maximum number of iterations to the programs. Using the `while` statement, this can be accomplished as:

```
function x = mynewtontol(f,f1,x0,tol)
    % Solves f(x) = 0 using Newton's method until |f(x)| < tol.
    %     Safety stop after 1000 iterations
    % Inputs: f -- the function
    %         f1 -- it's derivative
    %         x0 -- starting guess, a number
    %         tol -- desired tolerance, runs until |f(x)|<tol
    % Output: x -- the approximate solution
    x = x0;      % set x equal to the initial guess x0.
    i=0;         % set counter to zero
    y = f(x);
    while abs(y) > tol & i < 1000
        % Do until the tolerance is reached or max iter.
        x = x - y/f1(x)    % Newton's formula
        y = f(x)
        i = i+1;           % increment counter
    end
end
```

Exercises

4.1 In Calculus we learn that a geometric series has an exact sum

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r},$$

provided that $|r| < 1$. For instance, if $r = .5$ then the sum is exactly 2. Below is a script program that lacks one line as written. Put in the missing command and then use the program to verify the result above. How many steps does it take? How close is the answer to 2?

```
% Computes a geometric series until it seems to converge
format long
format compact
r = .5;
Snew = 0;           % start sum at 0
Sold = -1;          % set Sold to trick while the first time
i = 0;              % count iterations
while Snew > Sold    % is the sum still changing?
    Sold = Snew;     % save previous value to compare to
    Snew = Snew + r^i;
    i=i+1;
Snew                % prints the final value.
i                   % prints the # of iterations.
```

Add a line at the end of the program to compute the error of **Snew** (with respect to the exact value from the formula above). Run the script for $r = 0.9, 0.99, 0.999, 0.9999, 0.99999, \text{ and } 0.999999$. Turn in your program and a table showing the approximate sum, number of iterations needed and the error for each r .

4.2 Modify your program from exercise 3.3 to compute the total distance traveled by the ball while its bounces are at least 0.01 millimeter high. Use a **while** loop (instead of **for**) to decide when to stop summing (do not use a **for** loop or trial and error). Turn in your modified program and a brief summary of the results.

Lecture 5

The Bisection Method and Locating Roots

Bisecting and the `if ... else ... end` statement

Recall the bisection method. Suppose that $c = f(a) < 0$ and $d = f(b) > 0$. If f is continuous, then obviously it must be zero at some x^* between a and b . The bisection method then consists of looking half way between a and b for the zero of f , i.e. let $x = (a + b)/2$ and evaluate $y = f(x)$. Unless this is zero, then from the signs of c , d and y we can decide which new interval to subdivide. In particular, if c and y have the same sign, then $[x, b]$ should be the new interval, but if c and y have different signs, then $[a, x]$ should be the new interval. (See Figure 5.1.)

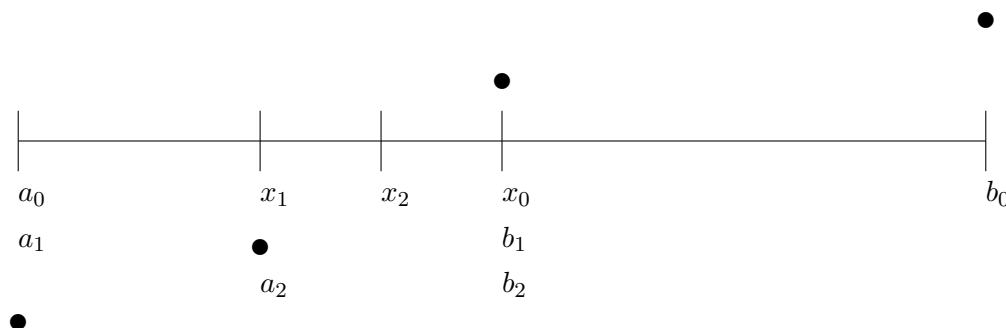


Figure 5.1: The bisection method.

Deciding to do different things in different situations in a program is called *flow control*. The most common way to do this is the `if ... else ... end` statement which is an extension of the `if ... end` statement we have used already.

Bounding the Error

One good thing about the bisection method, which we do not have with Newton's method, is that we always know that the actual solution x^* is inside the current interval $[a, b]$, since $f(a)$ and $f(b)$ have different signs. This allows us to be sure about what the maximum error can be. Precisely, the error is always less than half of the length of the current interval $[a, b]$, i.e.

$$\{\text{Absolute Error}\} = |x - x^*| < (b - a)/2,$$

where x is the center point between the current a and b .

The following function program (available to download as `mybisection.m`) does n iterations of the bisection method and returns not only the final value, but also the maximum possible error:

```
function [x e] = mybisection(f,a,b,n)
% function [x e] = mybisection(f,a,b,n)
% Does n iterations of the bisection method for a function f
% Inputs: f -- a function
%         a,b -- left and right edges of the interval
%         n -- the number of bisections to do.
% Outputs: x -- the estimated solution of f(x) = 0
%         e -- an upper bound on the error

% evaluate at the ends and make sure there is a sign change
c = f(a); d = f(b);
if c*d > 0.0
    error('Function has same sign at both endpoints.')
end
for i = 1:n
    % find the middle and evaluate there
    x = (a + b)/2;
    y = f(x);
    if y == 0.0      % solved the equation exactly
        a = x;
        b = x;
        break       % jumps out of the for loop
    end
    % decide which half to keep, so that the signs at the ends differ
    if c*y < 0
        b=x;
    else
        a=x;
    end
end
% set the best estimate for x and the error bound
x = (a + b)/2;
e = (b-a)/2;
end
```

Another important aspect of bisection is that it always works. We saw that Newton's method can fail to converge to x^* if x_0 is not close enough to x^* . In contrast, the current interval $[a, b]$ will always be decreased by a factor of 2 at each step and so it will always eventually shrink down as small as you wish.

Locating the roots (if any)

The bisection method and Newton's method are both used to obtain closer and closer approximations of a solution, but both require starting places. The bisection method requires two points a and b that have a root between them, and Newton's method requires one point x_0 which is reasonably close to a root. How do you come up with these starting points? It depends. If you are solving an equation once, then the best thing to

do first is to just graph it. From an accurate graph you can see approximately where the graph crosses zero.

There are other situations where you are not just solving an equation once, but have to solve the same equation many times, but with different coefficients. This happens often when you are developing software for a specific application. In this situation the first thing you want to take advantage of is the natural domain of the problem, i.e. on what interval is a solution physically reasonable. If that is known, then it is easy to get close to the root by simply checking the sign of the function at a fixed number of points inside the interval. Whenever the sign changes from one point to the next, there is a root between those points. The following program will look for the roots of a function f on a specified interval $[a_0, b_0]$.

```
function [a,b] = myrootfind(f,a0,b0)
% function [a,b] = myrootfind(f,a0,b0)
% Looks for subintervals where the function changes sign
% Inputs: f -- a function
%          a0 -- the left edge of the domain
%          b0 -- the right edge of the domain
% Outputs: a -- an array, giving the left edges of subintervals
%           on which f changes sign
%           b -- an array, giving the right edges of the subintervals
n = 1001;    % number of test points to use
a = [];      % start empty array
b = [];
% split the interval into n-1 intervals and evaluate at the break points
x = linspace(a0,b0,n);
y = f(x);
% loop through the intervals
for i = 1:(n-1)
    if y(i)*y(i+1) <= 0 % The sign changed, record it
        a = [a x(i)];
        b = [b x(i+1)];
    end
end
if size(a,1) == 0
    warning('no roots were found')
end
end
```

To see this program in action try the following:

```
f = @(x) sin(x)-2*x.^4+0.5
x = -1:.01:1;
y = f(x);
plot(x,y) % see that there are two roots
[a,b] = myrootfind(f,-1,1) % observe that it finds two roots
```

The final situation is writing a program that will look for roots with no given information. This is a difficult problem and one that is not often encountered in actual applications.

Once a root has been located on an interval $[a, b]$, these a and b can serve as the beginning points for the bisection and secant methods (see the next section). For Newton's method one would want to choose x_0

between a and b . One obvious choice would be to let x_0 be the bisector of a and b , i.e. $x_0 = (a + b)/2$. An even better choice would be to use the secant method to choose x_0 .

Exercises

- 5.1 Modify `mybisection` to solve until the absolute error is bounded by a given tolerance. Use a `while` loop to do this. Run your program on the function $f(x) = 2x^3 + 3x - 1$ with starting interval $[0, 1]$ and a tolerance of 10^{-8} . How many steps does the program use to achieve this tolerance? (You can count the steps by adding 1 to a counting variable `i` in the loop of the program.) How big is the final residual $f(x)$? Turn in your program and a brief summary of the results.
- 5.2 Perform 3 iterations of the bisection method on the function $f(x) = x^2 - 5$, with starting interval $[1, 3]$. (By hand, but use a calculator.) Calculate the errors and percentage errors of x_0 , x_1 , x_2 , and x_3 . Compare the errors with those in exercise 3.2.

Lecture 6

Secant Methods

In this lecture we introduce two additional methods to find numerical solutions of the equation $f(x) = 0$. Both of these methods are based on approximating the function by secant lines just as Newton's method was based on approximating the function by tangent lines.

The Secant Method

The secant method requires two initial approximations x_0 and x_1 , preferably both reasonably close to the solution x^* . From x_0 and x_1 we can determine that the points $(x_0, y_0 = f(x_0))$ and $(x_1, y_1 = f(x_1))$ both lie on the graph of f . Connecting these points gives the (secant) line

$$y - y_1 = \frac{y_1 - y_0}{x_1 - x_0}(x - x_1).$$

Since we want $f(x) = 0$, we set $y = 0$, solve for x , and use that as our next approximation. Repeating this process gives us the iteration

$$x_{i+1} = x_i - \frac{x_i - x_{i-1}}{y_i - y_{i-1}}y_i \tag{6.1}$$

with $y_i = f(x_i)$. See Figure 6.1 for an illustration.

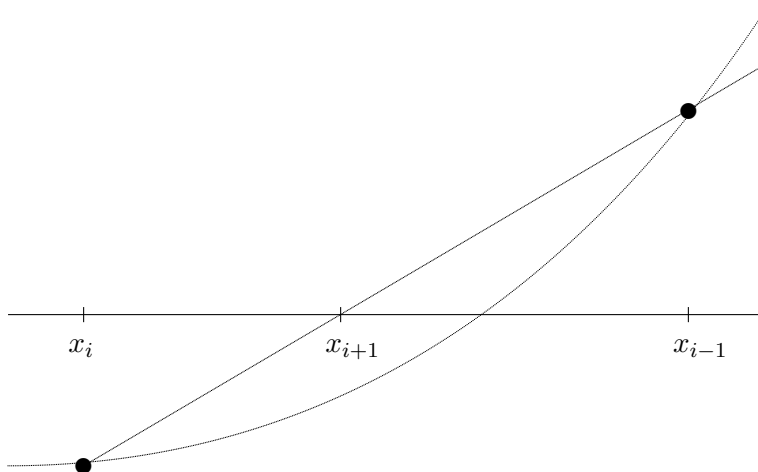


Figure 6.1: The secant method in the case where the root is bracketed.

For example, suppose $f(x) = x^4 - 5$, which has a solution $x^* = \sqrt[4]{5} \approx 1.5$. Choose $x_0 = 1$ and $x_1 = 2$ as initial approximations. Next we have that $y_0 = f(1) = -4$ and $y_1 = f(2) = 11$. We may then calculate x_2 from the formula (6.1):

$$x_2 = 2 - \frac{2-1}{11-(-4)} 11 = \frac{19}{15} \approx 1.2666\dots$$

Plugging $x_2 = 19/15$ into $f(x)$ we obtain $y_2 = f(19/15) \approx -2.425758\dots$. In the next step we would use $x_1 = 2$ and $x_2 = 19/15$ in the formula (6.1) to find x_3 and so on.

Below is a program for the secant method (available to download as `mysecant.m`). Notice that it requires two input guesses x_0 and x_1 , but it does not require the derivative to be input.

```
function x = mysecant(f,x0,x1,n)
% Solves f(x) = 0 by doing n steps of the secant method
% starting with x0 and x1.
% Inputs: f -- the function
%         x0 -- starting guess, a number
%         x1 -- second starting guess
%         n -- the number of steps to do
% Output: x -- the approximate solution
y0 = f(x0);
y1 = f(x1);
for i = 1:n
    % Do n times
    x = x1 - (x1-x0)*y1/(y1-y0) % secant formula.
    y=f(x) % y value at the new approximate solution.
    % Move numbers to get ready for the next step
    x0=x1;
    y0=y1;
    x1=x;
    y1=y;
end
end
```

The *Regula Falsi* (False Position) Method

The *Regula Falsi* method is a combination of the secant method and bisection method. As in the bisection method, we have to start with two approximations a and b for which $f(a)$ and $f(b)$ have different signs. As in the secant method, we follow the secant line to get a new approximation, which gives a formula similar to (6.1),

$$x = b - \frac{b-a}{f(b)-f(a)} f(b).$$

Then, as in the bisection method, we check the sign of $f(x)$; if it is the same as the sign of $f(a)$ then x becomes the new a and otherwise let x becomes the new b . Note that in general either $a \rightarrow x^*$ or $b \rightarrow x^*$ but not both, so $b-a \not\rightarrow 0$. For example, for the function in Figure 6.1, $a \rightarrow x^*$ but b would never move.

Convergence

If we can begin with a good choice x_0 , then Newton's method will converge to x^* rapidly. The secant method is a little slower than Newton's method and the *Regula Falsi* method is slightly slower than that. However, both are still much faster than the bisection method.

If we do not have a good starting point or interval, then the secant method, just like Newton's method, can fail altogether. The *Regula Falsi* method, just like the bisection method, always works because it keeps the solution inside a definite interval.

Simulations and Experiments

Although Newton's method converges faster than any other method, there are contexts when it is not convenient, or even impossible. One obvious situation is when it is difficult to calculate a formula for $f'(x)$ even though one knows the formula for $f(x)$. This is often the case when $f(x)$ is not defined explicitly, but implicitly. There are other situations, which are very common in engineering and science, where even a formula for $f(x)$ is not known. This happens when $f(x)$ is the result of experiment or simulation rather than a formula. In such situations, the secant method is usually the best choice.

Exercises

- 6.1 Perform 3 iterations of the secant method on the function $f(x) = x^2 - 5$, with starting points $x_{-1} = 1$ and $x_0 = 3$. (By hand, but use a calculator.) Calculate the errors and percentage errors of x_1 , x_2 , and x_3 . Compare the errors with those in exercise 3.2 and 5.2.
- 6.2 Create and graph the function `g = @(x) log(x)+x.^2`. In the plot window, use the Tools menu to zoom in on the root (there is only one) to 2 decimal places. From this determine good starting points `a0` and `b0` and use the program `mysecant.m` to approximate the root x^* to 15 decimal places. Turn in your zoomed-in plot and your approximation.
- 6.3 Modify the program `mysecant.m` to iterate until the absolute value of the residual is less than a given tolerance. (Let `tol` be an input instead of `n`.) Modify the comments appropriately. Test program on the two functions in the exercises above with `tol = 10-10`. Turn in the results and the program.

Lecture 7

Symbolic Computations

The focus of this course is on numerical computations, i.e. calculations, usually approximations, with floating point numbers. However, MATLAB can also do *symbolic* computations, which means exact calculations using symbols as in Algebra or Calculus.

Note: To do symbolic computations in MATLAB one must have the Symbolic Toolbox.

Defining functions and basic operations

Before doing any symbolic computation, one must declare the variables used to be symbolic:

```
>> syms x y
```

An expression representing a function may be defined by simply typing the formula:

```
>> f = cos(x) + 3*x^2
```

Note that coefficients must be multiplied using `*`. To find specific values, you must use the command `subs`:

```
>> subs(f, pi)
```

This command stands for *substitute*, it substitutes π for x in the formula for f . If we define another function:

```
>> g = exp(-y^2)
```

then we can compose the functions:

```
>> h = compose(g, f)
```

i.e. $h(x) = g(f(x))$. We can also form new functions by multiplying:

```
>> yg = y*g
```

We can do simple calculus operations, like differentiation:

```
>> f1 = diff(f)
```

```
>> h1 = diff(h)
```

indefinite integrals (antiderivatives):

```
>> F = int(f)
```

and definite integrals:

```
>> int(f,0,2*pi)
```

To change a symbolic answer into a numerical answer, use the `double` command, which stands for *double precision* (not times 2):

```
>> double(ans)
```

Note that some antiderivatives cannot be found in terms of elementary functions; for some of these the antiderivative can be expressed in terms of special functions:

```
>> G = int(g)
```

and for others MATLAB does the best it can:

```
>> int(h)
```

For definite integrals that cannot be evaluated exactly, MATLAB does nothing and prints a warning:

```
>> int(h,0,1)
```

We will see later that even functions that don't have an antiderivative can be integrated numerically. You can change the last answer to a numerical answer using:

```
>> double(ans)
```

Plotting a symbolic function can be done as follows:

```
>> fplot(f)
```

or the domain can be specified:

```
>> fplot(g,[-10,10])
```

```
>> fplot(g,[-2,2])
```

It is important to keep in mind that even though we have defined our variables to be symbolic variables, plotting can only plot a finite set of points. For instance:

```
>> fplot(cos(x^5))
```

will produce a plot with some small mistakes, because it does not plot enough points.

Functions of Two Variables and Partial Derivatives

Since $f = \cos(x) + 3x^2$ and $g = \exp(-y^2)$ are functions of different variables, their product is a function of two variables:

$$k(x, y) = f(x)g(y) = (\cos(x) + 3x^2) \exp(-y^2)$$

In MATLAB, set

```
>> k = f*g
```

To evaluate, we need to specify which value goes in which variable:

```
>> subs(k, [x, y], [0, 1])
```

To plot a symbolic function of two variables use:

```
>> ezsurf(k)
```

If we want to differentiate a function with more than one variable in MATLAB, we must specify which variable we are differentiating with respect to:

```
>> k1x = diff(k, x)
```

Here we have differentiated with respect to x . This is called the **partial derivative** with respect to x and we use the symbol $\frac{\partial k}{\partial x}$ to denote it. There is also a partial derivative with respect to y , i.e. $\frac{\partial k}{\partial y}$:

```
>> k1y = diff(k, y)
```

To calculate a partial derivative by hand you differentiate as normal with respect to one variable, but treat the other variables as if they were constants. For example if $f(x, y) = xy^2$, then

$$\frac{\partial f}{\partial x} = y^2 \quad \text{and} \quad \frac{\partial f}{\partial y} = 2xy.$$

Other useful symbolic operations

MATLAB allows you to do simple algebra. For instance:

```
>> poly = (x - 3)^5
>> polyex = expand(poly)
>> polysi = simplify(polyex)
```

To find the symbolic solutions of an equation, $f(x) = 0$, use:

```
>> solve(f)
>> solve(g)
>> solve(polyex)
```

Another useful property of symbolic functions is that you can substitute numerical vectors for the variables:

```
>> X = 2:0.1:4;
>> Y = subs(polyex, X);
>> plot(X, Y)
```

New Notation

MATLAB recently expanded its notation for symbolic computations:

```
>> syms x
>> f(x) = x^2;
>> f(2)
```

The function must be given as $f(x)$. If given as $f = x^2$ this will not work. Then we have to use

```
>> subs(f,2)
```

It works for multivariable functions also:

```
>> syms x y a b
>> f(x,y) = x + 2*y;
>> f(a,b)
```

Exercises

- 7.1 Starting from `mynewton` write a well-commented **function** program `mysymnewton` that takes as its input a symbolic function f and the ordinary variables x_0 and n . Let the program take the symbolic derivative f' . You will need to use the command `subs` to obtain values of $f(x)$ and $f'(x)$. Test it on $f(x) = x^3 - 4$ starting with $x_0 = 2$. Turn in the program and a brief summary of the results.
- 7.2 Find a *complicated* function in an engineering or science textbook or website. Make a well-commented **script** program that defines a symbolic version of this function and takes its derivative and indefinite integral symbolically (if possible). Plot the function on the domain that is relevant for the application. In the comments of the script describe what the function is and properly reference where you got it. Turn in your script and the plot.
- 7.3 Calculate all the partial derivative for the functions below. Do them by hand, then check them using the symbolic toolbox.

$$(a) f(x, y) = \sqrt{xy} + x^2 \sin^2 y \qquad (b) g(u, v) = \frac{uv}{u + v}.$$

Review of Part I

Methods and Formulas

Solving equations numerically:

$f(x) = 0$ — an equation we wish to solve.

x^* — a true solution.

x_0 — starting approximation.

x_n — approximation after n steps.

$e_n = x_n - x^*$ — error of n -th step.

$r_n = y_n = f(x_n)$ — residual at step n . Often $|r_n|$ is sufficient.

Newton's method:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Bisection method:

$f(a)$ and $f(b)$ must have different signs.

$x = (a + b)/2$

Choose $a = x$ or $b = x$, depending on signs.

x^* is always inside $[a, b]$.

$e < (b - a)/2$, current maximum error.

Secant method:

$$x_{i+1} = x_i - \frac{x_i - x_{i-1}}{y_i - y_{i-1}} y_i$$

Regula Falsi

- a hybrid between secant and bisection methods.

$$x = b - \frac{b - a}{f(b) - f(a)} f(b)$$

Choose $a = x$ or $b = x$, depending on signs.

Convergence:

Bisection is very slow.

Newton is very fast.

Secant methods are intermediate in speed.

Bisection and Regula Falsi never fail to converge.

Newton and Secant can fail if x_0 is not close to x^* .

Locating roots:

Use knowledge of the problem to begin with a reasonable domain.

Systematically search for sign changes of $f(x)$.

Choose x_0 between sign changes using bisection or secant.

Usage:

For Newton's method one must have formulas for $f(x)$ and $f'(x)$.

Secant methods are better for experiments and simulations.

Bisection and *Regula Falsi* are slower, but keep the root within the current bounds.

Matlab**Commands:**

`v = [0 1 2 3]` Make a row vector.
`u = [0; 1; 2; 3]` Make a column vector.
`w = v'` Transpose: row vector $\leftrightarrow$ column vector
`x = linspace(0,1,11)` Make an evenly spaced vector of length 11.
`x = -1:.1:1` Make an evenly spaced vector, with increments 0.1.
`y = x.^2` Square all entries.
`plot(x,y)` plot y vs. x.
`f = @(x) 2*x.^2 - 3*x + 1` Make an anonymous function.
`y = f(x)` A function can act on a vector.
`plot(x,y,'*', 'red')` A plot with options.
`Control-c` Stops a computation.

Program structures:

`for ... end` example:

```
for i=1:20
    S = S + i;
end
```

`if ... end` example:

```

if y == 0
    disp('An exact solution has been found')
end

```

while ... end example:

```

while i <= 20
    S = S + i;
    i = i + 1;
end

```

if ... else ... end example:

```

if c*y>0
    a = x;
else
    b = x;
end

```

Function Programs:

- Begin with the word **function**.
- There are inputs and outputs.
- The outputs, name of the function and the inputs must appear in the first line.
i.e. `function x = mynewton(f,x0,n)`
- The body of the program must assign values to the outputs.
- Internal variables are not visible outside the function.
- A function program may not use variables in the current workspace unless they are inputs.

Script Programs:

- There are no inputs and outputs.
- A script program may use, create, change and even delete variables in the current workspace.

Symbolic:

```

syms x y
f = 2*x^2 - sqrt(3*x)
subs(f,sym(pi))
double(ans)
g = log(abs(y)) ..... MATLAB uses log for natural logarithm.
h(x) = compose(g,f)

```

```

k(x,y) = f*g
fplot(f)
fplot(g,-10,10)
ezsurf(k)
f1 = diff(f,'x')
F = int(f,'x') ..... indefinite integral (antiderivative)
int(f,0,2*pi) ..... definite integral
poly = x*(x - 3)*(x-2)*(x-1)*(x+1)
polyex = expand(poly)
polysi = simple(polyex)
solve(f)
solve(g)
solve(polyex)

```


Part II

Linear Algebra

©Copyright, Todd Young and Martin Mohlenkamp, Department of Mathematics, Ohio University, 2026

Lecture 8

Matrices and Matrix Operations in Matlab

You should review the vector operations in Lecture 1.

Matrix operations

Recall how to multiply a matrix A times a vector $\mathbf{v}$:

$$A\mathbf{v} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} -1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \cdot (-1) + 2 \cdot 2 \\ 3 \cdot (-1) + 4 \cdot 2 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}.$$

This is a special case of matrix multiplication. To multiply two matrices, A and B you proceed as follows:

$$AB = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} -1 & -2 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} -1+4 & -2+2 \\ -3+8 & -6+4 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 5 & -2 \end{pmatrix}.$$

Here both A and B are 2×2 matrices. Matrices can be multiplied together in this way provided that the number of columns of A match the number of rows of B . We always list the size of a matrix by rows, then columns, so a 3×5 matrix would have 3 rows and 5 columns. So, if A is $m \times n$ and B is $p \times q$, then we can multiply AB if and only if $n = p$. A column vector can be thought of as a $p \times 1$ matrix and a row vector as a $1 \times q$ matrix. Unless otherwise specified we will assume a vector $\mathbf{v}$ to be a column vector and so $A\mathbf{v}$ makes sense as long as the number of columns of A matches the number of entries in $\mathbf{v}$.

Printing matrices on the screen takes up a lot of space, so you may want to use

```
>> format compact
```

Enter a matrix into Matlab either as

```
>> A = [ 1 3 -2 5 ; -1 -1 5 4 ; 0 1 -9 0]
```

or

```
>> A = [1,3,-2,5; -1,-1,5,4; 0,1,-9,0]
```

Also enter a vector $\mathbf{u}$:

```
>> u = [ 1 2 3 4]'
```

To multiply a matrix times a vector $A\mathbf{u}$ use $*$:

```
>> A*u
```

Since A is 3 by 4 and $\mathbf{u}$ is 4 by 1 this multiplication is valid and the result is a 3 by 1 vector.

Now enter another matrix B using

```
>> B = [3 2 1; 7 6 5; 4 3 2]
```

You can multiply B times A with

```
>> B*A
```

but A times B is not defined and

```
>> A*B
```

will result in an error message.

You can multiply a matrix by a scalar:

```
>> 2*A
```

Adding matrices $A + A$ will give the same result:

```
>> A + A
```

You can even add a number to a matrix:

```
>> A + 3      % add 3 to every entry of A
```

Component-wise operations

Just as for vectors, adding a `'.'` before `'*'`, `'/'`, or `'^'` produces entry-wise multiplication, division and exponentiation. If you enter

```
>> B*B
```

the result will be BB , i.e. matrix multiplication of B times itself. But, if you enter

```
>> B.*B
```

the entries of the resulting matrix will contain the squares of the same entries of B . Similarly if you want B multiplied by itself 3 times then enter

```
>> B^3
```

but, if you want to cube all the entries of B then enter

```
>> B.^3
```

Note that $B*B$ and B^3 only make sense if B is square, but $B.*B$ and $B.^3$ make sense for any size matrix.

The identity matrix and the inverse of a matrix

The $n \times n$ *identity matrix* is a square matrix with ones on the diagonal and zeros everywhere else. It is called the identity because it plays the same role that 1 plays in multiplication, i.e.

$$AI = A, \quad IA = A, \quad I\mathbf{v} = \mathbf{v}$$

for any matrix A or vector $\mathbf{v}$ where the sizes match. An identity matrix in MATLAB is produced by the command

```
>> I = eye(3)
```

A square matrix A can have an *inverse* which is denoted by A^{-1} . The definition of the inverse is that

$$AA^{-1} = I \quad \text{and} \quad A^{-1}A = I.$$

In theory an inverse is very important, because if you have an equation

$$A\mathbf{x} = \mathbf{b}$$

where A and $\mathbf{b}$ are known and $\mathbf{x}$ is unknown (as we will see, such problems are very common and important) then the theoretical solution is

$$\mathbf{x} = A^{-1}\mathbf{b}.$$

We will see later that this is not a practical way to solve an equation, and A^{-1} is only important for the purpose of derivations. In MATLAB we can calculate a matrix's inverse very conveniently:

```
>> C = randn(5,5)
>> inv(C)
```

However, not all square matrices have inverses:

```
>> D = ones(5,5)
>> inv(D)
```

The “Norm” of a matrix

For a vector, the “norm” means the same thing as the length (geometrically, not the number of entries). Another way to think of it is how far the vector is from being the zero vector. We want to measure a matrix in much the same way and the *norm* is such a quantity. The usual definition of the norm of a matrix is

Definition 1 Suppose A is a $m \times n$ matrix. The norm of A is

$$\|A\| \equiv \max_{\|\mathbf{v}\|=1} \|A\mathbf{v}\|.$$

The maximum in the definition is taken over all vectors with length 1 (unit vectors), so the definition means the largest factor that the matrix stretches (or shrinks) a unit vector. This definition seems cumbersome at first, but it turns out to be the best one. For example, with this definition we have the following inequality for any vector $\mathbf{v}$:

$$\|A\mathbf{v}\| \leq \|A\|\|\mathbf{v}\|.$$

In MATLAB the norm of a matrix is obtained by the command

```
>> norm(A)
```

For instance the norm of an identity matrix is 1:

```
>> norm(eye(100))
```

and the norm of a zero matrix is 0:

```
>> norm(zeros(50,50))
```

For a matrix the norm defined above and calculated by MATLAB is not the square root of the sum of the square of its entries. That quantity is called the *Frobenius norm*, which is also sometimes useful, but we will not need it.

Some other useful commands

```
C = rand(5,5) ..... random matrix with uniform distribution in [0,1].
size(C) ..... gives the dimensions ( $m \times n$ ) of  $C$ .
det(C) ..... the determinant of the matrix.
max(C) ..... the maximum of each column.
min(C) ..... the minimum in each column.
sum(C) ..... sums each column.
mean(C) ..... the average of each column.
diag(C) ..... just the diagonal elements.
C' ..... tranpose the matrix.
```

In addition to `ones`, `eye`, `zeros`, `rand` and `randn`, MATLAB has several other commands that automatically produce special matrices:

```
hilb(6)
pascal(5)
```

Exercises

8.1 Enter the matrix A by

```
>> A = [3 2 1; 6 5 4; 9 8 7]
```

and also the matrix

$$B = \begin{bmatrix} 7 & 8 & 9 \\ 4 & 5 & 6 \\ 1 & 2 & 3 \end{bmatrix}.$$

Find (a) $A*A$, (b) A^2 , (c) $A.^2$, (d) $A.*B$, (e) $A*B$. Turn in the output.

8.2 By hand, calculate $A\mathbf{v}$, AB , and BA for:

$$A = \begin{pmatrix} 2 & 4 & -1 \\ -2 & 1 & 9 \\ -1 & -1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & -1 & -1 \\ 1 & 0 & 2 \\ -1 & -2 & 0 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} 3 \\ 1 \\ -1 \end{pmatrix}.$$

Check the results using MATLAB. Think about how fast computers are. Turn in your hand work.

8.3 Write a well-commented MATLAB **function** program `myinvcheck` that

- makes a $n \times n$ random matrix (normally distributed, $A = \text{randn}(n,n)$),
- calculates its inverse ($B = \text{inv}(A)$),
- multiplies the two back together,
- calculates the residual (difference between AB and $\text{eye}(n)$), and
- returns the scalar residual (norm of the difference).

Turn in your program.

8.4 Write a well-commented MATLAB **script** program `myinvcheckplot` that calls `myinvcheck` for $n = 10, 20, 40, \dots, 2^i 10$ for i as large as possible, records the results of each trial, and plots the scalar residual versus n using a log plot. (See `help loglog`.)

What happens to the scalar residual as n gets big? Turn in the program, the plot, and a very brief report on the results of your experiments. (Do not print any large random matrices.)

Lecture 9

Introduction to Linear Systems

How linear systems occur

Linear systems of equations naturally occur in many places in engineering, such as structural analysis, dynamics and electric circuits. Computers have made it possible to quickly and accurately solve larger and larger systems of equations. Not only has this allowed engineers to handle more and more complex problems where linear systems naturally occur, but has also prompted engineers to use linear systems to solve problems where they do not naturally occur such as thermodynamics, internal stress-strain analysis, fluids and chemical processes. It has become standard practice in many areas to analyze a problem by transforming it into a linear systems of equations and then solving those equation by computer. In this way, computers have made linear systems of equations the most frequently used tool in modern engineering.

In Figure 9.1 we show a truss with equilateral triangles. In Statics you may use the “method of joints” to write equations for each node of the truss¹. This set of equations is an example of a linear system. Making the approximation $\sqrt{3}/2 \approx .8660$, the equations for this truss are

$$\begin{aligned} .5 T_1 + T_2 &= R_1 = f_1 \\ .866 T_1 &= -R_2 = -.433 f_1 - .5 f_2 \\ -.5 T_1 + .5 T_3 + T_4 &= -f_1 \\ .866 T_1 + .866 T_3 &= 0 \\ -T_2 - .5 T_3 + .5 T_5 + T_6 &= 0 \\ .866 T_3 + .866 T_5 &= f_2 \\ -T_4 - .5 T_5 + .5 T_7 &= 0, \end{aligned} \tag{9.1}$$

where T_i represents the tension in the i -th member of the truss.

You could solve this system by hand with a little time and patience; systematically eliminating variables and substituting. Obviously, it would be a lot better to put the equations on a computer and let the computer solve it. In the next few lectures we will learn how to use a computer effectively to solve linear systems. The first key to dealing with linear systems is to realize that they are equivalent to matrices, which contain numbers, not variables.

As we discuss various aspects of matrices, we wish to keep in mind that the matrices that come up in engineering systems are *really large*. It is not unusual in real engineering to use matrices whose dimensions are in the thousands! It is frequently the case that a method that is fine for a 2×2 or 3×3 matrix is totally inappropriate for a 2000×2000 matrix. We thus want to emphasize methods that work for large matrices.

¹See <http://en.wikipedia.org/wiki/Truss> or <http://en.wikibooks.org/wiki/Statics> for reference.

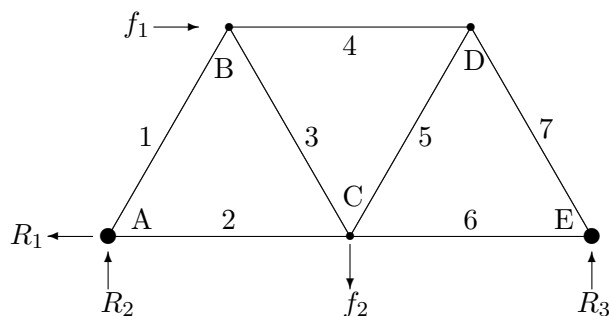


Figure 9.1: An equilateral truss. Joints or nodes are labeled alphabetically, $A, B, \dots$ and Members (edges) are labeled numerically: $1, 2, \dots$. The forces f_1 and f_2 are applied loads and R_1, R_2 and R_3 are reaction forces applied by the supports.

Linear systems are equivalent to matrix equations

The system of linear equations

$$\begin{aligned}x_1 - 2x_2 + 3x_3 &= 4 \\2x_1 - 5x_2 + 12x_3 &= 15 \\2x_2 - 10x_3 &= -10\end{aligned}$$

is equivalent to the matrix equation

$$\begin{pmatrix} 1 & -2 & 3 \\ 2 & -5 & 12 \\ 0 & 2 & -10 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 15 \\ -10 \end{pmatrix},$$

which is equivalent to the *augmented matrix*

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 2 & -5 & 12 & 15 \\ 0 & 2 & -10 & -10 \end{array} \right).$$

The advantage of the augmented matrix, is that it contains only numbers, not variables. The reason this is better is because computers are much better in dealing with numbers than variables. To solve this system, the main steps are called *Gaussian elimination* and *back substitution*.

The augmented matrix for the equilateral truss equations (9.1) is given by

$$\left(\begin{array}{ccccccc|c} .5 & 1 & 0 & 0 & 0 & 0 & 0 & f_1 \\ .866 & 0 & 0 & 0 & 0 & 0 & 0 & -.433f_1 - .5f_2 \\ -.5 & 0 & .5 & 1 & 0 & 0 & 0 & -f_1 \\ .866 & 0 & .866 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -.5 & 0 & .5 & 1 & 0 & 0 \\ 0 & 0 & .866 & 0 & .866 & 0 & 0 & f_2 \\ 0 & 0 & 0 & -1 & -.5 & 0 & .5 & 0 \end{array} \right). \quad (9.2)$$

Notice that a lot of the entries are 0. Matrices like this, called *sparse*, are common in applications and there are methods specifically designed to efficiently handle sparse matrices.

Triangular matrices and back substitution

Consider a linear system whose augmented matrix happens to be

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 0 & -1 & 6 & 7 \\ 0 & 0 & 2 & 4 \end{array} \right). \quad (9.3)$$

Recall that each row represents an equation and each column a variable. The last row represents the equation $2x_3 = 4$. The equation is easily solved, i.e. $x_3 = 2$. The second row represents the equation $-x_2 + 6x_3 = 7$, but since we know $x_3 = 2$, this simplifies to: $-x_2 + 12 = 7$. This is easily solved, giving $x_2 = 5$. Finally, since we know x_2 and x_3 , the first row simplifies to: $x_1 - 10 + 6 = 4$. Thus we have $x_1 = 8$ and so we know the whole solution vector: $\mathbf{x} = \langle 8, 5, 2 \rangle$. The process we just did is called *back substitution*, which is both efficient and easily programmed. The property that made it possible to solve the system so easily is that A in this case is *upper triangular*. In the next section we show an efficient way to transform an augmented matrix into an upper triangular matrix.

Gaussian Elimination

Consider the matrix

$$A = \left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 2 & -5 & 12 & 15 \\ 0 & 2 & -10 & -10 \end{array} \right).$$

The first step of Gaussian elimination is to get rid of the 2 in the (2,1) position by subtracting 2 times the first row from the second row, i.e. (new 2nd = old 2nd - (2) 1st). We can do this because it is essentially the same as adding equations, which is a valid algebraic operation. This leads to

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 0 & -1 & 6 & 7 \\ 0 & 2 & -10 & -10 \end{array} \right).$$

There is already a zero in the lower left corner, so we don't need to eliminate anything there. To eliminate the third row, second column, we need to subtract -2 times the second row from the third row, (new 3rd = old 3rd - (-2) 2nd), to obtain

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 0 & -1 & 6 & 7 \\ 0 & 0 & 2 & 4 \end{array} \right).$$

This is now just exactly the matrix in equation (9.3), which we can now solve by back substitution.

Matlab's matrix solve command

In MATLAB the standard way to solve a system $A\mathbf{x} = \mathbf{b}$ is by the command

```
>> x = A \ b
```

This syntax is meant to suggest dividing by A from the left as in

$$A\mathbf{x} = \mathbf{b} \quad \Leftrightarrow \quad A \backslash A\mathbf{x} = A \backslash \mathbf{b} \quad \Leftrightarrow \quad \mathbf{x} = A \backslash \mathbf{b}.$$

Such division is not meaningful mathematically, but it helps for remembering the syntax.

This command carries out Gaussian elimination and back substitution. We can do the above computations as follows:

```
>> A = [1 -2 3 ; 2 -5 12 ; 0 2 -10]
>> b = [4 15 -10] '
>> x = A \ b
```

Next, use the MATLAB commands above to solve $A\mathbf{x} = \mathbf{b}$ when the augmented matrix for the system is

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{array} \right),$$

by entering

```
>> x1 = A \ b
```

Check the result by entering

```
>> A*x1 - b
```

You will see that the resulting answer satisfies the equation exactly. Next try solving using the inverse of A :

```
>> x2 = inv(A)*b
```

This answer can be seen to be inaccurate by checking

```
>> A*x2 - b
```

Thus we see one of the reasons why the inverse is never used for actual computations, only for theory.

Exercises

9.1 Set $f_1 = 1000N$ and $f_2 = 5000N$ in the equations (9.1) for the equilateral truss. Input the coefficient matrix A and the right hand side vector b in (9.2) into MATLAB. Solve the system using the command `\` to find the tension in each member of the truss. Save the matrix A as **A_equil_truss** and keep it for later use. (Enter `save A_equil_truss A`.) Print out and turn in A , $\mathbf{b}$ and the solution $\mathbf{x}$.

9.2 Write each system of equations as an augmented matrix. By hand, find the solutions using *Gaussian elimination and back substitution* (i.e. the exact algorithm in this chapter). Check your solutions using MATLAB.

(a)

$$\begin{aligned} x_1 + x_2 &= 2 \\ 4x_1 + 5x_2 &= 10 \end{aligned}$$

(b)

$$\begin{aligned} x_1 + 2x_2 + 3x_3 &= -1 \\ 4x_1 + 7x_2 + 14x_3 &= 3 \\ x_1 + 4x_2 + 4x_3 &= 1 \end{aligned}$$

Lecture 10

Some Facts About Linear Systems

Some inconvenient truths

In the last lecture we learned how to solve a linear system using Matlab. Input the following:

```
>> A = ones(4,4)
>> b = randn(4,1)
>> x = A \ b
```

As you will find, there is no solution to the equation $A\mathbf{x} = \mathbf{b}$. This unfortunate circumstance is mostly the fault of the matrix, A , which is too simple, its columns (and rows) are all the same. Now try

```
>> b = ones(4,1)
>> x = [ 1 0 0 0] '
>> A*x
```

So the system $A\mathbf{x} = \mathbf{b}$ does have a solution. Still unfortunately, that is not the only solution. Try

```
>> x = [ 0 1 0 0] '
>> A*x
```

We see that this x is also a solution. Next try

```
>> x = [ -4 5 2.27 -2.27] '
>> A*x
```

This x is a solution! It is not hard to see that there are endless possibilities for solutions of this equation.

Basic theory

The most basic theoretical fact about linear systems is

Theorem 1 *A linear system $A\mathbf{x} = \mathbf{b}$ may have 0, 1, or infinitely many solutions.*

In most (but not all!) engineering applications we would want to have exactly one solution. The following two theorems tell us exactly when we can and cannot expect this.

Theorem 2 *Suppose A is a square $(n \times n)$ matrix. The following are all equivalent:*

1. The equation $A\mathbf{x} = \mathbf{b}$ has exactly one solution for any $\mathbf{b}$.
2. $\det(A) \neq 0$.
3. A has an inverse.
4. The only solution of $A\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$.
5. The columns of A are linearly independent (as vectors).
6. The rows of A are linearly independent.

If A has these properties then it is called **non-singular**.

On the other hand, a matrix that does not have these properties is called **singular**.

Theorem 3 Suppose A is a square matrix. The following are all equivalent:

1. The equation $A\mathbf{x} = \mathbf{b}$ has 0 or ∞ many solutions depending on $\mathbf{b}$.
2. $\det(A) = 0$.
3. A does not have an inverse.
4. The equation $A\mathbf{x} = \mathbf{0}$ has solutions other than $\mathbf{x} = \mathbf{0}$.
5. The columns of A are linearly dependent as vectors.
6. The rows of A are linearly dependent.

To see how the two theorems work, define two matrices (type in **A1** then scroll up and modify to make **A2**) :

$$A1 = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad A2 = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 8 \end{pmatrix},$$

and two vectors:

$$b1 = \begin{pmatrix} 0 \\ 3 \\ 6 \end{pmatrix}, \quad b2 = \begin{pmatrix} 1 \\ 3 \\ 6 \end{pmatrix}.$$

First calculate the determinants of the matrices:

```
>> det(A1)
>> det(A2)
```

Then attempt to find the inverses:

```
>> inv(A1)
>> inv(A2)
```

Which matrix is singular and which is non-singular? Finally, attempt to solve all the possible equations $A\mathbf{x} = \mathbf{b}$:

```

>> x = A1 \ b1
>> x = A1 \ b2
>> x = A2 \ b1
>> x = A2 \ b2

```

As you can see, equations involving the non-singular matrix have one and only one solution, but equation involving a singular matrix are more complicated.

The Residual

Recall that the residual for an approximate solution x of an equation $f(x) = 0$ is defined as $r = \|f(x)\|$. It is a measure of how close the equation is to being satisfied. For a linear system of equations we define the residual vector of an approximate solution $\mathbf{x}$ by

$$\mathbf{r} = A\mathbf{x} - \mathbf{b}.$$

If the solution vector $\mathbf{x}$ were exactly correct, then $\mathbf{r}$ would be exactly the zero vector. The size (norm) of $\mathbf{r}$ is an indication of how close we have come to solving $A\mathbf{x} = \mathbf{b}$. We will refer to this number as the **scalar residual** or just the **residual** of the approximation solution:

$$r = \|A\mathbf{x} - \mathbf{b}\|. \quad (10.1)$$

Exercises

- 10.1 By hand, find all the solutions (if any) of the following linear system using the augmented matrix and Gaussian elimination (following exactly the algorithm in the notes):

$$\begin{aligned} x_1 + 2x_2 + 3x_3 &= 4, \\ 4x_1 + 5x_2 + 6x_3 &= 10, \\ 7x_1 + 8x_2 + 9x_3 &= 14. \end{aligned}$$

Try solving this system in MATLAB using the command $\mathbf{x} = A \setminus \mathbf{b}$. What happens? Turn in your hand work.

- 10.2 (a) Write a well-commented MATLAB **function** program `mysolvecheck` with input a number n that makes a random $n \times n$ matrix A and a random vector $\mathbf{b}$, solves the linear system $A\mathbf{x} = \mathbf{b}$, calculates the scalar residual $r = \|A\mathbf{x} - \mathbf{b}\|$, and outputs that number as r .
- (b) Write a well-commented MATLAB **script** program that calls `mysolvecheck` 10 times each for $n = 5, 10, 20, 40, 80$, and 160, then records and averages the results and makes a log-log plot of the average r vs. n . Once your program is running correctly, increase the maximum n (by factors of 2) until the program stops running within 5 minutes.

Turn in the plot and the two programs. (Do not print any large random matrices.)

Lecture 11

Accuracy, Condition Numbers and Pivoting

In this lecture we will discuss two separate issues of accuracy in solving linear systems. The first, *pivoting*, is a method that ensures that Gaussian elimination proceeds as accurately as possible. The second, *condition number*, is a measure of how bad a matrix is. We will see that if a matrix has a bad condition number, the solutions are unstable with respect to small changes in data.

The effect of rounding

All computers store numbers as finite strings of binary floating point digits (bits). This limits numbers to a fixed number of significant digits and implies that after even the most basic calculations, rounding must happen.

Consider the following exaggerated example. Suppose that our computer can only store 2 significant digits and it is asked to do Gaussian elimination on

$$\left(\begin{array}{cc|c} .001 & 1 & 3 \\ 1 & 2 & 5 \end{array} \right).$$

Doing the elimination exactly would produce

$$\left(\begin{array}{cc|c} .001 & 1 & 3 \\ 0 & -998 & -2995 \end{array} \right),$$

but rounding to 2 digits, our computer would store this as

$$\left(\begin{array}{cc|c} .001 & 1 & 3 \\ 0 & -1000 & -3000 \end{array} \right).$$

Backsolving this reduced system gives

$$x_1 = 0 \quad \text{and} \quad x_2 = 3.$$

This seems fine until you realize that backsolving the unrounded system gives

$$x_1 = -1 \quad \text{and} \quad x_2 = 3.001.$$

Row Pivoting

A way to fix the problem is to use pivoting, which means to switch rows of the matrix. Since switching rows of the augmented matrix just corresponds to switching the order of the equations, no harm is done:

$$\left(\begin{array}{cc|c} 1 & 2 & 5 \\ .001 & 1 & 3 \end{array} \right).$$

Exact elimination would produce

$$\left(\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & .998 & 2.995 \end{array} \right).$$

Storing this result with only 2 significant digits gives

$$\left(\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & 1 & 3 \end{array} \right).$$

Now backsolving produces

$$x_1 = -1 \quad \text{and} \quad x_2 = 3,$$

which is the true solution (rounded to 2 significant digits).

The reason this worked is because 1 is bigger than 0.001. To pivot we switch rows so that the largest entry in a column is the one used to eliminate the others. In bigger matrices, after each column is completed, compare the diagonal element of the next column with all the entries below it. Switch it (and the entire row) with the one with greatest absolute value. For example in the following matrix, the first column is finished and before doing the second column, pivoting should occur since $|-2| > |1|$:

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 0 & 1 & 6 & 7 \\ 0 & -2 & -10 & -10 \end{array} \right).$$

Pivoting the 2nd and 3rd rows would produce

$$\left(\begin{array}{ccc|c} 1 & -2 & 3 & 4 \\ 0 & -2 & -10 & -10 \\ 0 & 1 & 6 & 7 \end{array} \right).$$

Condition number

In some systems, problems occur even without rounding. Consider the following augmented matrices:

$$\left(\begin{array}{cc|c} 1 & 1/2 & 3/2 \\ 1/2 & 1/3 & 1 \end{array} \right) \quad \text{and} \quad \left(\begin{array}{cc|c} 1 & 1/2 & 3/2 \\ 1/2 & 1/3 & 5/6 \end{array} \right).$$

Here we have the same A , but two different input vectors:

$$\mathbf{b}_1 = (3/2, 1)' \quad \text{and} \quad \mathbf{b}_2 = (3/2, 5/6)'$$

which are pretty close to one another. You would expect then that the solutions $\mathbf{x}_1$ and $\mathbf{x}_2$ would also be close. Notice that this matrix does not need pivoting. Eliminating exactly we get

$$\left(\begin{array}{cc|c} 1 & 1/2 & 3/2 \\ 0 & 1/12 & 1/4 \end{array} \right) \quad \text{and} \quad \left(\begin{array}{cc|c} 1 & 1/2 & 3/2 \\ 0 & 1/12 & 1/12 \end{array} \right).$$

Now solving we find

$$\mathbf{x}_1 = (0, 3)' \quad \text{and} \quad \mathbf{x}_2 = (1, 1)'$$

which are *not close at all* despite the fact that we did the calculations exactly. This poses a new problem: some matrices are very sensitive to small changes in input data. The extent of this sensitivity is measured

by the **condition number**. The definition of condition number is: consider all small changes δA and $\delta \mathbf{b}$ in A and $\mathbf{b}$ and the resulting change, $\delta \mathbf{x}$, in the solution $\mathbf{x}$. Then

$$\text{cond}(A) \equiv \max \left(\frac{\|\delta \mathbf{x}\| / \|\mathbf{x}\|}{\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}} \right) = \max \left(\frac{\text{Relative error of output}}{\text{Relative error of inputs}} \right).$$

Put another way, changes in the input data get multiplied by the condition number to produce changes in the outputs. Thus a high condition number is bad. It implies that small errors in the input can cause large errors in the output. It is not obvious from our definition above, but one can prove that the condition number of a matrix is at least 1.

In MATLAB enter

```
>> H = hilb(2)
```

which should result in the matrix above. MATLAB produces the condition number of a matrix with the command

```
>> cond(H)
```

Thus for this matrix small errors in the input can get magnified by 19 in the output. Next try the matrix

```
>> A = [ 1.2969 0.8648 ; .2161 .1441]
>> cond(A)
```

For this matrix small errors in the input can get magnified by 2.5×10^8 in the output! (We will see this happen in the exercise.) This is obviously not very good for engineering where all measurements, constants and inputs are approximate.

Is there a solution to the problem of bad condition numbers? Usually, bad condition numbers in engineering contexts result from poor design. So, the engineering solution to bad conditioning is **redesign**.

Finally, find the determinant of the matrix A above:

```
>> det(A)
```

which will be small. If $\det(A) = 0$ then the matrix is singular, which is bad because it implies there will not be a unique solution. The case here, $\det(A) \approx 0$, is also bad, because it means the matrix is almost singular. Although $\det(A) \approx 0$ generally indicates that the condition number will be large, they are actually independent things. To see this, find the determinant and condition number of the matrix $[1\text{e-}10, 0; 0, 1\text{e-}10]$ and the matrix $[1\text{e+}10, 0; 0, 1\text{e-}10]$.

Exercises

11.1 Let

$$A = \begin{bmatrix} 1.2969 & .8648 \\ .2161 & .1441 \end{bmatrix}.$$

- (a) Find the condition number, determinant and inverse of A (using MATLAB).
- (b) Let B be the matrix obtained from A by rounding off to three decimal places ($1.2969 \mapsto 1.297$). Find the inverse of B . How do A^{-1} and B^{-1} differ? Explain how this happened.
- (c) Set $\mathbf{b1} = [1.2969; 0.2161]$ and do $\mathbf{x} = \mathbf{A} \setminus \mathbf{b1}$. Repeat the process but with a vector $\mathbf{b2}$ obtained from $\mathbf{b1}$ by rounding off to three decimal places. Explain exactly what happened. Why was the first answer so simple? Why do the two answers differ by so much?

11.2 To see how to solve linear systems symbolically, try

```
>> B = [sin(sym(1)) sin(sym(2)); sin(sym(3)) sin(sym(4))]
>> c = [1; 2]
>> x = B \ c
>> pretty(x)
```

Now input the matrix

$$\mathbf{Cs} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

symbolically as above by wrapping each number in `sym`. Create a numerical version via `Cn = double(Cs)` and define the two vectors `d1 = [4; 8]` and `d2 = [1; 1]`. Solve the systems (as in the third line of code above) `Cs*x = d1`, `Cn*x = d1`, `Cs*x = d2`, and `Cn*x = d2`. Explain the results. Does the symbolic or non-symbolic way give more information?

Lecture 12

LU Decomposition

In many applications where linear systems appear, one needs to solve $A\mathbf{x} = \mathbf{b}$ for many different vectors $\mathbf{b}$. For instance, a structure must be tested under several different loads, not just one. As in the example of a truss (9.2), the loading in such a problem is usually represented by the vector $\mathbf{b}$. Gaussian elimination with pivoting is the most efficient and accurate way to solve a linear system. Most of the work in this method is spent on the matrix A itself. If we need to solve several different systems with the same A , and A is big, then we would like to avoid repeating the steps of Gaussian elimination on A for every different $\mathbf{b}$. This can be accomplished by the *LU decomposition*, which in effect records the steps of Gaussian elimination.

LU decomposition

The main idea of the LU decomposition is to record the steps used in Gaussian elimination on A in the places where the zero is produced. Consider the matrix

$$A = \begin{pmatrix} 1 & -2 & 3 \\ 2 & -5 & 12 \\ 0 & 2 & -10 \end{pmatrix}.$$

The first step of Gaussian elimination is to subtract 2 times the first row from the second row. In order to record what we have done, we will put the multiplier, 2, into the place it was used to make a zero, i.e. the second row, first column. In order to make it clear that it is a record of the step and not an element of A , we will put it in parentheses. This leads to

$$\begin{pmatrix} 1 & -2 & 3 \\ (2) & -1 & 6 \\ 0 & 2 & -10 \end{pmatrix}.$$

There is already a zero in the lower left corner, so we don't need to eliminate anything there. We record this fact with a (0). To eliminate the third row, second column, we need to subtract -2 times the second row from the third row. Recording the -2 in the spot it was used we have

$$\begin{pmatrix} 1 & -2 & 3 \\ (2) & -1 & 6 \\ (0) & (-2) & 2 \end{pmatrix}.$$

Let U be the upper triangular matrix produced, and let L be the lower triangular matrix with the records and ones on the diagonal, i.e.

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix} \quad \text{and} \quad U = \begin{pmatrix} 1 & -2 & 3 \\ 0 & -1 & 6 \\ 0 & 0 & 2 \end{pmatrix},$$

then we have the following wonderful property:

$$LU = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix} \begin{pmatrix} 1 & -2 & 3 \\ 0 & -1 & 6 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & -2 & 3 \\ 2 & -5 & 12 \\ 0 & 2 & -10 \end{pmatrix} = A.$$

Thus we see that A is actually the product of L and U . Here L is lower triangular and U is upper triangular. When a matrix can be written as a product of simpler matrices, we call that a *decomposition* of A and this one we call the LU decomposition.

Using LU to solve equations

If we also include pivoting, then an LU decomposition for A consists of three matrices P , L and U such that

$$PA = LU. \quad (12.1)$$

The pivot matrix P is the identity matrix, with the same rows switched as the rows of A are switched in the pivoting. For instance,

$$P = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

would be the pivot matrix if the second and third rows of A are switched by pivoting. MATLAB will produce an LU decomposition with pivoting for a matrix A with the command

```
> [L U P] = lu(A)
```

where P is the pivot matrix. To use this information to solve $A\mathbf{x} = \mathbf{b}$ we first pivot both sides by multiplying by the pivot matrix:

$$PA\mathbf{x} = P\mathbf{b} \equiv \mathbf{d}.$$

Substituting LU for PA we get

$$LU\mathbf{x} = \mathbf{d}.$$

Then we define $\mathbf{y} = U\mathbf{x}$, which is unknown since $\mathbf{x}$ is unknown. Using forward-substitution, we can (easily) solve

$$L\mathbf{y} = \mathbf{d}$$

for $\mathbf{y}$ and then using back-substitution we can (easily) solve

$$U\mathbf{x} = \mathbf{y}$$

for $\mathbf{x}$. In MATLAB this would work as follows:

```
>> A = rand(5,5)
>> [L U P] = lu(A)
>> b = rand(5,1)
>> d = P*b
>> y = L\d
>> x = U\y
>> rnorm = norm(A*x - b) % Check the result
```

We can then solve for any other $\mathbf{b}$ without redoing the LU step. Repeat the sequence for a new right hand side: $\mathbf{c} = \text{randn}(5,1)$; you can start at the third line. While this may not seem like a big savings, it would be if A were a large matrix from an actual application.

The LU decomposition is an example of *Matrix Decomposition* which means taking a general matrix A and breaking it down into components with simpler properties. Here L and U are simpler because they are lower and upper triangular. There are many other matrix decompositions that are useful in various contexts. Some of the most useful of these are the QR decomposition, the Singular Value decomposition and Cholesky decomposition. Often a decomposition is associated with an algorithm, e.g., finding the LU decomposition is equivalent to completing Gaussian Elimination.

Exercises

- 12.1 Solve the systems below by hand using Gaussian elimination and back substitution (exactly as above) on the augmented matrix. As a by-product, give the LU decomposition of A . Pivot wherever appropriate (the number being eliminated should be smaller than the number eliminating it). Check by hand that $LU = PA$ and $A\mathbf{x} = \mathbf{b}$ and compare with MATLAB.

$$(a) \quad A = \begin{pmatrix} 0.5 & 4 \\ 2 & 4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$$

$$(b) \quad A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 1 & 1 \\ 2 & 3 & 9 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

- 12.2 Finish the following MATLAB function program:

```
function [x1, r1, x2, r2] = mysolve(A,b)
    % Solves linear systems using the LU decomposition with pivoting
    % and also with the built-in solve function A\b.
    % Inputs: A -- the matrix
    %          b -- the right-hand vector
    % Outputs: x1 -- the solution using the LU method
    %          r1 -- the scalar residual using the LU method
    %          x2 -- the solution using the built-in method
    %          r2 -- the scalar residual using the
    %                built-in method
```

Using `format long`, test the program on both random matrices (`randn(n,n)`) and Hilbert matrices (`hilb(n)`) with n large (as big as you can make it and the program still run). Print your program and summarize your observations. (Do not print any random matrices or vectors.)

Lecture 13

Nonlinear Systems - Newton's Method

An Example

The LORAN (LOng RANge Navigation) system calculates the position of a boat at sea using signals from fixed transmitters. From the time differences of the incoming signals, the boat obtains differences of distances to the transmitters. This leads to two equations each representing hyperbolas defined by the differences of distance of two points (foci). An example of such equations¹ are

$$\begin{aligned}\frac{x^2}{186^2} - \frac{y^2}{300^2 - 186^2} &= 1 \quad \text{and} \\ \frac{(y - 500)^2}{279^2} - \frac{(x - 300)^2}{500^2 - 279^2} &= 1.\end{aligned}\tag{13.1}$$

Solving two quadratic equations with two unknowns would require solving a 4 degree polynomial equation. We could do this by hand, but for a navigational system to work well, it must do the calculations automatically and numerically. We note that the Global Positioning System (GPS) works on similar principles and must do similar computations.

Vector Notation

In general, we can usually find solutions to a system of equations when the number of unknowns matches the number of equations. Thus, we wish to find solutions to systems that have the form

$$\begin{aligned}f_1(x_1, x_2, x_3, \dots, x_n) &= 0 \\ f_2(x_1, x_2, x_3, \dots, x_n) &= 0 \\ f_3(x_1, x_2, x_3, \dots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, x_3, \dots, x_n) &= 0.\end{aligned}\tag{13.2}$$

For convenience we can think of $(x_1, x_2, x_3, \dots, x_n)$ as a vector $\mathbf{x}$ and $(f_1, f_2, \dots, f_n)$ as a vector-valued function $\mathbf{f}$. With this notation, we can write the system of equations (13.2) simply as

$$\mathbf{f}(\mathbf{x}) = \mathbf{0},$$

i.e. we wish to find a vector that makes the vector function equal to the zero vector.

¹E. Johnston, J. Mathews, *Calculus*, Addison-Wesley, 2001

As in Newton's method for one variable, we need to start with an initial guess $\mathbf{x}_0$. In theory, the more variables one has, the harder it is to find a good initial guess. In practice, this must be overcome by using physically reasonable assumptions about the possible values of a solution, i.e. take advantage of engineering knowledge of the problem.

Linear Approximation for Vector Functions

In the single variable case, Newton's method was derived by considering the linear approximation of the function f at the initial guess $\mathbf{x}_0$. From Calculus, the following is the linear approximation of $\mathbf{f}$ at $\mathbf{x}_0$, for vectors and vector-valued functions:

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{f}(\mathbf{x}_0) + D\mathbf{f}(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0).$$

Here $D\mathbf{f}(\mathbf{x}_0)$ is an $n \times n$ matrix whose entries are the various partial derivative of the components of $\mathbf{f}$, evaluated at $\mathbf{x}_0$. Specifically,

$$D\mathbf{f}(\mathbf{x}_0) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}_0) & \frac{\partial f_1}{\partial x_2}(\mathbf{x}_0) & \frac{\partial f_1}{\partial x_3}(\mathbf{x}_0) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}_0) \\ \frac{\partial f_2}{\partial x_1}(\mathbf{x}_0) & \frac{\partial f_2}{\partial x_2}(\mathbf{x}_0) & \frac{\partial f_2}{\partial x_3}(\mathbf{x}_0) & \cdots & \frac{\partial f_2}{\partial x_n}(\mathbf{x}_0) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1}(\mathbf{x}_0) & \frac{\partial f_n}{\partial x_2}(\mathbf{x}_0) & \frac{\partial f_n}{\partial x_3}(\mathbf{x}_0) & \cdots & \frac{\partial f_n}{\partial x_n}(\mathbf{x}_0) \end{pmatrix}. \quad (13.3)$$

Newton's Method

We wish to find $\mathbf{x}$ that makes $\mathbf{f}$ equal to the zero vectors, so let's choose $\mathbf{x}_1$ so that

$$\mathbf{f}(\mathbf{x}_0) + D\mathbf{f}(\mathbf{x}_0)(\mathbf{x}_1 - \mathbf{x}_0) = \mathbf{0}.$$

Since $D\mathbf{f}(\mathbf{x}_0)$ is a square matrix, we can solve this equation by

$$\mathbf{x}_1 = \mathbf{x}_0 - (D\mathbf{f}(\mathbf{x}_0))^{-1}\mathbf{f}(\mathbf{x}_0),$$

provided that the inverse exists. The formula is the vector equivalent of the Newton's method formula we learned before. However, in practice we never use the inverse of a matrix for computations, so we cannot use this formula directly. Rather, we can do the following. First solve the equation

$$D\mathbf{f}(\mathbf{x}_0)\Delta\mathbf{x} = -\mathbf{f}(\mathbf{x}_0), \quad (13.4)$$

where we want to have

$$\Delta\mathbf{x} = \mathbf{x}_1 - \mathbf{x}_0.$$

Since $D\mathbf{f}(\mathbf{x}_0)$ is a known matrix and $-\mathbf{f}(\mathbf{x}_0)$ is a known vector, this equation is just a system of linear equations, which can be solved efficiently and accurately. Once we have the solution vector $\Delta\mathbf{x}$, we can obtain our improved estimate $\mathbf{x}_1$ by

$$\mathbf{x}_1 = \mathbf{x}_0 + \Delta\mathbf{x}.$$

For subsequent steps, we have the following process:

- Solve $D\mathbf{f}(\mathbf{x}_i)\Delta\mathbf{x} = -\mathbf{f}(\mathbf{x}_i)$ for $\Delta\mathbf{x}$.
- Let $\mathbf{x}_{i+1} = \mathbf{x}_i + \Delta\mathbf{x}$

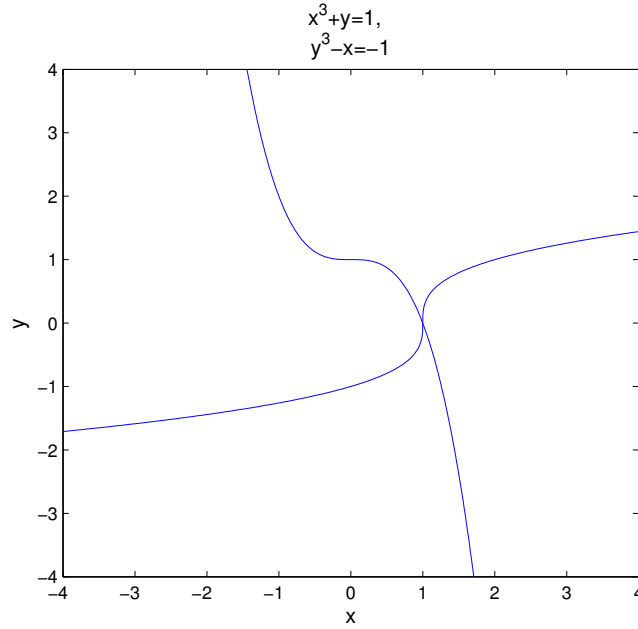


Figure 13.1: Graphs of the equations $x^3 + y = 1$ and $y^3 - x = -1$. There is one and only one intersection; at $(x, y) = (1, 0)$.

An Experiment

We will solve the following set of equations:

$$\begin{aligned} x^3 + y &= 1 \\ y^3 - x &= -1. \end{aligned} \tag{13.5}$$

You can easily check that $(x, y) = (1, 0)$ is a solution of this system. By graphing both of the equations you can also see that $(1, 0)$ is the only solution (Figure 13.1).

We can put these equations into vector-function form (13.2) by letting $x_1 = x$, $x_2 = y$ and

$$\begin{aligned} f_1(x_1, x_2) &= x_1^3 + x_2 - 1 \\ f_2(x_1, x_2) &= x_2^3 - x_1 + 1, \end{aligned}$$

or, equivalently,

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} x_1^3 + x_2 - 1 \\ x_2^3 - x_1 + 1 \end{pmatrix}.$$

The partial derivatives are

$$\frac{\partial f_1}{\partial x_1} = 3x_1^2, \quad \frac{\partial f_1}{\partial x_2} = 1, \quad \frac{\partial f_2}{\partial x_1} = -1, \quad \text{and} \quad \frac{\partial f_2}{\partial x_2} = 3x_2^2. \tag{13.6}$$

Thus,

$$D\mathbf{f} = \begin{pmatrix} 3x_1^2 & 1 \\ -1 & 3x_2^2 \end{pmatrix}. \tag{13.7}$$

Now that we have the equations in vector-function form, we can write the following script program:

```
format long;
f = @(x)[ x(1)^3+x(2)-1 ; x(2)^3-x(1)+1 ]
x = [.5;.5]
x = fsolve(f,x)
```

Save this program as `mysolve.m` and run it. You will see that the internal MATLAB solving command `fsolve` approximates the solution, but only to about 7 decimal places. While that would be close enough for most applications, one would expect that we could do better on such a simple problem.

Next we will implement Newton's method for this problem. Modify your `mysolve` program to:

```
% mymultnewton - solve an example nonlinear system
format long;
n=8 % set some number of iterations, may need adjusting
f = @(x)[x(1)^3+x(2)-1 ; x(2)^3-x(1)+1] % the vector function
% the matrix of partial derivatives
Df = @(x)[3*x(1)^2, 1 ; -1, 3*x(2)^2]
x = [.5;.5] % starting guess
for i = 1:n
    Dx = -Df(x)\f(x); % solve for increment
    x = x + Dx % add on to get new guess
    f(x) % see if f(x) is really zero
end
```

Save and run this program (as `mymultnewton`) and you will see that it finds the root exactly (to machine precision) in only 6 iterations. Why is this simple program able to do better than MATLAB's built-in program?

Exercises

- 13.1 (a) Put the LORAN equations (13.1) into the function form (13.2).
- (b) Calculate (by hand) the partial derivatives of $\mathbf{f}$ and construct the matrix $D\mathbf{f}$ (see (13.6) and (13.7)).
- (c) Adapt the `mymultnewton` program to find a solution for these equations. By trying different starting vectors, find at least three different solutions. (There are actually four solutions.)
- (d) Think of at least one way that the navigational system could determine the correct solution.

Lecture 14

Eigenvalues and Eigenvectors

Suppose that A is a square ($n \times n$) matrix. We say that a nonzero vector $\mathbf{v}$ is an eigenvector and a number λ is its eigenvalue if

$$A\mathbf{v} = \lambda\mathbf{v}. \quad (14.1)$$

Geometrically this means that $A\mathbf{v}$ is in the same direction as $\mathbf{v}$, since multiplying a vector by a number changes its length, but not its direction.

MATLAB has a built-in routine for finding eigenvalues and eigenvectors:

```
>> A = pascal(4)
>> [v e] = eig(A)
```

The results are a matrix $\mathbf{v}$ that contains eigenvectors as columns and a diagonal matrix $\mathbf{e}$ that contains eigenvalues on the diagonal. We can check this by

```
>> v1 = v(:,1)
>> A*v1
>> e(1,1)*v1
```

Finding Eigenvalues for 2×2 and 3×3

If A is 2×2 or 3×3 then we can find its eigenvalues and eigenvectors by hand. Notice that Equation (14.1) can be rewritten as

$$A\mathbf{v} - \lambda\mathbf{v} = \mathbf{0}.$$

It would be nice to factor out the $\mathbf{v}$ from the right-hand side of this equation, but we can't because A is a matrix and λ is a number. However, since $I\mathbf{v} = \mathbf{v}$, we can do the following:

$$\begin{aligned} A\mathbf{v} - \lambda\mathbf{v} &= A\mathbf{v} - \lambda I\mathbf{v} \\ &= (A - \lambda I)\mathbf{v} \\ &= \mathbf{0} \end{aligned}$$

If $\mathbf{v}$ is nonzero, then by Theorem 3 in Lecture 10 the matrix $(A - \lambda I)$ must be singular. By the same theorem, we must have

$$\det(A - \lambda I) = 0.$$

This is called the *characteristic equation*.

For a 2×2 matrix, $A - \lambda I$ is calculated as in the following example:

$$\begin{aligned} A - \lambda I &= \begin{pmatrix} 1 & 4 \\ 3 & 5 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 4 \\ 3 & 5 \end{pmatrix} - \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix} \\ &= \begin{pmatrix} 1 - \lambda & 4 \\ 3 & 5 - \lambda \end{pmatrix}. \end{aligned}$$

The determinant of $A - \lambda I$ is then

$$\begin{aligned} \det(A - \lambda I) &= (1 - \lambda)(5 - \lambda) - 4 \cdot 3 \\ &= -7 - 6\lambda + \lambda^2. \end{aligned}$$

The characteristic equation $\det(A - \lambda I) = 0$ is simply a quadratic equation:

$$\lambda^2 - 6\lambda - 7 = 0.$$

The roots of this equation are $\lambda_1 = 7$ and $\lambda_2 = -1$. These are the eigenvalues of the matrix A . Now to find the corresponding eigenvectors we return to the equation $(A - \lambda I)\mathbf{v} = \mathbf{0}$. For $\lambda_1 = 7$, the equation for the eigenvector $(A - \lambda I)\mathbf{v} = \mathbf{0}$ is equivalent to the augmented matrix

$$\left(\begin{array}{cc|c} -6 & 4 & 0 \\ 3 & -2 & 0 \end{array} \right). \quad (14.2)$$

Notice that the first and second rows of this matrix are multiples of one another. Thus Gaussian elimination would produce all zeros on the bottom row. Thus this equation has infinitely many solutions, i.e. infinitely many eigenvectors. Since only the direction of the eigenvector matters, this is okay, we only need to find one of the eigenvectors. Since the second row of the augmented matrix represents the equation

$$3x - 2y = 0,$$

we can let

$$\mathbf{v}_1 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

This comes from noticing that $(x, y) = (2, 3)$ is a solution of $3x - 2y = 0$.

For $\lambda_2 = -1$, $(A - \lambda I)\mathbf{v} = \mathbf{0}$ is equivalent to the augmented matrix

$$\left(\begin{array}{cc|c} 2 & 4 & 0 \\ 3 & 6 & 0 \end{array} \right).$$

Once again the first and second rows of this matrix are multiples of one another. For simplicity we can let

$$\mathbf{v}_2 = \begin{pmatrix} -2 \\ 1 \end{pmatrix}.$$

One can always check an eigenvector and eigenvalue by multiplying:

$$\begin{aligned} A\mathbf{v}_1 &= \begin{pmatrix} 1 & 4 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 14 \\ 21 \end{pmatrix} = 7 \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 7\mathbf{v}_1 \quad \text{and} \\ A\mathbf{v}_2 &= \begin{pmatrix} 1 & 4 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} -2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix} = -1 \begin{pmatrix} -2 \\ 1 \end{pmatrix} = -1\mathbf{v}_2. \end{aligned}$$

For a 3×3 matrix we could complete the same process. The $\det(A - \lambda I) = 0$ would be a cubic polynomial and we would expect to usually get 3 roots, which are the eigenvalues.

Larger Matrices

For a $n \times n$ matrix with $n \geq 4$ this process is too long and cumbersome to complete by hand. Further, this process is not well suited even to implementation on a computer program since it involves determinants and solving a n -degree polynomial. For $n \geq 4$ we need more ingenious methods. These methods rely on the geometric meaning of eigenvectors and eigenvalues rather than solving algebraic equations. We will overview these methods in Lecture 16.

Complex Eigenvalues

It turns out that the eigenvalues of some matrices are complex numbers, even when the matrix only contains real numbers. When this happens the complex eigenvalues must occur in conjugate pairs, i.e.

$$\lambda_{1,2} = \alpha \pm i\beta.$$

The corresponding eigenvectors must also come in conjugate pairs:

$$\mathbf{w} = \mathbf{u} \pm i\mathbf{v}.$$

In applications, the imaginary part of the eigenvalue, β , often is related to the frequency of an oscillation. This is because of Euler's formula

$$e^{\alpha+i\beta} = e^{\alpha}(\cos \beta + i \sin \beta).$$

Certain kinds of matrices that arise in applications can only have real eigenvalues and eigenvectors. The most common such type of matrix is the symmetric matrix. A matrix is symmetric if it is equal to its own transpose, i.e. it is symmetric across the diagonal. For example,

$$\begin{pmatrix} 1 & 3 \\ 3 & -5 \end{pmatrix}$$

is symmetric and so we know beforehand that its eigenvalues will be real, not complex.

Exercises

14.1 Find the eigenvalues and eigenvectors of the following matrix by hand:

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

14.2 Find the eigenvalues and eigenvectors of the following matrix by hand:

$$B = \begin{pmatrix} 1 & -3 \\ 3 & 1 \end{pmatrix}.$$

Can you guess the eigenvalues of the matrix

$$C = \begin{pmatrix} a & -b \\ b & a \end{pmatrix}?$$

Lecture 15

An Application of Eigenvectors: Vibrational Modes and Frequencies

One application of eigenvalues and eigenvectors is in the analysis of vibration problems. A simple nontrivial vibration problem is the motion of two objects with equal masses m attached to each other and fixed outer walls by equal springs with spring constants k , as shown in Figure 15.1.

Let x_1 denote the displacement of the first mass and x_2 the displacement of the second, and note the displacement of the walls is zero. Each mass experiences forces from the adjacent springs proportional to the stretch or compression of the spring. Ignoring any friction, Newton's law of motion $ma = F$, leads to

$$\begin{aligned} m\ddot{x}_1 &= -k(x_1 - 0) + k(x_2 - x_1) &= -2kx_1 + kx_2 \quad \text{and} \\ m\ddot{x}_2 &= -k(x_2 - x_1) + k(0 - x_2) &= kx_1 - 2kx_2 \quad . \end{aligned} \quad (15.1)$$

Dividing both sides by m we can write these equations in matrix form

$$\ddot{\mathbf{x}} = -A\mathbf{x}, \quad (15.2)$$

where

$$A = \begin{pmatrix} 2\frac{k}{m} & -1\frac{k}{m} \\ -1\frac{k}{m} & 2\frac{k}{m} \end{pmatrix}. \quad (15.3)$$

For this type of equation, the general solution is

$$\mathbf{x}(t) = c_1 \mathbf{v}_1 \sin(\sqrt{\lambda_1} t + \phi_1) + c_2 \mathbf{v}_2 \sin(\sqrt{\lambda_2} t + \phi_2) \quad (15.4)$$

where λ_1 and λ_2 are eigenvalues of A with corresponding eigenvectors $\mathbf{v}_1$ and $\mathbf{v}_2$. One can check that this is a solution by substituting it into the equation (15.2).

The eigenvalues of A are the squares of the frequencies of oscillation. Let's set $m = 1$ and $k = 1$ in A . We can find the eigenvalues and eigenvectors of A using Matlab:

```
>> A = [2 -1 ; -1 2]
>> [v e] = eig(A)
```

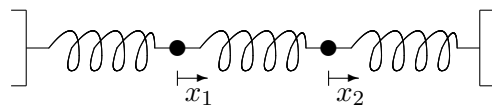


Figure 15.1: Two equal masses attached to each other and fixed walls by equal springs.

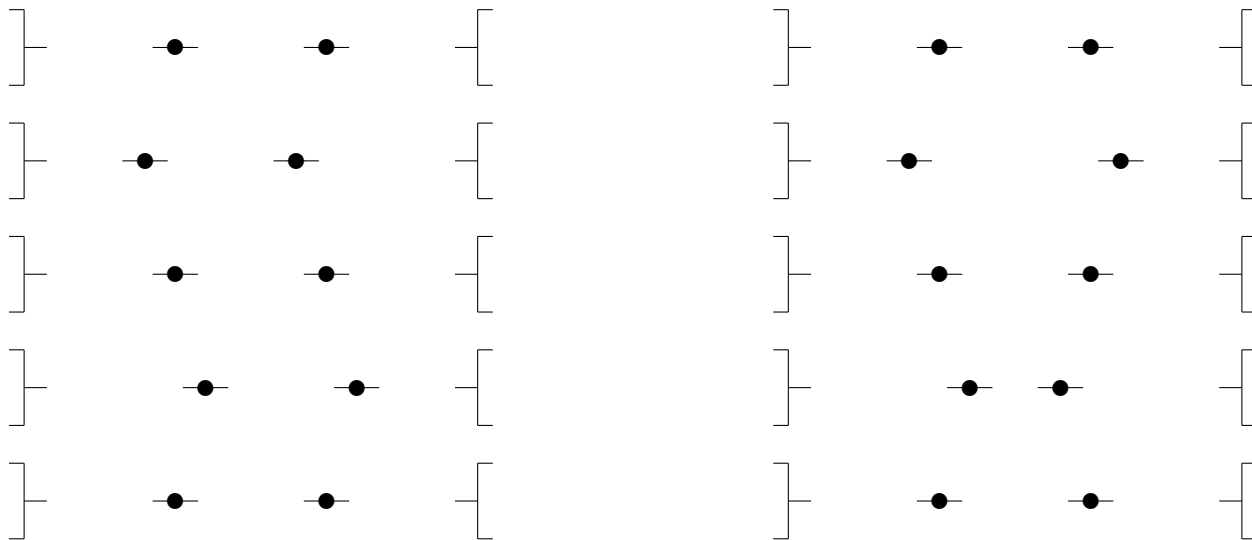


Figure 15.2: Two vibrational modes of a simple oscillating system. In the left mode the weights move together and in the right mode they move opposite. Note that the two modes actually move at different speeds.

This should produce a matrix $\mathbf{v}$ whose columns are the eigenvectors of A and a diagonal matrix $\mathbf{e}$ whose entries are the eigenvalues of A . In the first eigenvector, $\mathbf{v}_1$, the two entries are equal. This represents the mode of oscillation where the two masses move in sync with each other. The second eigenvector, $\mathbf{v}_2$, has the same entries but opposite signs. This represents the mode where the two masses oscillate in anti-synchronization. See Figure 15.2. Notice that the frequency for anti-sync motion is $\sqrt{3}$ times that of synchronous motion.

Which of the two modes is the most dangerous for a structure or machine? It is the one with the *lowest frequency* because that mode can have the largest displacement. Sometimes this mode is called the *fundamental mode*.

We can do the same for three equal masses. With $m = 1$, $k = 1$ the corresponding matrix A would be

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}.$$

Find the eigenvectors and eigenvalues as above. There are three different modes. Interpret them from the eigenvectors.

Exercises

- 15.1 Find the modes and their frequencies for 4 equal masses with $m = 3$ kg and equal springs with $k = 10$ N/m. Describe the modes (a sketch will suffice).
- 15.2 Find the modes and their frequencies for three unequal masses $m_1 = 2$ kg, $m_2 = 3$ kg and $m_3 = 4$ kg connected by 4 equal springs with $k = 5$ N/m. How do unequal masses affect the modes? (You must start with the equations of motion to do this correctly.)

Lecture 16

Numerical Methods for Eigenvalues

As mentioned above, the eigenvalues and eigenvectors of an $n \times n$ matrix where $n \geq 4$ must be found numerically instead of by hand. The numerical methods that are used in practice depend on the geometric meaning of eigenvalues and eigenvectors which is equation (14.1). The essence of all these methods is captured in the Power method, which we now introduce.

The Power Method

In the command window of MATLAB enter

```
>> A = hilb(5)
>> x = ones(5,1)
>> x = A*x
>> e1 = max(x)
>> x = x/e1
```

Compare the new value of $\mathbf{x}$ with the original. Repeat the last three lines (you can use the scroll up button). Compare the newest value of $\mathbf{x}$ with the previous one and the original. Notice that there is less change between the second two. Repeat the last three commands over and over until the values stop changing. You have completed what is known as the *Power Method*. Now try the command

```
>> [v e] = eig(A)
```

The last entry in $\mathbf{e}$ should be the final $\mathbf{e1}$ we computed. The last column in $\mathbf{v}$ is $\mathbf{x}/\text{norm}(\mathbf{x})$. Below we explain why our commands gave this eigenvalue and eigenvector.

For illustration consider a 2×2 matrix whose eigenvalues are $1/3$ and -2 and whose corresponding eigenvectors are $\mathbf{v}_1$ and $\mathbf{v}_2$. Let $\mathbf{x}_0$ be any vector which is a combination of $\mathbf{v}_1$ and $\mathbf{v}_2$, e.g.,

$$\mathbf{x}_0 = \mathbf{v}_1 + \mathbf{v}_2.$$

Now let $\mathbf{x}_1$ be A times $\mathbf{x}_0$. It follows from (14.1) that

$$\begin{aligned}\mathbf{x}_1 &= A\mathbf{v}_1 + A\mathbf{v}_2 \\ &= \frac{1}{3}\mathbf{v}_1 + -2\mathbf{v}_2.\end{aligned}\tag{16.1}$$

Thus the $\mathbf{v}_1$ part is shrunk while the $\mathbf{v}_2$ is stretched. If we repeat this process k times then

$$\begin{aligned}\mathbf{x}_k &= A\mathbf{x}_{k-1} \\ &= A^k\mathbf{x}_0 \\ &= \left(\frac{1}{3}\right)^k \mathbf{v}_1 + (-2)^k \mathbf{v}_2.\end{aligned}\tag{16.2}$$

Clearly, $\mathbf{x}_k$ grows in the direction of $\mathbf{v}_2$ and shrinks in the direction of $\mathbf{v}_1$. This is the principle of the Power Method, vectors multiplied by A are stretched most in the direction of the eigenvector whose eigenvalue has the largest absolute value.

The eigenvalue with the largest absolute value is called the *dominant* eigenvalue. In many applications this quantity will necessarily be positive for physical reasons. When this is the case, the MATLAB code above will work since `max(v)` will be the element with the largest absolute value. In applications where the dominant eigenvalue may be negative, the program must use flow control to determine the correct number.

Summarizing the Power Method:

- Repeatedly multiply $\mathbf{x}$ by A and divide by the element with the largest absolute value.
- The element of largest absolute value converges to largest absolute eigenvalue.
- The vector converges to the corresponding eigenvector.

Note that this logic only works when the eigenvalue largest in magnitude is real. If the matrix and starting vector are real then the power method can never give a result with an imaginary part. Eigenvalues with imaginary part mean the matrix has a rotational component, so the eigenvector would not settle down either.

Try

```
>> A = randn(15,15);
>> e = eig(A)
```

You can see that for a random square matrix, many of the eigenvalues are complex.

However, matrices in applications are not just random. They have structure, and this can lead to real eigenvalues as seen in the next section.

Symmetric, Positive-Definite Matrices

As noted in the previous paragraph, the power method can fail if A has complex eigenvalues. One class of matrices that appear often in applications and for which the eigenvalues are always real are called the symmetric matrices. A matrix is *symmetric* if

$$A' = A,$$

i.e. A is symmetric with respect to reflections about its diagonal.

Try

```
>> A = rand(5,5)
>> C = A'*A
>> e = eig(C)
```

You can see that the eigenvalues of these symmetric matrices are real.

Next we consider an even more specialized class for which the eigenvalues are not only real, but positive. A symmetric matrix is called *positive definite* if for all vectors $\mathbf{v} \neq \mathbf{0}$ the following holds:

$$A\mathbf{v} \cdot \mathbf{v} > 0.$$

Geometrically, A does not rotate any vector by more than $\pi/2$. In summary:

- If A is symmetric then its eigenvalues are real.
- If A is symmetric positive definite, then its eigenvalues are positive numbers.

Notice that the B matrices in the previous section were symmetric and the eigenvalues were all real. Notice that the Hilbert and Pascal matrices are symmetric.

The residual of an approximate eigenvector-eigenvalue pair

If $\mathbf{v}$ and λ are an eigenvector-eigenvalue pair for A , then they are supposed to satisfy the equations: $A\mathbf{v} = \lambda\mathbf{v}$. Thus a scalar residual for approximate $\mathbf{v}$ and λ would be

$$r = \|A\mathbf{v} - \lambda\mathbf{v}\|.$$

The Inverse Power Method

In the application of vibration analysis, the mode (eigenvector) with the lowest frequency (eigenvalue) is the most dangerous for the machine or structure. The Power Method gives us instead the largest eigenvalue, which is the least important frequency. In this section we introduce a method, the *Inverse Power Method* which produces exactly what is needed.

The following facts are at the heart of the Inverse Power Method:

- If λ is an eigenvalue of A then $1/\lambda$ is an eigenvalue for A^{-1} .
- The eigenvectors for A and A^{-1} are the same.

Thus if we apply the Power Method to A^{-1} we will obtain the largest absolute eigenvalue of A^{-1} , which is exactly the reciprocal of the smallest absolute eigenvalue of A . We will also obtain the corresponding eigenvector, which is an eigenvector for both A^{-1} and A . Recall that in the application of vibration mode analysis, the smallest eigenvalue and its eigenvector correspond exactly to the frequency and mode that we are most interested in, i.e. the one that can do the most damage.

Here as always, we do not really want to calculate the inverse of A directly if we can help it. Fortunately, multiplying $\mathbf{x}_i$ by A^{-1} to get $\mathbf{x}_{i+1}$ is equivalent to solving the system $A\mathbf{x}_{i+1} = \mathbf{x}_i$, which can be done

efficiently and accurately. Since iterating this process involves solving a linear system with the same A but many different right hand sides, it is a perfect time to use the LU decomposition to save computations. The following function program does n steps of the Inverse Power Method.

```
function [x e] = myipm(A,n)
    % Performs the inverse power method.
    % Inputs: A -- a square matrix
    %          n -- number of iterations.
    % Outputs: x -- estimated eigenvector
    %          e -- estimated smallest eigenvalue.
    [L U P] = lu(A); % LU decomposition of A with pivoting
    m = size(A,1); % determine the size of A
    x = ones(m,1); % make an initial vector with ones
    for i = 1:n
        px = P*x; % Apply pivot
        y = L\px; % solve via LU
        x = U\y;
        % find the maximum entry in absolute value, retaining its sign
        M = max(x);
        m = min(x);
        if abs(M) >= abs(m)
            el = M;
        else
            el = m;
        end
        x = x/el; % divide by the estimated eigenvalue of the inverse of A
    end
    e = 1/el; % reciprocate to get an eigenvalue of A
end
```

Create a 5×5 Hilbert matrix H and use the program to find its smallest eigenvalue and corresponding eigenvector. Also do $[V, E] = \text{eig}(H)$ and compare.

Exercises

16.1 For each of the matrices

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} -2 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -3 \end{bmatrix},$$

perform two iterations of the power method by hand starting with a vector of all ones. State the resulting approximations of the eigenvalue and eigenvector.

- 16.2 (a) Write a well-commented MATLAB **function** program `mypm.m` that inputs a matrix and a tolerance, applies the power method until the scalar residual is less than the tolerance, and outputs the estimated eigenvalue and eigenvector, the number of steps, and the scalar residual.
- (b) Test your program on the matrices A and B in the previous exercise and check that you get the same results as your hand calculations for the first 2 iterations. Turn in your program only.

Lecture 17

The QR Method*

The Power Method and Inverse Power Method each give us only one eigenvalue-eigenvector pair. While both of these methods can be modified to give more eigenvalues and eigenvectors, there is a better method for obtaining all the eigenvalues called the *QR method*. This is the basis of all modern eigenvalue software, including MATLAB, so we summarize it briefly here.

The QR method uses the fact that any square matrix has a *QR decomposition*. That is, for any A there are matrices Q and R such the $A = QR$ where Q has the property

$$Q^{-1} = Q'$$

and R is upper triangular. A matrix Q with the property that its transpose equals its inverse is called an *orthogonal* matrix, because its column vectors are mutually orthogonal.

The QR method consists of iterating following steps:

- Transform A into a tridiagonal matrix H .
- Decompose H in QR .
- Multiply Q and R together in reverse order to form a new H .

The diagonal of H will converge to the eigenvalues.

The details of what makes this method converge are beyond the scope of the this book. However, we note the following theory behind it for those with more familiarity with linear algebra. First the Hessian matrix H is obtained from A by a series of similarity transformation, thus it has the same eigenvalues as A . Secondly, if we denote by $H_0, H_1, H_2, \dots$, the sequence of matrices produced by the iteration, then

$$H_{i+1} = R_i Q_i = Q_i^{-1} Q_i R_i Q_i = Q_i' H_i Q_i.$$

Thus each H_{i+1} is a related to H_i by an (orthogonal) similarity transformation and so they have the same eigenvalues as A .

There is a built-in QR decomposition in MATLAB which is called with the command: `[Q R] = qr(A)`. Thus the following program implements QR method until it converges:

```

function [E,steps] = myqrmeth(A)
% Computes all the eigenvalues of a matrix using the QR method.
% Input: A -- square matrix
% Outputs: E -- vector of eigenvalues
%          steps -- the number of iterations it took
[m n] = size(A);
if m ~= n
    warning('The input matrix is not square.')
    return
end
% Set up initial estimate
H = hess(A);
E = diag(H);
change = 1;
steps = 0;
% loop while estimate changes
while change > 0
    Eold = E;
    % apply QR method
    [Q R] = qr(H);
    H = R*Q;
    E = diag(H);
    % test change
    change = norm(E - Eold);
    steps = steps + 1;
end
end

```

As you can see the main steps of the program are very simple. The really hard calculations are contained in the built-in commands `hess(A)` and `qr(H)`.

Run this program and compare the results with MATLAB's built in command:

```

>> format long
>> format compact
>> A = hilb(5)
>> [Eqr,steps] = myqrmeth(A)
>> Eml = eig(A)
>> diff = norm(Eml - flipud(Eqr))

```

Exercises

- 17.1 Modify `myqrmeth` to stop after 1000 iterations. Use the modified program on the matrix `A = hilb(n)` with `n` equal to 10, 50, and 200. Use the norm to compare the results to the eigenvalues obtained from MATLAB's built-in program `eig`. Turn in your program and a brief report on the experiment.

Lecture 18

Iterative solution of linear systems*

Newton refinement

Conjugate gradient method

Review of Part II

Methods and Formulas

Basic Matrix Theory:

Identity matrix: $AI = A$, $IA = A$, and $I\mathbf{v} = \mathbf{v}$

Inverse matrix: $AA^{-1} = I$ and $A^{-1}A = I$

Norm of a matrix: $\|A\| \equiv \max_{\|\mathbf{v}\|=1} \|A\mathbf{v}\|$

A matrix may be singular or nonsingular. See Lecture 10.

Solving Process:

Learn the exact Gaussian Elimination algorithm:

Row $j \mapsto$ Row j - (ratio) Row i

Gaussian Elimination this way produces LU decomposition

Row Pivoting (bigger absolute number on top)

Back Substitution

Condition number:

$$\text{cond}(A) \equiv \max \left(\frac{\|\delta \mathbf{x}\| / \|\mathbf{x}\|}{\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}} \right) = \max \left(\frac{\text{Relative error of output}}{\text{Relative error of inputs}} \right).$$

A big condition number is bad; in engineering it usually results from poor design.

LU factorization:

The LU factorization is a by-product of Gaussian Elimination (if done with the correct algorithm).

$$PA = LU.$$

Solving steps:

Multiply by P: $\mathbf{d} = P\mathbf{b}$

Forwardsolve: $L\mathbf{y} = \mathbf{d}$

Backsolve: $U\mathbf{x} = \mathbf{y}$

Eigenvalues and eigenvectors:

A nonzero vector $\mathbf{v}$ is an eigenvector and a number λ is its eigenvalue if

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Characteristic equation: $\det(A - \lambda I) = 0$

Equation of the eigenvector: $(A - \lambda I)\mathbf{v} = \mathbf{0}$

Residual for an approximate eigenvector-eigenvalue pair: $r = \|A\mathbf{v} - \lambda\mathbf{v}\|$

Complex eigenvalues:

Occur in conjugate pairs: $\lambda_{1,2} = \alpha \pm i\beta$

and eigenvectors must also come in conjugate pairs: $\mathbf{w} = \mathbf{u} \pm i\mathbf{v}$.

Vibrational modes:

Eigenvalues are frequencies squared. Eigenvectors represent modes.

Power Method:

- Repeatedly multiply $\mathbf{x}$ by A and divide by the element with the largest absolute value.
- The element of largest absolute value converges to largest absolute eigenvalue.
- The vector converges to the corresponding eigenvector.
- Convergence assured for a real symmetric matrix, but not for an arbitrary matrix, which may not have real eigenvalues at all.

Inverse Power Method:

- Apply power method to A^{-1} .
- Use solving rather than the inverse.
- If λ is an eigenvalue of A then $1/\lambda$ is an eigenvalue for A^{-1} .
- The eigenvectors for A and A^{-1} are the same.

Symmetric and Positive definite:

- Symmetric: $A = A'$.
- If A is symmetric its eigenvalues are real.
- Positive definite: $A\mathbf{x} \cdot \mathbf{x} > 0$.
- If A is positive definite, then its eigenvalues are positive.

QR method (Not covered in MATH 3600 at Ohio):

- Transform A into H the Hessenberg form of A .
- Decompose H in QR .
- Multiply Q and R together in reverse order to form a new H .
- Repeat
- The diagonal of H will converge to the eigenvalues of A .

Matlab**Matrix arithmetic:**

```

A = [ 1  3 -2 5 ; -1 -1 5 4 ; 0 1 -9  0] ..... Manually enter a matrix.
u = [ 1  2  3  4] '
A*u
B = [3 2 1; 7 6 5; 4 3 2]
B*A ..... multiply B times A.
2*A ..... multiply a matrix by a scalar.
A + A ..... add matrices.
A + 3 ..... add 3 to every entry of a matrix.
B.*B ..... component-wise multiplication.
B.^3 ..... component-wise exponentiation.

```

Special matrices:

```

I = eye(3) ..... identity matrix
D = ones(5,5)
O = zeros(10,10)
C = rand(5,5) ..... random matrix with uniform distribution in [0,1].
C = randn(5,5) ..... random matrix with normal distribution.
hilb(6)
pascal(5)

```

General matrix commands:

```

size(C) ..... gives the dimensions ( $m \times n$ ) of  $A$ .
norm(C) ..... gives the norm of the matrix.
det(C) ..... the determinant of the matrix.
max(C) ..... the maximum of each row.
min(C) ..... the minimum in each row.
sum(C) ..... sums each row.
mean(C) ..... the average of each row.
diag(C) ..... just the diagonal elements.
inv(C) ..... inverse of the matrix.
C' ..... transpose of the matrix.

```

Matrix decompositions: $[L \ U \ P] = \text{lu}(C)$ $[Q \ R] = \text{qr}(C)$ $H = \text{hess}(C)$ transform into a Hessian tri-diagonal matrix, which has the same eigenvalues as A .

Part III

Functions and Data

©Copyright, Todd Young and Martin Mohlenkamp, Department of Mathematics, Ohio University, 2026

Lecture 19

Polynomial and Spline Interpolation

A Chemical Reaction

In a chemical reaction the concentration level y of the product at time t was measured every half hour. The following results were found:

t	0	.5	1.0	1.5	2.0
y	0	.19	.26	.29	.31

We can input this data into MATLAB as

```
>> t1 = 0:.5:2  
>> y1 = [ 0 .19 .26 .29 .31 ]
```

and plot the data with

```
>> plot(t1,y1)
```

MATLAB automatically connects the data with line segments, so the graph has corners. What if we want a smoother graph? Try

```
>> plot(t1,y1,'*')
```

which will produce just asterisks at the data points. Next click on **Tools**, then click on the **Basic Fitting** option. This should produce a small window with several fitting options. Begin clicking them one at a time, clicking them off before clicking the next. Which ones produce a good-looking fit? You should note that the spline, the shape-preserving interpolant, and the 4th degree polynomial produce very good curves. The others do not.

Polynomial Interpolation

Suppose we have n data points $\{(x_i, y_i)\}_{i=1}^n$. A interpolant is a function $f(x)$ such that $y_i = f(x_i)$ for $i = 1, \dots, n$. The most general polynomial with degree d is

$$p_d(x) = a_d x^d + a_{d-1} x^{d-1} + \dots + a_1 x + a_0,$$

which has $d + 1$ coefficients. A polynomial interpolant with degree d thus must satisfy

$$y_i = p_d(x_i) = a_d x_i^d + a_{d-1} x_i^{d-1} + \dots + a_1 x_i + a_0 \quad \text{for } i = 1, \dots, n.$$

This system is a linear system in the unknowns $a_0, \dots, a_n$ and *solving linear systems is what computers do best*. If $n = d + 1$, then the system of equations has n equations and n unknowns, so in general there is a

unique solution. This is the case in the example above: there are 5 data points so there is exactly one 4th degree polynomial that interpolates the data.

If $n < d + 1$ then the system is underdetermined and so in general will have an infinite number of solutions. When we tried to use a 5th or higher degree polynomial above, MATLAB returned a warning that the polynomial is not unique since “degree $\geq$ number of data points”.

If the data $\{(x_i, y_i)\}_{i=1}^n$ has repeated x -values with distinct y -values, then the system of equations is inconsistent and there will be no solution. If $n > d + 1$ then the system has more equations than unknowns, so it is overdetermined and in general will have no solution. In these cases MATLAB does produce a function, but it does not satisfy $p(x_i) = y_i$ for $i = 1, \dots, n$ and so is not an interpolant. Instead it is a least-squares fit, which we will study in Lecture 20.

Predicting the future?

Suppose we want to use the data to extrapolate into the future. Set the plot to the 4th degree polynomial. Then click the **Edit** button and select the **Axes Properties** option. A box should appear that allows you to adjust the domain of the x axes. Change the upper limit of the x -axis from 2 to 4. Based on the 4th degree polynomial, what will the chemical reaction do in the future? Is this reasonable?

Next change from 4th degree polynomial to spline interpolant. According to the spline, what will the chemical reaction do in the future? Try the shape-preserving interpolant also.

From our (limited) knowledge of chemical reactions, what should be the behavior as time goes on? It should reach a limiting value (chemical equilibrium). Could we use the data to predict this equilibrium value? Yes, we could and it is done all the time in many different contexts, but to do so we need to know that there is an equilibrium to predict. This requires that we understand the chemistry of the problem. Thus we have the following principle: To *extrapolate* beyond the data, one must have some knowledge of the process.

More data

Generally one would think that more data is better. Input the following data vectors:

```
>> t2 = [ 0   .1   .4   .5   .6   1.0   1.4  1.5  1.6  1.9  2.0]
>> y2 = [ 0   .06  .17  .19  .21  .26   .29  .29  .30  .31  .31]
```

There are 11 data points, so a 10-th degree polynomial will fit the data. However, this does not give a good graph. Thus: **Polynomial interpolation is better for small data sets.**

A challenging data set

Input the following data set:

```
>> x = -4:1:5
>> y = [ 0 0 0 1 1 1 0 0 0 0]
```

and plot it:

» `plot(x,y,'*')`

There are 10 data points, so there is a unique 9 degree polynomial that fits the data. Under **Tools** and **Basic Fitting** select the 9th degree polynomial fit. How does it look? De-select the 9th degree polynomial and select the spline interpolant. This should produce a much more satisfactory graph and the shape-preserving spline should be even better.

The idea of a spline

The general idea of a spline is this: on each interval between data points, represent the graph with a simple function. The simplest spline is something very familiar to you; it is obtained by connecting the data with lines. Since linear is the most simple function of all, linear interpolation is the simplest form of spline. The next simplest function is quadratic. If we put a quadratic function on each interval then we should be able to make the graph a lot smoother. If we were really careful then we should be able to make the curve smooth at the data points themselves by matching up the derivatives. This can be done and the result is called a quadratic spline. Using cubic functions or 4th degree functions should be smoother still. So, where should we stop? There is an almost universal consensus that *cubic* is the optimal degree for splines and so we focus the rest of the lecture on cubic splines.

Cubic spline

Again, the basic idea of the cubic spline is that we represent the function by a different cubic function on each interval between data points. That is, if there are n data points, then the spline $S(x)$ is the function

$$S(x) = \begin{cases} C_1(x), & x_0 \leq x \leq x_1 \\ C_i(x), & x_{i-1} \leq x \leq x_i \\ C_n(x), & x_{n-1} \leq x \leq x_n \end{cases}$$

where each C_i is a cubic function. The most general cubic function has the form

$$C_i(x) = a_i + b_i x + c_i x^2 + d_i x^3.$$

To determine the spline we must determine the coefficients, a_i , b_i , c_i , and d_i for each i . Since there are n intervals, there are $4n$ coefficients to determine. First we require that the spline interpolate by requiring

$$C_i(x_{i-1}) = y_{i-1} \quad \text{and} \quad C_i(x_i) = y_i,$$

at every data point. In other words,

$$a_i + b_i x_{i-1} + c_i x_{i-1}^2 + d_i x_{i-1}^3 = y_{i-1} \quad \text{and} \quad a_i + b_i x_i + c_i x_i^2 + d_i x_i^3 = y_i.$$

Notice that there are $2n$ of these conditions. Then to make $S(x)$ as smooth as possible we require

$$\begin{aligned} C'_i(x_i) &= C'_{i+1}(x_i) \quad \text{and} \\ C''_i(x_i) &= C''_{i+1}(x_i) \end{aligned}$$

at all the internal points, i.e. $x_1, x_2, x_3, \dots, x_{n-1}$. In terms of the points these conditions can be written as

$$\begin{aligned} b_i + 2c_i x_i + 3d_i x_i^2 &= b_{i+1} + 2c_{i+1} x_i + 3d_{i+1} x_i^2 \quad \text{and} \\ 2c_i + 6d_i x_i &= 2c_{i+1} + 6d_{i+1} x_i. \end{aligned}$$

There are $2(n-1)$ of these conditions. Since each C_i is cubic, there are a total of $4n$ coefficients in the formula for $S(x)$. So far we have $4n-2$ equations, so we are 2 equations short of being able to determine all the coefficients. At this point we have to make a choice. The usual choice is to require

$$C_1''(x_0) = C_n''(x_n) = 0.$$

These are called *natural* or *simple* boundary conditions. The other common option is called *clamped* boundary conditions:

$$C_1'(x_0) = C_n'(x_n) = 0.$$

The terminology used here is obviously parallel to that used for beams. That is not the only parallel between beams and cubic splines. It is an interesting fact that a cubic spline is exactly the shape of a (linear) beam restrained to match the data by simple supports.

Note that the equations above are all linear equations with respect to the unknowns (coefficients). This feature makes splines easy to calculate since *solving linear systems is what computers do best*.

Exercises

19.1 You are given the following data:

```
>> t = [ 0   .1   .499   .5   .6   1.0   1.4   1.5   1.899   1.9   2.0 ]
>> y = [ 0   .06   .17   .19   .21   .26   .29   .29   .30   .31   .31 ]
```

- (a) Plot the data, using '*' at the data points, then try a polynomial fit of the correct degree to interpolate this number of data points.
 - What do you observe? Look closely at the data and identify the points that force the interpolating polynomial to have large derivative.
 - What would happen if the `t` value of 1.899 was rounded to 1.9? Is the problem of converting this data to its interpolating polynomial *well conditioned* or *badly conditioned*?
- (b) Plot the data along with a spline interpolant. How does this compare with the plot above? What is a way to make the plot better?

Lecture 20

Least Squares Fitting: Noisy Data

Suppose we have n data points $\{(x_i, y_i)\}_{i=1}^n$. In the previous section we learned about interpolation, where the fitting function $f(x)$ is required to satisfy $y_i = f(x_i)$ for $i = 1, \dots, n$. Very often data has a significant amount of noise, so it makes more sense to only ask for $y_i \approx f(x_i)$. The least squares fitting method produces such an f . The next illustration shows the effects noise can have and how least squares is used.

Traffic flow model

Suppose you are interested in the time it takes to travel on a certain section of highway for the sake of planning. According to theory, assuming up to a moderate amount of traffic, the time should be approximately

$$T(x) = ax + b$$

where b is the travel time when there is no other traffic and x is the current number of cars on the road (in hundreds). To determine the coefficients a and b you could run several experiments which consist of driving the highway at different times of day and also estimating the number of cars on the road using a counter. Of course both of these measurements will contain *noise*, i.e. random fluctuations.

We could simulate such data in MATLAB as follows:

```
>> x = 1:.1:6;  
>> T = .1*x + 1;  
>> Tn = T + .1*randn(size(x));  
>> plot(x, Tn, 'r')
```

The data should look like it lies on a line, but with noise. Click on the **Tools** button and choose **Basic fitting**. Then choose a **linear** fit. The resulting line should go through the data in what looks like a very reasonable way. Click on **show equations**. Compare the equation with $T(x) = .1x + 1$. The coefficients should be pretty close considering the amount of noise in the plot. Next, try to fit the data with a spline. The result should be ugly. We can see from this example that **splines are not suited to noisy data**.

How does MATLAB obtain a very nice line to approximate noisy data? The answer is a very standard numerical/statistical method known as *least squares*.

Least squares

The least squares method requires you to first select what type of f to consider (sometimes called a model) and what parameters it depends on. For example, you might choose

$$f(x) = a_1 \exp(a_2 x) + a_0 \quad \text{or} \quad f(x) = a_2 x^2 + a_1 x + a_0.$$

Given data $\{(x_i, y_i)\}_{i=1}^n$, we can then determine the error at each point as $e_i = y_i - f(x_i)$. To make f reasonable, we wish to simultaneously minimize all the errors $\{e_1, e_2, \dots, e_n\}$. There are many possible ways one could go about this, but the standard one is to try to minimize the *sum of the squares* of the errors. That is, we denote by $\mathcal{E}$ the sum of the squares

$$\mathcal{E} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - f(x_i))^2 .$$

Since f depends on some parameters, $\mathcal{E}$ is a function of those parameters. To minimize $\mathcal{E}$, we can compute the partial derivatives of it with respect to each of the parameters, set the results equal to zero, and solve the resulting system of equations.

Linear least squares

If we choose f to depend linearly on the parameters, then the resulting system of equations will be a linear system, which is easy to solve. This does not mean we have to choose f to be a line. For example, the function $f(x) = a_2x^2 + a_1x + a_0$ is a quadratic in x , but depends linearly on each of a_2 , a_1 , and a_0 . To illustrate the linear least squares process, we will go through it using this quadratic f .

The error is

$$\mathcal{E}(a_0, a_1, a_2) = \sum_{i=1}^n (y_i - (a_2x_i^2 + a_1x_i + a_0))^2 ,$$

which has partial derivatives

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial a_0} &= -2 \sum_{i=1}^n (y_i - (a_2x_i^2 + a_1x_i + a_0)) , \\ \frac{\partial \mathcal{E}}{\partial a_1} &= -2 \sum_{i=1}^n (y_i - (a_2x_i^2 + a_1x_i + a_0)) x_i , \quad \text{and} \\ \frac{\partial \mathcal{E}}{\partial a_2} &= -2 \sum_{i=1}^n (y_i - (a_2x_i^2 + a_1x_i + a_0)) x_i^2 . \end{aligned}$$

Setting these equal to zero and rearranging yields the linear system

$$\begin{pmatrix} \sum_{i=1}^n 1 & \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 & \sum_{i=1}^n x_i^3 \\ \sum_{i=1}^n x_i^2 & \sum_{i=1}^n x_i^3 & \sum_{i=1}^n x_i^4 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n y_i x_i \\ \sum_{i=1}^n y_i x_i^2 \end{pmatrix} .$$

The entries in the matrix are determined from simple formulas using the data. Since this is a linear system, it is easily solved. The process is quick and easily automated, which is one reason it is very standard. MATLAB's basic fitting tool uses this process to obtain a d degree polynomial fit whenever the number of data points is more than $d + 1$.

Drag coefficients

Drag due to air resistance is proportional to the square of the velocity, i.e. $d = kv^2$. In a wind tunnel experiment the velocity v can be varied by setting the speed of the fan and the drag can be measured directly (it is the force on the object). In this and every experiment some random noise will occur. The following sequence of commands replicates the data one might receive from a wind tunnel:

```
>> v = 0:1:60;
>> d = .1234*v.^2;
>> dn = d + .4*v.*randn(size(v));
>> plot(v,dn,'*')
```

The plot should look like a quadratic, but with some noise. Using the tools menu, add a quadratic fit and enable the “show equations” option. What is the coefficient of x^2 ? How close is it to 0.1234?

Note that whenever you select a polynomial in MATLAB with a degree less than $n - 1$ MATLAB will produce a least squares fit.

You will notice that the quadratic fit includes both a constant and linear term. We know from the physical situation that these should not be there; they are remnants of noise and the fitting process. Since we know the curve should be kv^2 , we can do better by employing that knowledge. For instance, we know that the graph of d versus v^2 should be a straight line. We can produce this easily:

```
>> z = v.^2;
>> plot(z,dn,'*')
```

By changing the independent variable from v to $z = v^2$ we produced a plot that looks like a line with noise. Add a linear fit. What is the linear coefficient? This should be closer to 0.1234 than using a quadratic fit.

The second fit still has a constant term, which we know should not be there. If there was no noise, then at every data point we would have $k = d/v^2$. We can express this as a linear system $\mathbf{z}'\mathbf{k} = \mathbf{dn}'$, which is badly overdetermined since there are more equations than unknowns. Since there is noise, each point will give a different estimate for k . In other words, the overdetermined linear system is also inconsistent. When MATLAB encounters such systems, it automatically gives a least squares solution of the matrix problem, i.e. one that minimizes the sum of the squared errors, which is exactly what we want. To get the least squares estimate for k , do

```
>> k = z'\dn'
```

This will produce a number close to .1234.

Note that this is an application where we have physical knowledge. In this situation extrapolation would be meaningful. For instance we could use the plot to find the predicted drag at 80 mph.

Exercises

- 20.1 Find two tables of data in an engineering textbook or engineering website. Plot each (with '*' at data points) and decide if the data is best represented by a polynomial interpolant, spline interpolant, or least squares fit polynomial. Label the axes and include a title. Turn in the best plot of each. Indicate the source and meaning of the data.
- 20.2 The following table contains information from a chemistry experiment in which the concentration of a product was measured at one minute intervals:

Time	0	1	2	3	4	5	6	7
Concentration	3.033	3.306	3.672	3.929	4.123	4.282	4.399	4.527

Plot this data. Suppose it is known that this chemical reaction should follow the law: $c = a - b \exp(-0.2t)$. Following the example in the notes about the drag coefficients, change one of the variables so that the law is a linear function. Then plot the new variables and use the linear fit option to estimate a and b . What will be the eventual concentration? Finally, plot the graph of $a - b \exp(-0.2t)$ on the interval $[0,15]$ (as a solid curve), along with the data from the table (using '*').

Lecture 21

Integration: Left, Right and Trapezoid Rules

The Left and Right endpoint rules

In this section, we wish to approximate a definite integral

$$\int_a^b f(x) dx,$$

where $f(x)$ is a continuous function. In calculus we learned that integrals are (signed) areas and can be approximated by sums of smaller areas, such as the areas of rectangles. We begin by choosing points $\{x_i\}$ that subdivide $[a, b]$:

$$a = x_0 < x_1 < \dots < x_{n-1} < x_n = b.$$

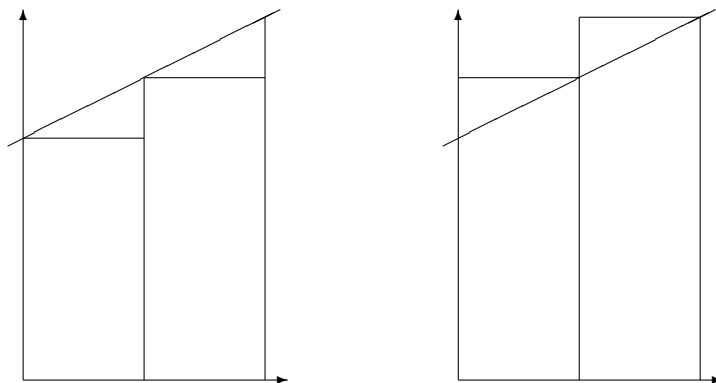
The subintervals $[x_{i-1}, x_i]$ determine the width Δx_i of each of the approximating rectangles. For the height, we learned that we can choose any height of the function $f(x_i^*)$ where $x_i^* \in [x_{i-1}, x_i]$. The resulting approximation is

$$\int_a^b f(x) dx \approx \sum_{i=1}^n f(x_i^*) \Delta x_i.$$

To use this to approximate integrals with actual numbers, we need to have a specific x_i^* in each interval. The two simplest (and worst) ways to choose x_i^* are as the left-hand point or the right-hand point of each interval. This gives concrete approximations which we denote by L_n and R_n given by

$$L_n = \sum_{i=1}^n f(x_{i-1}) \Delta x_i \quad \text{and} \quad R_n = \sum_{i=1}^n f(x_i) \Delta x_i.$$

```
function L = myleftsum(x,y)
    % produces the left sum from data input.
    % Inputs: x -- vector of the x coordinates of the partition
    %          y -- vector of the corresponding y coordinates
    % Output: returns the approximate integral
    n = length(x);
    L = 0;
    for i = 1:n-1
        % accumulate height times width
        L = L + y(i)*(x(i+1) - x(i));
    end
end
```

Figure 21.1: The left and right sums, L_n and R_n .

Often we can take $\{x_i\}$ to be *evenly spaced*, with each interval having the same width:

$$h = \frac{b - a}{n},$$

where n is the number of subintervals. If this is the case, then L_n and R_n simplify to

$$L_n = h \sum_{i=0}^{n-1} f(x_i) \quad \text{and} \quad (21.1)$$

$$R_n = h \sum_{i=1}^n f(x_i). \quad (21.2)$$

The foolishness of choosing left or right endpoints is illustrated in Figure 21.1. As you can see, for a very simple function like $f(x) = 1 + .5x$, each rectangle of L_n is too short, while each rectangle of R_n is too tall. This will hold for any increasing function. For decreasing functions L_n will always be too large while R_n will always be too small.

The Trapezoid rule

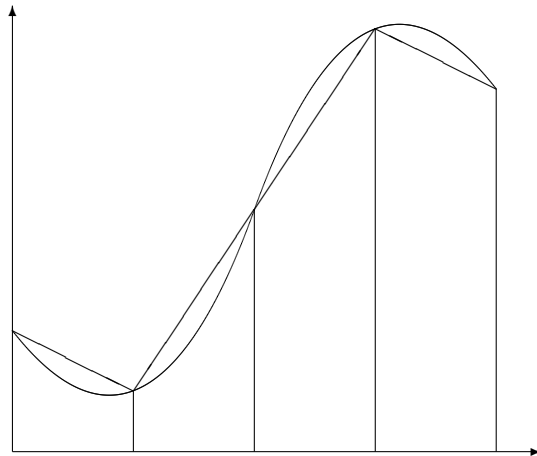
Knowing that the errors of L_n and R_n are of opposite sign, a very reasonable way to get a better approximation is to take an average of the two. We will call the new approximation T_n :

$$T_n = \frac{L_n + R_n}{2}.$$

This method also has a straight-forward geometric interpretation. On each subrectangle we are using

$$A_i = \frac{f(x_{i-1}) + f(x_i)}{2} \Delta x_i,$$

which is exactly the area of the *trapezoid* with sides $f(x_{i-1})$ and $f(x_i)$. We thus call the method the trapezoid method. See Figure 21.2. We can rewrite T_n as

Figure 21.2: The trapezoid rule, T_n .

$$T_n = \sum_{i=1}^n \frac{f(x_{i-1}) + f(x_i)}{2} \Delta x_i.$$

In the evenly spaced case, we can write this as

$$T_n = \frac{h}{2} (f(x_0) + 2f(x_1) + \dots + 2f(x_{n-1}) + f(x_n)). \quad (21.3)$$

Caution: The convention used here is to begin numbering the points at 0, i.e. $x_0 = a$; this allows n to be the number of subintervals and the index of the last point x_n . However, MATLAB's indexing convention begins at 1. Thus, when programming in MATLAB, the first entry in $\mathbf{x}$ will be x_0 , i.e. $\mathbf{x}(1) = x_0$ and $\mathbf{x}(\mathbf{n}+1) = x_n$.

If we are given data about the function, rather than a formula for the function, often the data are not evenly spaced. The following function program could then be used.

```
function T = mytrap(x,y)
    % Calculates the Trapezoid rule approximation of the integral from data
    % Inputs: x -- vector of the x coordinates of the partition
    %          y -- vector of the corresponding y coordinates
    % Output: returns the approximate integral
    n = length(x);
    T = 0;
    for i = 1:n-1
        % accumulate twice the signed area of the trapezoids
        T = T + (y(i) + y(i+1)) * (x(i+1) - x(i));
    end
    T = T/2; % correct for the missing 1/2
end
```

Using the Trapezoid rule for areas in the plane

Multi-variable Calculus tells us that you can calculate the area of a region R in the plane by calculating the line integral

$$A = - \oint_C y \, dx, \quad (21.4)$$

where C is the counter-clockwise curve around the boundary of the region. We can represent such a curve by consecutive points on it, i.e. $\bar{x} = (x_0, x_1, x_2, \dots, x_{n-1}, x_n)$, and $\bar{y} = (y_0, y_1, y_2, \dots, y_{n-1}, y_n)$. Since we are assuming the curve ends where it begins, we require $(x_n, y_n) = (x_0, y_0)$. Applying the trapezoid method to the integral (21.4) gives

$$A = - \sum_{i=1}^n \frac{y_{i-1} + y_i}{2} (x_i - x_{i-1}).$$

This formula then is the basis for calculating areas when coordinates of boundary points are known, but not necessarily formulas for the boundaries such as in a land survey.

In the following script, we can use this method to approximate the area of a unit circle using n points on the circle:

```
% Calculates pi using a trapezoid approximation of the unit circle.
format long
n = 10;           % evaluate points on the circle
t = linspace(0,2*pi,n+1);
x = cos(t);
y = sin(t);
plot(x,y)
A = 0             % accumulate (twice) the trapezoid area
for i = 1:n
    A = A - (y(i)+y(i+1))*(x(i+1)-x(i));
end
A = A/2 % correct for the missing 1/2
```

Vector Operations using Slicing and Summing

In the programs above we used loops to explicitly accumulate sums. For example, in `mytrap` we had

```
T = 0;
for i = 1:n-1
    T = T + .5*(y(i)+y(i+1))*(x(i+1) - x(i));
end
```

The alternative is to use vector operations by taking slices out of the vectors and using the `sum` function. We can replace the above code by

```
T = .5*sum( (y(1:n-1)+y(2:n)) .* (x(2:n)-x(1:n-1)) );
```

Generally, explicit loops are easier to understand but vector operations are more efficient and compact.

Exercises

- 21.1 For the integral $\int_1^2 \sqrt{x} dx$ calculate L_4 , R_4 , and T_4 with even spacing (by hand, but use a calculator and a lot of digits) using formulas (21.1), (21.2) and (21.3). Find the percentage error of these approximations, using the exact value.
- 21.2 Write a well-commented MATLAB **function** program **myints** whose inputs are f , a , b and n and whose outputs are L , R and T , the left, right and trapezoid integral approximations for f on $[a, b]$ with n subintervals. To make it efficient,
- take advantage of the fact that Δx_i is constant,
 - use `x = linspace(a,b,n+1)` to make the x values,
 - use `y = f(x)` to make the y values,
 - use slice and `sum` to add the 2nd to n th y entries once,
 - and then add on the missing terms to obtain the left, right and trapezoid approximations.

Change to `format long` and apply your program to the integral $\int_1^2 \sqrt{x} dx$. Compare with the results of the previous exercise. Also find L_{100} , R_{100} and T_{100} and the percentage errors of these approximations. Turn in the program and a brief summary of the results.

Lecture 22

Integration: Midpoint and Simpson's Rules

Midpoint rule

If we use the endpoints of the subintervals to approximate the integral, we run the risk that the values at the endpoints do not accurately represent the average value of the function on the subinterval. A point which is much more likely to be close to the average would be the midpoint of each subinterval. Using the midpoint in the sum is called the *midpoint rule*. On the i -th interval $[x_{i-1}, x_i]$ we will call the midpoint $\bar{x}_i$, i.e.

$$\bar{x}_i = \frac{x_{i-1} + x_i}{2}.$$

If $\Delta x_i = x_i - x_{i-1}$ is the length of each interval, then using midpoints to approximate the integral would give the formula

$$M_n = \sum_{i=1}^n f(\bar{x}_i) \Delta x_i.$$

For even spacing, $\Delta x_i = h = (b - a)/n$, and the formula is

$$M_n = h \sum_{i=1}^n f(\bar{x}_i) = h (\hat{y}_1 + \hat{y}_2 + \dots + \hat{y}_n), \quad (22.1)$$

where we define $\hat{y}_i = f(\bar{x}_i)$.

While the midpoint method is obviously better than L_n or R_n , it is not obvious that it is actually better than the trapezoid method T_n , but it is.

Simpson's rule

Consider Figure 22.1. If f is not linear on a subinterval, then it can be seen that the errors for the midpoint and trapezoid rules behave in a very predictable way, they have opposite sign. For example, if the function is concave up then T_n will be too high, while M_n will be too low. Thus it makes sense that a better estimate would be to average T_n and M_n . However, in this case we can do better than a simple average. The error will be minimized if we use a weighted average. To find the proper weight we take advantage of the fact that for a quadratic function the errors EM_n and ET_n are exactly related by

$$|EM_n| = \frac{1}{2} |ET_n|.$$

Thus we take the following weighted average of the two, which is called Simpson's rule:

$$S_{2n} = \frac{2}{3} M_n + \frac{1}{3} T_n.$$

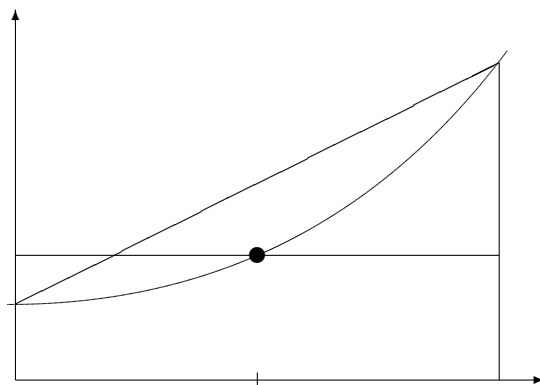


Figure 22.1: Comparing the trapezoid and midpoint method on a single subinterval. The function is concave up, in which case T_n is too high, while M_n is too low.

If we use this weighting on a quadratic function the two errors will exactly cancel.

Notice that we write the subscript as $2n$. That is because we usually think of $2n$ subintervals in the approximation; the n subintervals of the trapezoid are further subdivided by the midpoints. We can then number all the points using integers. If we number them this way we notice that the number of subintervals must be an even number.

The formula for Simpson's rule if the subintervals are evenly spaced is the following (with n intervals, where n is even):

$$S_n = \frac{h}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)) .$$

Note that if we are presented with data $\{x_i, y_i\}$ where the x_i points are evenly spaced with $x_{i+1} - x_i = \Delta x$, it is easy to apply Simpson's method:

$$S_n = \frac{h}{3} (y_0 + 4y_1 + 2y_2 + 4y_3 + \dots + 2y_{n-2} + 4y_{n-1} + y_n) . \quad (22.2)$$

Notice the pattern of the coefficients. The following program will produce these coefficients for n intervals, if n is an even number. Try it for $n = 6, 7, 100$.

```
function w = mysimpweights(n)
    % Computes the weights for Simpson's rule
    % Input:  n -- the number of intervals, must be even
    % Output: w -- a (column) vector with the weights, length n+1
    if rem(n,2) ~= 0
        error('n must be even for Simpsons rule')
    end
    w = 2*ones(n+1,1); % column vector, starts all 2's
    w(1) = 1; w(n+1) = 1; % set ends to 1's
    w(2:2:n)=4; % set even # entries to 4.
end
```

Simpson's rule is incredibly accurate. We will consider just how accurate in the next section. The one drawback is that the points used must either be evenly spaced, or at least the odd number points must lie exactly at the midpoint between the even numbered points. In applications where you can choose the spacing, this is not a problem. In applications such as experimental or simulated data, you might not have control over the spacing and then you cannot use Simpson's rule.

Error bounds

The trapezoid, midpoint, and Simpson's rules are all approximations. As with any approximation, before you can safely use it, you must know how good (or bad) the approximation might be. For these methods there are formulas that give upper bounds on the error. In other words, the worst case errors for the methods. These bounds are given by the following statements:

- Suppose f'' is continuous on $[a, b]$. Let $K_2 = \max_{x \in [a, b]} |f''(x)|$. Then the errors ET_n and EM_n of the Trapezoid and Midpoint rules, respectively, applied to $\int_a^b f dx$ satisfy

$$|ET_n| \leq K_2 \frac{b-a}{12} h^2 \quad \text{and}$$

$$|EM_n| \leq K_2 \frac{b-a}{24} h^2.$$

- Suppose $f^{(4)}$ is continuous on $[a, b]$. Let $K_4 = \max_{x \in [a, b]} |f^{(4)}(x)|$. Then the error ES_n of Simpson's rule applied to $\int_a^b f dx$ satisfies

$$|ES_n| \leq K_4 \frac{b-a}{180} h^4.$$

In practice K_2 and K_4 are themselves approximated from the values of f at the evaluation points.

The most important thing in these error bounds is the dependence on h . To emphasize this dependence, we sometimes use the **order notation** $O(\cdot)$. The trapezoid and midpoint method errors are $O(h^2)$, so the methods have order 2. The Simpson's rule error is $O(h^4)$, so it has order 4. If h is just moderately small, then there is a huge advantage with Simpson's method.

In MATLAB there is a built-in command for definite integrals: `integral(f,a,b)` where the `f` is a function and `a` and `b` are the endpoints. The command uses "adaptive Simpson quadrature", a form of Simpson's rule that checks its own accuracy and adjusts the grid size where needed. Here is an example of its usage:

```
>> f = @(x) x.^(1/3).*sin(x.^3)
>> I = integral(f,0,1)
```

Exercises

- 22.1 Using formulas (22.1) and (22.2), for the integral $\int_1^2 \sqrt{x} dx$ calculate M_4 and S_4 (by hand, but use a calculator and a lot of digits). Find the percentage error of these approximations, using the exact value. Compare with exercise 21.1.
- 22.2 Write a well-commented MATLAB **function** program `mymidpoint` that calculates the midpoint rule approximation for $\int f$ on the interval $[a, b]$ with n subintervals. The inputs should be f , a , b and n . Use your program on the integral $\int_1^2 \sqrt{x} dx$ to obtain M_4 and M_{100} . Compare these with exercise 22.1 and the true value of the integral.
- 22.3 Write a well-commented MATLAB **function** program `mysimpson` that calculates the Simpson's rule approximation for $\int f$ on the interval $[a, b]$ with n subintervals. It should call the program `mysimpweights` to produce the coefficients. Use your program on the integral $\int_1^2 \sqrt{x} dx$ to obtain S_4 and S_{100} . Compare these with exercise 22.1, exercise 22.2, and the true value of the integral.

Lecture 23

Plotting Functions of Two Variables

Functions on Rectangular Grids

Suppose you wish to plot a function $f(x, y)$ on the rectangle $a \leq x \leq b$ and $c \leq y \leq d$. The graph of a function of two variables is of course a three dimensional object. Visualizing the graph is often very useful.

For example, suppose you have a formula

$$f(x, y) = x \sin(xy)$$

and you are interested in the function on the region $0 \leq x \leq 5$, $\pi \leq y \leq 2\pi$. A way to plot this function in MATLAB would be the following sequence of commands:

```
>> f = @(x,y) x.*sin(x.*y)
>> [X,Y] = meshgrid(0:.1:5, pi:.01*pi:2*pi);
>> Z = f(X,Y)
>> mesh(X,Y,Z)
```

This will produce a 3-D plot that you can rotate by clicking on the rotate icon and then dragging with the mouse. Instead of the command `mesh`, you could use the command

```
>> surf(X,Y,Z)
```

The key command in this sequence is `[X Y] = meshgrid(a:h:b, c:k:d)`, which produces *matrices of x and y values* in `X` and `Y`. Enter:

```
>> size(X)
>> size(Y)
>> size(Z)
```

to see that each of these variables is a 101×51 matrix. To see the first few entries of `X` enter

```
>> X(1:6, 1:6)
```

and to see the first few values of `Y` type

```
>> Y(1:6, 1:6)
```

You should observe that the x values in `X` begin at 0 on the left column and increase from left to right. The y values on the other have start at π at the top and increase from top to bottom. Note that this arrangement is flipped from the usual arrangement in the x - y plane.

In the command `[X Y] = meshgrid(a:h:b,c:k:d)`, h is the increment in the x direction and k is the increment in the y direction. Often we will calculate

$$h = \frac{b-a}{m} \quad \text{and} \quad k = \frac{d-c}{n},$$

where m is the number of *intervals* in the x direction and n is the number of intervals in the y direction. To obtain a good plot it is best if m and n can be set between 10 and 100.

A common way of visualizing a function of two variables is by a *Contour Plot*. In a contour plot we draw several *level curves* of the function, which are the curves at which the function is equal to a few values. A topographical map is an example of a contour plot. To produce a contour plot for the function $f(x, y)$ as above, since we have input the function itself and created a meshgrid on which we want to plot it, we simply input:

```
>> contour(X,Y,Z,10)
```

The optional number “10” specify how many contour curves to display.

For another example of `meshgrid`, `contour` and `mesh`, try the following and look at `X` and `Y`.

```
>> [X,Y] = meshgrid(0:.05:4,1:.02:2);
>> Z = (X+Y)./(1+X.^2+Y.^2);
>> contour(X,Y,Z,11)
>> mesh(X,Y,Z)
```

Scattered Data and Triangulation

Often we are interested in objects whose bases are not rectangular. For instance, data does not usually come arranged in a nice rectangular grid; rather, measurements are taken where convenient.

In MATLAB we can produce triangles for a region by recording the coordinates of the vertices and recording which vertices belong to each triangle. The following script program produces such a set of triangles:

```
% mytriangles
% Program to produce a triangulation.
% V contains vertices, which are (x,y) pairs
V = [ 1/2 1/2 ; 1 1 ; 3/2 1/2 ; .5 1.5 ; 0 0
      1 0 ; 2 0 ; 2 1 ; 1.5 1.5 ; 1 2
      0 2 ; 0 1]
% x, y are row vectors containing coordinates of vertices
x = V(:,1)';
y = V(:,2)';
% Assign the triangles using Delaunay's algorithm
T = delaunay(x,y)
```

You can plot the triangles using

```
>> trimesh(T,x,y)
```

You can also prescribe values (heights) at each vertex directly (say from a survey):

```
>> z1 = [ 2 3 2.5 2 1 1 .5 1.5 1.6 1.7 .9 .5 ];
```

or using a function:

```
>> f = @(x,y) abs(sin(x.*y)).^(3/2);
>> z2 = f(x,y);
```

The resulting profiles can be plotted:

```
>> trimesh(T,x,y,z1)
>> trisurf(T,x,y,z2)
```

Each row of the matrix **T** corresponds to a triangle, so **T(i,:)** gives triangle number **i**. The three corners of triangle number **i** are at indices **T(i,1)**, **T(i,2)**, and **T(i,3)**. So for example to get the *y*-coordinate of the second point of triangle number 5, enter

```
>> y(T(5,2))
```

To see other examples of regions defined by triangles, download **mywedge.m** and **mywasher.m** and run them. Each of these programs defines vectors **x** and **y** of *x* and *y* values of vertices and a matrix **T**. As before **T** is a list of sets of three integers. Each triple of integers indicates which vertices to connect in a triangle.

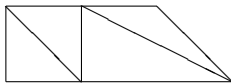
To plot a function, say $f(x,y) = x^2 - y^2$ on the washer figure try

```
>> mywasher
>> z = x.^2 - y.^2
>> trisurf(T,x,y,z)
```

Note again that this plot can be rotated using the icon and mouse.

Exercises

- 23.1 Plot the function $f(x,y) = \sin(x) e^{-x^2-y^2}$ on the rectangle $-3 \leq x \leq 3$, $-2 \leq y \leq 2$ using **meshgrid** and **mesh**. Make an appropriate choice of *m* and *n* and if necessary a rotation to produce a good plot. Calculate the *h* and *k* corresponding to your *m* and *n*. Turn in your plot and the calculation of *h* and *k*.



- 23.2 Modeling after **mywasher.m**, produce using integer coordinates for the vertices. Use the **axis** command to zoom out so the outside edges are clearly visible. Compute $z = 3x + y^2$ and plot the graph. Turn in your program and the plots.

Lecture 24

The Gradient and Max-Min Problems

The gradient of a function of multiple variables

If $f(x_1, x_2, \dots, x_n)$ is a real-valued function of n variables, i.e. $x_1, x_2, \dots, x_n$, then the *gradient* of f is the vector:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix},$$

where we use the vector notation $\mathbf{x} = (x_1, x_2, \dots, x_n)'$. For example, if $f(x_1, x_2) = x_1 e^{-x_1^2 - x_2^2 + \sin(x_1 x_2)}$, then

$$\nabla f(\mathbf{x}) = \begin{pmatrix} (1 + x_1(-2x_1 + \cos(x_1 x_2)x_2))e^{-x_1^2 - x_2^2 + \sin(x_1 x_2)} \\ x_1(-2x_2 + \cos(x_1 x_2)x_1)e^{-x_1^2 - x_2^2 + \sin(x_1 x_2)} \end{pmatrix}.$$

Note that the gradient is a vector-valued function of multiple variables. This type of function is often called a *vector field*.

The gradient vector has important features with geometric meaning.

- The gradient vector points in the direction in which f is increasing the most.
- The length of the gradient vector is equal to the derivative of f in that direction.
- The gradient vector is perpendicular to the level curve through that point.

We can visualize a gradient (or other vector field) using the Matlab command `quiver` (makes arrows).

```
>> [x1,x2] = meshgrid(-2:0.2:2);
>> z = x1.*exp(-x1.^2-x2.^2+sin(x1.*x2)); % function then partial derivatives
>> dz_dx1 = (1+x1.*(-2*x1+cos(x1.*x2).*x2)).*exp(-x1.^2-x2.^2+sin(x1.*x2));
>> dz_dx2 = x1.*(-2*x2+cos(x1.*x2).*x1).*exp(-x1.^2-x2.^2+sin(x1.*x2));
>> contour(x1,x2,z,10)
>> hold on
>> quiver(x1,x2,dz_dx1,dz_dx2)
>> hold off
```

Compare this to the surface plot:

```
>> figure()
>> surf(x1,x2,z)
```

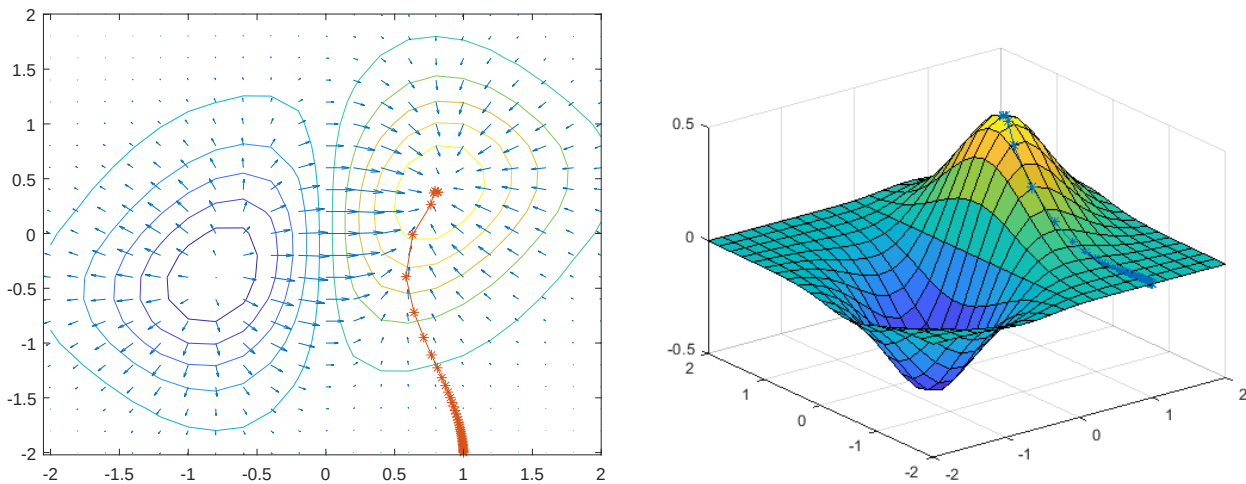


Figure 24.1: Gradient ascent on $f(x_1, x_2) = x_1e^{-x_1^2-x_2^2+\sin(x_1x_2)}$.

Optimization in multiple variables

If a differentiable function of multiple variables, f , has a maximum or minimum at point $\mathbf{x}^*$, then we will have:

$$\nabla f(\mathbf{x}^*) = \mathbf{0} \quad \text{the zero vector.}$$

Thus one way to locate max or min points for a specific function is to solve $\nabla f(\mathbf{x}^*) = \mathbf{0}$. This will often lead to systems of non-linear equations as we encountered in Lecture 13. For example, for $f(x_1, x_2) = x_1e^{-x_1^2-x_2^2+\sin(x_1x_2)}$, to attempt to find critical point we would need to try to solve:

$$(1 + x_1(-2x_1 + \cos(x_1x_2)x_2))e^{-x_1^2-x_2^2+\sin(x_1x_2)} = 0 \quad \text{and} \quad x_1(-2x_2 + \cos(x_1x_2)x_1)e^{-x_1^2-x_2^2+\sin(x_1x_2)} = 0.$$

That, obviously, is going to be very hard and in fact is impossible using algebra. Thus we will have to use numerical methods to find any maximum or minimum points. The multiple variable Newton's method that we learned in Lecture 13 would be one option. In the next section we introduce another common algorithm.

The gradient ascent or descent method

We will take advantage of the fact that the gradient vector points in the direction of greatest increase in the function f . If f is a function of x and y , then we can think of the graph of $z = f(x, y)$ as a landscape and the gradient points “uphill”. That gives us a very handy way to find maximum points, just follow the gradient vectors as they will take you toward higher values of z . See Figure 24.1. Since the gradient vector field may change from point to point, we can only “follow” it numerically in finite steps. That is, starting from an initial guess $\mathbf{x}_0$, we generate a sequence of points by the formula

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \epsilon \nabla f(\mathbf{x}_n).$$

The hope then is that

$$\mathbf{x}_n \rightarrow \mathbf{x}^*.$$

The number ϵ is often called the “learning rate”. It should not be too small or too large. If it is too small the algorithm will be slow. If it is too large, the steps might skip over a critical point. There is ongoing research about how to choose it optimally.

Notice the similarities between this method and the Newton’s method for nonlinear systems in Lecture 13. In both cases we have a function of a vector and generate a sequence of vectors $\mathbf{x}_0, \mathbf{x}_1, \dots$.

Exercises

- 24.1 Find by hand the gradient of the function $f(x_1, x_2) = \sin(x_1) e^{-x_1^2 - x_2^2}$. Plot it on the rectangle $-3 \leq x_1 \leq 3$, $-2 \leq x_2 \leq 2$ using `meshgrid` and `quiver`. On the same figure, add the contour plot of f .
- 24.2 Write a well-commented **script** program that uses the gradient descent method to find a minimum point for the function $f(x_1, x_2) = \sin(x_1) e^{-x_1^2 - x_2^2}$, starting from $\mathbf{x}_0 = [-1; -2]$. Compare with the plot in exercise 24.1 to check that it worked. (Hint: Review the program `mymultnewton` in Lecture 13 for examples of making functions of vectors.)

Lecture 25

Double Integrals for Rectangles

The center point method

Suppose that we need to find the integral of a function, $f(x, y)$, on a rectangle

$$R = \{(x, y) : a \leq x \leq b, c \leq y \leq d\}.$$

Calculus tells us to do this by an iterated integral

$$I = \iint_R f \, dA = \int_a^b \int_c^d f(x, y) \, dy \, dx = \int_c^d \int_a^b f(x, y) \, dx \, dy.$$

For instance, if R is the rectangle $0 \leq x \leq 2$, $1 \leq y \leq 3$, then

$$\begin{aligned} \iint_R x^2 y \, dA &= \int_0^2 \left(\int_1^3 x^2 y \, dy \right) dx \\ &= \int_0^2 x^2 \left(\frac{y^2}{2} \Big|_1^3 \right) dx \\ &= \int_0^2 x^2 \left(\frac{9}{2} - \frac{1}{2} \right) dx \\ &= 4 \int_0^2 x^2 \, dx \\ &= 4 \cdot \frac{8}{3} = \frac{32}{3} = 10.66\dots \end{aligned}$$

Calculus also tells us that the integral is the limit of the Riemann sums of the function as the size of the subrectangles goes to zero. This means that the Riemann sums are approximations of the integral, which is precisely what we need for numerical methods.

For a rectangle R , we begin by subdividing into smaller subrectangles $\{R_{ij}\}$, in a systematic way. We will divide $[a, b]$ into m subintervals and $[c, d]$ into n subintervals. Then R_{ij} will be the “intersection” of the i -th subinterval in $[a, b]$ with the j -th subinterval of $[c, d]$. In this way the entire rectangle is subdivided into mn subrectangles, numbered as in Figure 25.1.

A Riemann sum using this subdivision would have the form

$$S = \sum_{i,j=1,1}^{m,n} f(x_{ij}^*) A_{ij} = \sum_{j=1}^n \left(\sum_{i=1}^m f(x_{ij}^*) A_{ij} \right),$$

d	R_{15}	R_{25}	R_{35}	R_{45}
	R_{14}	R_{24}	R_{34}	R_{44}
	R_{13}	R_{23}	R_{33}	R_{43}
	R_{12}	R_{22}	R_{32}	R_{42}
	R_{11}	R_{21}	R_{31}	R_{41}
c	a b			

Figure 25.1: Subdivision of the rectangle $R = [a, b] \times [c, d]$ into subrectangles R_{ij} . Note that the arrangement in the x - y plane is completely different from the convention in a matrix.

where $A_{ij} = \Delta x_i \Delta y_j$ is the area of R_{ij} , and x_{ij}^* is a point in R_{ij} . The theory of integrals tells us that if f is continuous, then this sum will converge to the same number, no matter how we choose x_{ij}^* . For instance, we could choose x_{ij}^* to be the point in the lower left corner of R_{ij} and the sum would still converge as the size of the subrectangles goes to zero. However, in practice we wish to choose x_{ij}^* in such a way to make S as accurate as possible even when the subrectangles are not very small. The obvious choice for the best point in R_{ij} would be the center point. The center point is most likely of all points to have a value of f close to the average value of f . If we denote the center points by c_{ij} , then the sum becomes

$$C_{mn} = \sum_{i,j=1,1}^{m,n} f(c_{ij}) A_{ij}.$$

Here

$$c_{ij} = \left(\frac{x_{i-1} + x_i}{2}, \frac{y_{i-1} + y_i}{2} \right).$$

Note that if the subdivision is evenly spaced then $\Delta x \equiv (b - a)/m$ and $\Delta y \equiv (d - c)/n$, and so in that case

$$C_{mn} = \frac{(b - a)(d - c)}{mn} \sum_{i,j=1,1}^{m,n} f(c_{ij}).$$

The four corners method

Another good idea would be to take the value of f not only at one point, but as the average of the values at several points. An obvious choice would be to evaluate f at all four corners of each R_{ij} then average those. If we note that the lower left corner is (x_i, y_j) , the upper left is (x_i, y_{j+1}) , the lower right is (x_{i+1}, y_i) and the upper right is (x_{i+1}, y_{i+1}) , then the corresponding sum will be

$$F_{mn} = \sum_{i,j=1,1}^{m,n} \frac{1}{4} (f(x_i, y_j) + f(x_i, y_{j+1}) + f(x_{i+1}, y_i) + f(x_{i+1}, y_{j+1})) A_{ij},$$

which we will call the *four-corners* method. If the subrectangles are evenly spaced, then we can simplify this expression. Notice that $f(x_i, y_j)$ gets counted multiple times depending on where (x_i, y_j) is located. For

instance if (x_i, y_j) is in the interior of R then it is the corner of 4 subrectangles. Thus the sum becomes

$$F_{mn} = \frac{A}{4} \left(\sum_{\text{corners}} f(x_i, y_j) + 2 \sum_{\text{edges}} f(x_i, y_j) + 4 \sum_{\text{interior}} f(x_i, y_j) \right),$$

where $A = \Delta x \Delta y$ is the area of the subrectangles. We can think of this as a weighted average of the values of f at the grid points (x_i, y_j) . The weights used are represented in the matrix

$$W = \begin{pmatrix} 1 & 2 & 2 & 2 & \cdots & 2 & 2 & 1 \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ 1 & 2 & 2 & 2 & \cdots & 2 & 2 & 1 \end{pmatrix}. \quad (25.1)$$

We could implement the four-corner method by forming a matrix (f_{ij}) of f values at the grid points, then doing entry-wise multiplication of the matrix with the weight matrix. Then the integral would be obtained by summing all the entries of the resulting matrix and multiplying that by $A/4$. The formula would be

$$F_{mn} = \frac{(b-a)(d-c)}{4mn} \sum_{i,j=1,1}^{m+1,n+1} W_{ij} f(x_i, y_j).$$

Notice that the four-corners method coincides with applying the trapezoid rule in each direction. Thus it is in fact a *double trapezoid* rule.

The double Simpson method

The next improvement one might make would be to take an average of the center point sum C_{mn} and the four corners sum F_{mn} . However, a more standard way to obtain a more accurate method is the Simpson double integral. It is obtained by applying Simpson's rule for single integrals to the iterated double integral. The resulting method requires that both m and n be even numbers and the grid be evenly spaced. If this is the case we sum up the values $f(x_i, y_j)$ with weights represented in the matrix

$$W = \begin{pmatrix} 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \end{pmatrix}. \quad (25.2)$$

The sum of the weighted values is multiplied by $A/9$ and the formula is

$$S_{mn} = \frac{(b-a)(d-c)}{9mn} \sum_{i,j=1,1}^{m+1,n+1} W_{ij} f(x_i, y_j).$$

MATLAB has a built in command for double integrals on rectangles: `integral2(f,a,b,c,d)`. Here is an example:

```

>> f = @(x,y) sin(x.*y)./sqrt(x+y)
>> I = integral2(f,0.5,1,0.5,2)

```

Note that here, as usual, operations are component-wise.

Below is a MATLAB function which will produce the matrix of weights needed for Simpson's rule for double integrals. It uses the function `mysimpweights` from Lecture 22.

```

function W = mydblsimpweights(m,n)
    % Return the matrix of weights for Simpson's rule for double integrals.
    % Inputs: m -- number of intervals in the row direction.
    %           must be even.
    %           n -- number of intervals in the column direction.
    %           must be even.
    % Output: W -- a (m+1)x(n+1) matrix of the weights
    if rem(m,2)~=0 || rem(n,2)~=0
        error('m and n must be even for Simpsons rule')
    end
    % get 1-dimensional weights
    u = mysimpweights(m);
    v = mysimpweights(n);
    W = u*v';
end

```

Exercises

- 25.1 Download the program `mylowerleft.m` and test it. Find and fix the bug in it. Modify it to make a new program `mycenter` that does the center point method. Implement the change by changing the “`meshgrid`” to put the grid points at the centers of the subrectangles. Look at the mesh plot produced to make sure the program is putting the grid where you want it. Use both programs to approximate the integral

$$\int_0^2 \int_1^5 \sqrt{xy} \, dy \, dx,$$

using $(m,n) = (10,18)$. Evaluate this integral exactly (by hand) and compare the accuracy of the two programs.

- 25.2 Write a well-commented MATLAB **function** program `mydblsimp` that computes the Simpson's rule approximation. Let it call the program `mydblsimpweights` (above) to make the weight matrix, w (25.2). Check the program on the integral in the previous problem using $(m,n) = (10,18)$ and $(m,n) = (100,100)$.
- 25.3 Using `mysimpweights` and `mydblsimpweights` as models make well-commented MATLAB **function** programs `mytrapweights` and `mydbltrapweights` that will produce the weights for the trapezoid rule and the weight matrix for the four corners (double trapezoid) method (25.1). Use it to approximate the integral in the previous problem using $(m,n) = (5,6)$ and $(m,n) = (100,100)$. Turn in the programs, the weights for a 5×6 grid, and the results of your tests on the integral.

Lecture 26

Double Integrals for Non-rectangles

In the previous lecture we considered only integrals over rectangular regions. In practice, regions of interest are rarely rectangles and so in this lecture we consider two strategies for evaluating integrals over other regions.

Calculus tells us that if the region can be described by simple functions, then we might be able to use iterated integrals. For instance, suppose that R is the region inside of a circle of radius 2. Since the boundary of that circle is given by $x^2 + y^2 = 2^2$, we could express the integral of $f(x, y)$ on this region by:

$$\int_{-2}^2 \int_{-\sqrt{4-x^2}}^{\sqrt{4-x^2}} f(x, y) dy dx.$$

Of course this integral may be complicated, even if $f(x, y)$ is simple. If $f(x, y) = x^2 y^2$, then

$$\begin{aligned} \int_{-2}^2 \int_{-\sqrt{4-x^2}}^{\sqrt{4-x^2}} x^2 y^2 dy dx &= \int_{-2}^2 x^2 \frac{y^3}{3} \Big|_{-\sqrt{4-x^2}}^{\sqrt{4-x^2}} dx \\ &= \frac{2}{3} \int_{-2}^2 x^2 (4 - x^2)^{\frac{3}{2}} dx. \end{aligned}$$

At this point we would need to do a trig substitution to find the value of the integral.

Redefining the function

One strategy is to redefine the function so that it is zero outside the region of interest, then integrate over a rectangle that includes the region.

For example, suppose we need to approximate the value of

$$I = \iint_T \sin^3(xy) dx dy$$

where T is the triangle with corners at $(0, 0)$, $(1, 0)$ and $(0, 2)$. Then we could let R be the rectangle $[0, 1] \times [0, 2]$ which contains the triangle T . Notice that the hypotenuse of the triangle has the equation $2x + y = 2$. Then make $f(x, y) = \sin^3(xy)$ if $2x + y \leq 2$ and $f(x, y) = 0$ if $2x + y > 2$. In MATLAB we can make this function with the command

```
>> f = @(x,y) sin(x.*y).^3.*(2*x + y <= 2)
```

In this command `<=` is a *logical* command. The term in parentheses is then a *logical statement* and is given the value 1 if the statement is true and 0 if it is false. We can then integrate the modified `f` on $[0, 1] \times [0, 2]$ using the command

```
>> I = integral2(f,0,1,0,2)
```

As another example, suppose we need to integrate the function $f(x, y) = 10 + (x - 1)(y - 2)$ inside the circle of radius 2 centered at $(1, 2)$. The equation for this circle is $(x - 1)^2 + (y - 2)^2 = 4$. Note that the inside of the circle is $(x - 1)^2 + (y - 2)^2 \leq 4$ and that the circle is contained in the rectangle $[-1, 3] \times [0, 4]$. Thus we can create the right function, plot it, and integrate it by

```
>> f = @(x,y) (10+(x-1).*(y-2)).*((x-1).^2 + (y-2).^2 <= 4)
>> [X Y] = meshgrid(-4:0.01:4, -3:0.01:5);
>> Z = f(X,Y);
>> mesh(X,Y,Z)
>> I = integral2(f, -1, 3, 0, 4)
```

Integration Based on Triangles

The second approach to integrating over non-rectangular regions is based on subdividing the region into triangles. Such a subdivision is called a *triangulation*. On regions where the boundary consists of line segments, this can be done exactly. Even on regions where the boundary contains curves, this can be done approximately. This is a very important idea for several reasons, the most important of which is that the finite elements method is based on it. Another reason this is important is that often the values of f are not given by a formula, but from data. For example, suppose you are surveying on a construction site and you want to know how much fill will be needed to bring the level up to the plan. You would proceed by taking elevations at numerous points across the site. However, if the site is irregularly shaped or if there are obstacles on the site, then you cannot make these measurements on an exact rectangular grid. In this case, you can use triangles by connecting your points with triangles. Many software packages will even choose the triangles for you (MATLAB will do it using the command `delaunay`).

The basic idea of integrals based on triangles is exactly the same as that for rectangles; the integral is approximated by a sum where each term is a value times an area

$$I \approx \sum_{i=1}^n f(x_i^*) A_i,$$

where n is the number of triangles, A_i is the area of the triangle and x^* a point in the triangle. However, rather than considering the value of f at just one point people often consider an average of values at several points. The most convenient of these is of course the corner points. We can represent this sum by

$$T_n = \sum_{i=1}^n \bar{f}_i A_i,$$

where $\bar{f}$ is the average of f at the corners.

If the triangle has vertices (x_1, y_1) , (x_2, y_2) and (x_3, y_3) , the formula for area is

$$A = \frac{1}{2} \left| \det \begin{pmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{pmatrix} \right|. \quad (26.1)$$

The function `mythreecorners.m` below implements this method.

```

function I = mythreecorners(f,V,T)
% Integrates a function based on a triangulation, using three corners
% Inputs: f -- the function to integrate
%         V -- the vertices.
%         Each row has the x and y coordinates of a vertex
%         T -- the triangulation.
%         Each row gives the indices of the three corners
% Output: the approximate integral
x = V(:,1); % extract x and y coordinates of all nodes
y = V(:,2);
I=0;          % start accumulator at 0
p = size(T,1); % get number of triangles
for i = 1:p   % loop through the triangles
    x1 = x(T(i,1)); % find coordinates of the three corners
    x2 = x(T(i,2));
    x3 = x(T(i,3));
    y1 = y(T(i,1));
    y2 = y(T(i,2));
    y3 = y(T(i,3));
    A = .5*abs(det([x1, x2, x3; y1, y2, y3; 1, 1, 1])); %find area
    z1 = f(x1,y1); % find values at the three corners
    z2 = f(x2,y2);
    z3 = f(x3,y3);
    zavg = (z1 + z2 + z3)/3; % average the values
    I = I + zavg*A; % accumulate integral
end
end

```

Another idea would be to use the center point (centroid) of each triangle. If the triangle has vertices (x_1, y_1) , (x_2, y_2) and (x_3, y_3) , then the centroid is given by the simple formulas

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3} \quad \text{and} \quad \bar{y} = \frac{y_1 + y_2 + y_3}{3}. \quad (26.2)$$

Exercises

- 26.1 a. Download the program `mywasher.m`. Plot $f(x, y) = \frac{x+y}{x^2+y^2}$ on the region produced by `mywasher.m` and use the program `mythreecorners.m` to calculate the integral of f on the washer. Is this accurate? How do you know?
- b. Download the program `mywedge.m`. Plot $g(x, y) = \sin(x) + \sqrt{y}$ on the region produced by `mywedge.m` use `mythreecorners.m` to calculate the integral of g on this wedge.
- 26.2 Modify the program `mythreecorners.m` to a new program `mycenters.m` that does the centerpoint method for triangles. Run the program on the region produced by `mywasher.m` with the function $f(x, y) = \frac{x+y}{x^2+y^2}$ and on the region produced by `mywedge.m` with the function $g(x, y) = \sin(x) + \sqrt{y}$.

Lecture 27

Numerical Differentiation

Approximating derivatives from data

Suppose that a variable y depends on another variable x , i.e. $y = f(x)$, but we only know the values of f at a finite set of points, e.g., as data from an experiment or a simulation:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

Suppose then that we need information about the derivative of $f(x)$. One obvious idea would be to approximate $f'(x_i)$ by the **Forward Difference**

$$f'(x_i) = y'_i \approx \frac{y_{i+1} - y_i}{x_{i+1} - x_i}.$$

This formula follows directly from the definition of the derivative in Calculus. An alternative would be to use a **Backward Difference**

$$f'(x_i) \approx \frac{y_i - y_{i-1}}{x_i - x_{i-1}}.$$

Since the errors for the forward difference and backward difference tend to have opposite signs, it would seem likely that averaging the two methods would give a better result than either alone. If the points are evenly spaced, i.e. $x_{i+1} - x_i = x_i - x_{i-1} = h$, then averaging the forward and backward differences leads to a symmetric expression called the **Central Difference**

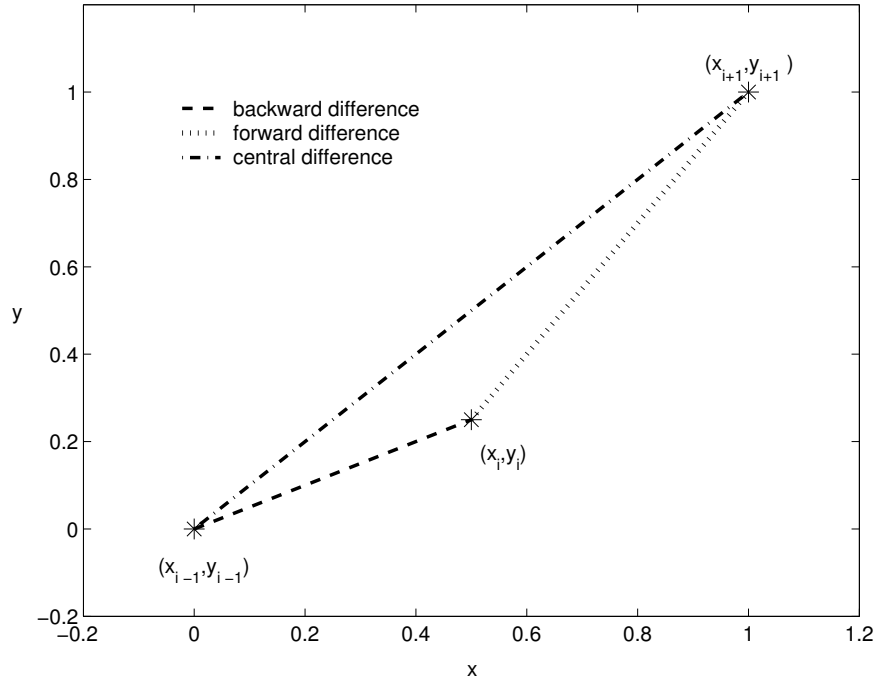
$$f'(x_i) = y'_i \approx \frac{y_{i+1} - y_{i-1}}{2h}.$$

See Figure 27.1 for an illustration of these three differences.

An example

The following code plots the function $y = \sin(x) + .5\sin(1.5x)$, its derivative, and the central difference approximation to the derivative.

```
>> x = linspace(0, 2*pi, 51);
>> h = x(2) - x(1);
>> mid = (x(1:end-1) + x(2:end))/2;
>> y = sin(x) + .5*sin(1.5*x);
>> dy1 = (y(2:end) - y(1:end-1))/h;
>> dy2 = cos(x) + .75*cos(1.5*x);
>> plot(x, y, 'k', mid, dy1, 'b*', x, dy2, 'r')
```

Figure 27.1: The three difference approximations of y'_i .

Errors of approximation

We can use Taylor polynomials to derive the accuracy of the forward, backward, and central difference formulas. For example the usual form of the Taylor polynomial with remainder (sometimes called Taylor's Theorem) is

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(c),$$

where c is some (unknown) number between x and $x+h$. Letting $x = x_i$, $x+h = x_{i+1}$ and solving for $f'(x_i)$ leads to

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} - \frac{h}{2}f''(c).$$

Notice that the quotient in this equation is exactly the forward difference formula. Thus the error of the forward difference is $-(h/2)f''(c)$ which means it is $O(h)$. Replacing h in the above calculation by $-h$ gives the error for the backward difference formula; it is also $O(h)$. For the central difference, the error can be found from the third degree Taylor polynomials with remainder

$$\begin{aligned} f(x_{i+1}) &= f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2}f''(x_i) + \frac{h^3}{3!}f'''(c_1) \quad \text{and} \\ f(x_{i-1}) &= f(x_i - h) = f(x_i) - hf'(x_i) + \frac{h^2}{2}f''(x_i) - \frac{h^3}{3!}f'''(c_2), \end{aligned}$$

where $x_i \leq c_1 \leq x_{i+1}$ and $x_{i-1} \leq c_2 \leq x_i$. Subtracting these two equations and solving for $f'(x_i)$ leads to

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} - \frac{h^2}{3!} \frac{f'''(c_1) + f'''(c_2)}{2}.$$

This shows that the error for the central difference formula is $O(h^2)$. Thus, central differences are significantly better and so: **It is best to use central differences whenever possible.**

There are also central difference formulas for higher order derivatives. These all have error of order $O(h^2)$:

$$\begin{aligned} f''(x_i) &= y_i'' \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}, \\ f'''(x_i) &= y_i''' \approx \frac{1}{2h^3} [y_{i+2} - 2y_{i+1} + 2y_{i-1} - y_{i-2}], \quad \text{and} \\ f^{(4)}(x_i) &= y_i^{(4)} \approx \frac{1}{h^4} [y_{i+2} - 4y_{i+1} + 6y_i - 4y_{i-1} + y_{i-2}]. \end{aligned}$$

Partial Derivatives

Suppose $u = u(x, y)$ is a function of two variables that we only know at grid points (x_i, y_j) . We will use the notation

$$u_{i,j} = u(x_i, y_j)$$

frequently throughout the rest of the lectures. We can suppose that the grid points are evenly spaced, with an increment of h in the x direction and k in the y direction. The central difference formulas for the partial derivatives would be

$$\begin{aligned} u_x(x_i, y_j) &\approx \frac{1}{2h} (u_{i+1,j} - u_{i-1,j}) \quad \text{and} \\ u_y(x_i, y_j) &\approx \frac{1}{2k} (u_{i,j+1} - u_{i,j-1}). \end{aligned}$$

The second partial derivatives are

$$\begin{aligned} u_{xx}(x_i, y_j) &\approx \frac{1}{h^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) \quad \text{and} \\ u_{yy}(x_i, y_j) &\approx \frac{1}{k^2} (u_{i,j+1} - 2u_{i,j} + u_{i,j-1}), \end{aligned}$$

and the mixed partial derivative is

$$u_{xy}(x_i, y_j) \approx \frac{1}{4hk} (u_{i+1,j+1} - u_{i+1,j-1} - u_{i-1,j+1} + u_{i-1,j-1}).$$

Caution: Notice that we have indexed u_{ij} so that as a matrix each row represents the values of u at a certain x_i and each column contains values at y_j . The arrangement in the matrix does not coincide with the usual orientation of the xy -plane.

Let's consider an example. Let the values of u at (x_i, y_j) be recorded in the matrix

$$(u_{ij}) = \begin{pmatrix} 5.1 & 6.5 & 7.5 & 8.1 & 8.4 \\ 5.5 & 6.8 & 7.8 & 8.3 & 8.9 \\ 5.5 & 6.9 & 9.0 & 8.4 & 9.1 \\ 5.4 & 9.6 & 9.1 & 8.6 & 9.4 \end{pmatrix} \quad (27.1)$$

Assume the indices begin at 1, i is the index for rows and j the index for columns. Suppose that $h = .5$ and $k = .2$. Then $u_y(x_2, y_4)$ would be approximated by the central difference

$$u_y(x_2, y_4) \approx \frac{u_{2,5} - u_{2,3}}{2k} \approx \frac{8.9 - 7.8}{2 \cdot 0.2} = 2.75.$$

The partial derivative $u_{xy}(x_2, y_4)$ is approximated by

$$u_{xy}(x_2, y_4) \approx \frac{u_{3,5} - u_{3,3} - u_{1,5} + u_{1,3}}{4hk} \approx \frac{9.1 - 9.0 - 8.4 + 7.5}{4 \cdot .5 \cdot .2} = -2.$$

Exercises

27.1 Suppose you are given the data in the following table.

x	0	5	10	15	20
y	0	19	26	29	31

- Give the forward, backward and central difference approximations of $f'(5)$.
- Give the central difference approximations for $f''(10)$, $f'''(10)$ and $f^{(4)}(10)$.

27.2 Suppose the position of a runner was recorded every half second and the results are given in the following table. Units of distance are meters.

t	0	.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0
y	0	.25	1	3	6	10	15	21	27	33	39	45	51

Using central differences everywhere possible, and a forward or backward difference where a central difference is not possible, calculate and plot the velocity and the acceleration of the runner as a function of time for $t=0$ to 6. You may do the calculations by hand or by MATLAB. Turn in your plots and calculations.

27.3 Suppose values of $u(x, y)$ at points (x_i, y_j) are given in the matrix (27.1). Suppose that $h = .1$ and $k = .3$. Approximate the following derivatives by central differences:

- $u_x(x_3, y_4)$
- $u_{xx}(x_2, y_2)$
- $u_{yy}(x_3, y_4)$
- $u_{xy}(x_3, y_3)$

Lecture 28

The Main Sources of Error

Truncation Error

Truncation error is defined as the error caused directly by an approximation method. For instance, all numerical integration methods are approximations and so there is error, even if the calculations are performed exactly. Numerical differentiation also has a truncation error, as will the differential equations methods we will study in Part IV, which are based on numerical differentiation formulas. There are two ways to minimize truncation error: (1) use a higher order method, and (2) use a finer grid so that points are closer together. Unless the grid is very small, truncation errors are usually much larger than roundoff errors. The obvious tradeoff is that a smaller grid requires more calculations, which in turn produces more roundoff errors and requires more running time.

Roundoff Error

Roundoff error always occurs when a finite number of digits are recorded after an operation. Fortunately, this error is extremely small. The standard measure of how small is called **machine epsilon**. It is defined as the smallest number that can be added to 1 to produce another number on the machine, i.e. if a smaller number is added the result will be rounded down to 1. In IEEE standard double precision (used by MATLAB and most serious software), machine epsilon is 2^{-52} or about 2.2×10^{-16} . A different, but equivalent, way of thinking about this is that the machine records 52 floating binary digits or about 15 floating decimal digits. Thus there are never more than 15 significant digits in any calculation. This of course is more than adequate for any application. However, there are ways in which this very small error can cause problems.

You can test that machine epsilon is 2^{-52} :

```
>> format long
>> (1 + 2^(-52)) - 1
>> (1 + 2^(-53)) - 1
```

MATLAB has a command that produces machine epsilon:

```
>> eps
```

To see an unexpected occurrence of round-off try

```
>> (2^52+1) - 2^52
>> (2^53+1) - 2^53
```

Thus roundoff isn't always small! It is just small compared with the scale of the numbers you are calculating. A number of magnitude 10^p will have roundoff of magnitude about $10^p \cdot 10^{-16} = 10^{p-16}$.

Loss of Precision (also called Loss of Significance)

Suppose we had some way to compute π that effectively did the calculation $(e \cdot 10^9 + \pi) - e \cdot 10^9$. Rounding everything to 16 digits, we are computing

$$(2718281828.459045 + 3.141592653589793) - 2718281828.459045.$$

The addition is performed first and the result rounded to 16 digits, giving

$$2718281831.600637 - 2718281828.459045.$$

Although roundoff caused some error, it is about a factor of 10^{-16} smaller than the numbers shown. In other words, all but perhaps the last digit shown is correct. Next the subtraction is performed, giving

$$0000000003.141592.$$

The leading zeros are not significant, so we lost 9 significant digits and have only 7 left. This type of error, where common leading significant digits cancel, is loss-of-precision (also called loss-of-significance) error. Computers will not display these leading zeros. Instead, the subtraction above yielded

$$3.141592025756836.$$

Although it is displayed with 16 digits, only the first 7 are correct. Usually, if you add two numbers of magnitude 10^p each with roundoff error 10^{p-16} , then the result is also of magnitude 10^p and the relative error due to roundoff is $10^{p-16}/10^p = 10^{-16}$. In this example, cancellation made the result of magnitude 10^{p-q} , so the relative error due to roundoff is $10^{p-16}/10^{p-q} = 10^{q-16}$, and so we lost $q = 9$ digits of precision.

This type of *loss of precision* can happen by accident, with catastrophic results, if you are not careful. For example in $f'(x) \approx (f(x+h) - f(x))/h$ you will lose precision when h gets too small. Try

```
>> format long
>> format compact
>> f = @(x) x^2
>> for i = 1:30
>>     h = 10^(-i)
>>     df = (f(1+h)-f(1))/h
>>     relerr = (2-df)/2
>> end
```

At first the relative error decreases since truncation error is reduced. Then loss of precision takes over and the relative error increases to 1. This happens because when $f(1)$ and $f(1+h)$ become close, the subtraction “cancels” digits.

Bad Conditioning

We encountered bad conditioning in Part II, when we talked about solving linear systems. Bad conditioning means that the problem is unstable in the sense that small input errors can produce large output errors. This can be a problem in a couple of ways. First, the measurements used to get the inputs cannot be completely accurate. Second, the computations along the way have roundoff errors. Errors in the computations near the beginning especially can be magnified by a factor close to the condition number of the matrix. Thus what was a very small problem with roundoff can become a very big problem.

It turns out that matrix equations are not the only place where condition numbers occur. In any problem one can define the condition number as the maximum ratio of the relative errors in the output versus input, i.e.

$$\text{condition \# of a problem} = \max \left(\frac{\text{Relative error of output}}{\text{Relative error of inputs}} \right).$$

An easy example is solving a simple equation

$$f(x) = 0.$$

Suppose that f' is close to zero at the solution x^* . Then a very small change in f (caused perhaps by an inaccurate measurement of some of the coefficients in f) can cause a large change in x^* . It can be shown that the condition number of this problem is $1/f'(x^*)$.

Summary

Error type:	Whose fault is it?	How to mitigate it?
Truncation	the method	higher-order method or finer grid
Round-off	the computer	usually okay, higher precision arithmetic
Loss of Precision	the programmer	avoid cancellation of significant digits
Bad Conditioning	the problem	check answers, redesign if possible

Table 28.1: A summary of the main sources of error.

Exercises

28.1 Identify the (main) source of error in each case and propose a way to reduce this error if possible.

(a) If we do $(\sqrt{3})^2$ we should get 3, but if we check

```
>> mythree = (sqrt(3))^2
>> mythree-3
```

we find the error is `ans = -4.4409e-16`.

(b) Since it is a quarter of a circle of radius 2, we should have $\int_0^2 \sqrt{4-x^2} dx = \frac{1}{4}\pi 2^2 = \pi$. We try to use `mytrap` from Lecture 21 and do

```
>> x = 0:.2:2;
>> y = sqrt(4-x.^2);
>> mypi = mytrap(x,y)
>> mypi-pi
```

and find the error is `ans = -0.0371`.

28.2 ¹ The function $f(x) = (x-2)^9$ could be evaluated as written, or first expanded as $f(x) = x^9 - 18x^8 + \dots$ and then evaluated. To find the expanded version, type

```
>> syms x
>> expand((x-2)^9)
>> clear
```

To evaluate it without expansion, type

```
>> f1 = @(x) (x-2).^9
>> x = 1.92:.001:2.08;
>> y1 = f1(x);
>> plot(x,y1,'blue')
```

To do it with expansion, convert the symbolic expansion above to an anonymous function `f2` and then type

```
>> y2 = f2(x);
>> hold on
>> plot(x,y2,'red')
```

Carefully study the resulting graphs. Should the graphs be the same? Which is more correct? MATLAB does calculations using approximately 16 decimal places. What is the largest error in the graph, and how big is it relative to 10^{-16} ? Which source of error is causing this problem?

¹From *Numerical Linear Algebra* by L. Trefethen and D. Bau, SIAM, 1997.

Review of Part III

Methods and Formulas

Polynomial Interpolation:

An exact fit to the data.
For n data points it is a $n - 1$ degree polynomial.
Only good for very few, accurate data points.
The coefficients are found by solving a linear system of equations.

Spline Interpolation:

Fit a simple function between each pair of points.
Joining points by line segments is the most simple spline.
Cubic is by far the most common and important.
Cubic matches derivatives and second derivatives at data points.
Simply supported and clamped ends are available.
Good for more, but accurate points.
The coefficients are found by solving a linear system of equations.

Least Squares:

Makes a “close fit” of a simple function to all the data.
Minimizes the sum of the squares of the errors.
Good for noisy data.
The coefficients are found by solving a linear system of equations.

Interpolation vs. Extrapolation:

Polynomials, Splines and Least Squares are generally used for *Interpolation*, fitting between the data. *Extrapolation*, i.e. making fits beyond the data, is much more tricky. To make predictions beyond the data, you must have knowledge of the underlying process, i.e. what the function should be.

Numerical Integration:

Left Endpoint:

$$L_n = \sum_{i=1}^n f(x_{i-1})\Delta x_i$$

Right Endpoint:

$$R_n = \sum_{i=1}^n f(x_i) \Delta x_i.$$

Trapezoid Rule:

$$T_n = \sum_{i=1}^n \frac{f(x_{i-1}) + f(x_i)}{2} \Delta x_i.$$

Midpoint Rule:

$$M_n = \sum_{i=1}^n f(\bar{x}_i) \Delta x_i \quad \text{where} \quad \bar{x}_i = \frac{x_{i-1} + x_i}{2}.$$

Numerical Integration Rules with Even Spacing:

For even spacing: $\Delta x = \frac{b-a}{n}$ where n is the number of subintervals, then:

$$L_n = \Delta x \sum_{i=0}^{n-1} y_i = \frac{b-a}{n} \sum_{i=0}^{n-1} f(x_i),$$

$$R_n = \Delta x \sum_{i=1}^n y_i = \frac{b-a}{n} \sum_{i=1}^n f(x_i),$$

$$T_n = \Delta x (y_0 + 2y_1 + \dots + 2y_{n-1} + y_n) = \frac{b-a}{2n} (f(x_0) + 2f(x_1) + \dots + 2f(x_{n-1}) + f(x_n)),$$

$$M_n = \Delta x \sum_{i=1}^n \bar{y}_i = \frac{b-a}{n} \sum_{i=1}^n f(\bar{x}_i),$$

Simpson's rule:

$$\begin{aligned} S_n &= \Delta x (y_0 + 4y_1 + 2y_2 + 4y_3 + \dots + 2y_{n-2} + 4y_{n-1} + y_n) \\ &= \frac{b-a}{3n} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)). \end{aligned}$$

Area of a region:

$$A = - \oint_C y \, dx,$$

where C is the counter-clockwise curve around the boundary of the region. We can represent such a curve, by consecutive points on it, i.e. $\bar{x} = (x_0, x_1, x_2, \dots, x_{n-1}, x_n)$, and $\bar{y} = (y_0, y_1, y_2, \dots, y_{n-1}, y_n)$ with $(x_n, y_n) = (x_0, y_0)$. Applying the trapezoid method to the integral (21.4):

$$A = - \sum_{i=1}^n \frac{y_{i-1} + y_i}{2} (x_i - x_{i-1})$$

Accuracy of integration rules:

Right and Left endpoint are $O(\Delta x)$

Trapezoid and Midpoint are $O(\Delta x^2)$

Simpson is $O(\Delta x^4)$

Double Integrals on Rectangles:**Centerpoint:**

$$C_{mn} = \sum_{i,j=1,1}^{m,n} f(c_{ij}) A_{ij}.$$

where

$$c_{ij} = \left(\frac{x_{i-1} + x_i}{2}, \frac{y_{i-1} + y_i}{2} \right).$$

Centerpoint – Evenly spaced:

$$C_{mn} = \Delta x \Delta y \sum_{i,j=1,1}^{m,n} z_{ij} = \frac{(b-a)(d-c)}{mn} \sum_{i,j=1,1}^{m,n} f(c_{ij}).$$

Four corners:

$$F_{mn} = \sum_{i,j=1,1}^{m,n} \frac{1}{4} (f(x_i, y_j) + f(x_i, y_{j+1}) + f(x_{i+1}, y_i) + f(x_{i+1}, y_{j+1})) A_{ij},$$

Four Corners – Evenly spaced:

$$\begin{aligned} F_{mn} &= \frac{A}{4} \left(\sum_{\text{corners}} f(x_i, y_j) + 2 \sum_{\text{edges}} f(x_i, y_j) + 4 \sum_{\text{interior}} f(x_i, y_j) \right) \\ &= \frac{(b-a)(d-c)}{4mn} \sum_{i,j=1,1}^{m,n} W_{ij} f(x_i, y_j). \end{aligned}$$

where

$$W = \begin{pmatrix} 1 & 2 & 2 & 2 & \cdots & 2 & 2 & 1 \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 2 & 4 & 4 & 4 & \cdots & 4 & 4 & 2 \\ 1 & 2 & 2 & 2 & \cdots & 2 & 2 & 1 \end{pmatrix}.$$

Double Simpson:

$$S_{mn} = \frac{(b-a)(d-c)}{9mn} \sum_{i,j=1,1}^{m,n} W_{ij} f(x_i, y_j).$$

where

$$W = \begin{pmatrix} 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \end{pmatrix}.$$

Integration based on triangles:

- Triangulation: Dividing a region up into triangles.
- Triangles are suitable for odd-shaped regions.
- A triangulation is better if the triangles are nearly equilateral.

Three corners:

$$T_n = \sum_{i=1}^n \bar{f}_i A_i$$

where $\bar{f}$ is the average of f at the corners of the i -th triangle.

Area of a triangle with corners (x_1, y_1) , (x_2, y_2) , (x_3, y_3) :

$$A = \frac{1}{2} \left| \det \begin{pmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{pmatrix} \right|.$$

Centerpoint:

$$C = \sum_{i=1}^n f(\bar{x}_i, \bar{y}_i) A_i, \quad \text{with}$$

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3} \quad \text{and} \quad \bar{y} = \frac{y_1 + y_2 + y_3}{3}.$$

Finite Differences

Forward Difference:

$$f'(x_i) = y'_i \approx \frac{y_{i+1} - y_i}{x_{i+1} - x_i}.$$

Backward Difference:

$$f'(x_i) \approx \frac{y_i - y_{i-1}}{x_i - x_{i-1}}.$$

Central Difference:

$$f'(x_i) = y'_i \approx \frac{y_{i+1} - y_{i-1}}{2h}.$$

Higher order central differences:

$$\begin{aligned}
 f''(x_i) &= y_i'' \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}, \\
 f'''(x_i) &= y_i''' \approx \frac{1}{2h^3} [y_{i+2} - 2y_{i+1} + 2y_{i-1} - y_{i-2}], \\
 f^{(4)}(x_i) &= y_i^{(4)} \approx \frac{1}{h^4} [y_{i+2} - 4y_{i+1} + 6y_i - 4y_{i-1} + y_{i-2}].
 \end{aligned}$$

Partial Derivatives: Denote $u_{i,j} = u(x_i, y_j)$.

$$\begin{aligned}
 u_x(x_i, y_j) &\approx \frac{1}{2h} (u_{i+1,j} - u_{i-1,j}), \\
 u_y(x_i, y_j) &\approx \frac{1}{2k} (u_{i,j+1} - u_{i,j-1}), \\
 u_{xx}(x_i, y_j) &\approx \frac{1}{h^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}), \\
 u_{yy}(x_i, y_j) &\approx \frac{1}{k^2} (u_{i,j+1} - 2u_{i,j} + u_{i,j-1}), \\
 u_{xy}(x_i, y_j) &\approx \frac{1}{4hk} (u_{i+1,j+1} - u_{i+1,j-1} - u_{i-1,j+1} + u_{i-1,j-1}).
 \end{aligned}$$

Sources of error:

Truncation – the method is an approximation.

Roundoff – double precision arithmetic uses ≈ 15 significant digits.

Loss of precision – an amplification of roundoff error due to cancellations.

Bad conditioning – the problem is sensitive to input errors.

Error can build up after multiple calculations.

See Table 28.

Matlab**Data Interpolation:**

Use the plot command `plot(x,y,'*')` to plot the data.

Use the Basic Fitting tool to make an interpolation or spline.

If you choose an $n - 1$ degree polynomial with n data points the result will be the exact polynomial interpolation.

If you select a polynomial degree less than $n - 1$, then MATLAB will produce a least squares approximation.

Functions of 2 Variables and Meshgrids:

```
f = @(x,y) sin(x.*y)./sqrt(x+y) .....make an anonymous function of x and y.
[X Y] = meshgrid(-1:.01:2, 0:.01:3) ..... make a 2-d grid of points.
```

`Z = f(X,Y)` evaluate the function at all points on the grid.
`mesh(X,Y,Z)` or `surf(X,Y,Z)` plot the function on the grid.

Integration:

`integral(f,a,b)` Numerical integral of $f(x)$ on $[a, b]$.
`integral2(f,a,b,c,d)` Integral of $f(x, y)$ on $[a, b] \times [c, d]$.

Example:

```
>> f = @(x,y) sin(x.*y)/sqrt(x+y)
>> I = integral2(f,0,1,0,2)
```

MATLAB uses an advanced form of Simpson's method.

Integration over non-rectangles:

Redefine the function to be zero outside the region. For example:

```
>> f = @(x,y) sin(x.*y).^3.*(2*x + y <= 2)
>> I = integral2(f,0,1,0,2)
```

Integrates $f(x, y) = \sin^3(xy)$ on the triangle with corners $(0, 0)$, $(0, 2)$, and $(1, 0)$.

Triangles:

MATLAB stores triangulations as a matrix of vertices `V` and triangles `T`.

`T = delaunay(V)` (or `delaunay(x,y)`) Produces triangles from vertices.

`trimesh(T,x,y)`

`trimesh(T,x,y,z)`

`trisurf(T,x,y)`

`trisurf(T,x,y,z)`

Logical expressions

$(2*x + y \leq 2)$, $(x.^2+y.^2 \leq 1)$ are examples of logical expressions.

If a logical expression is true it is given the value 1. If false, then it is assigned the value 0.

Part IV

Differential Equations

©Copyright, Todd Young and Martin Mohlenkamp, Department of Mathematics, Ohio University, 2026

Lecture 29

Reduction of Higher Order Equations to Systems

The motion of a pendulum

Consider the motion of an ideal pendulum that consists of a mass m attached to an arm of length ℓ . If we ignore friction, then Newton's laws of motion tell us

$$m\ddot{\theta} = -\frac{mg}{\ell} \sin \theta,$$

where θ is the angle of displacement.

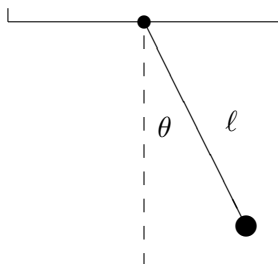


Figure 29.1: A pendulum.

If we also incorporate moving friction and sinusoidal forcing then the equation takes the form

$$m\ddot{\theta} + \gamma\dot{\theta} + \frac{mg}{\ell} \sin \theta = A \sin \Omega t.$$

Here γ is the coefficient of friction and A and Ω are the amplitude and frequency of the forcing. Usually, this equation would be rewritten by dividing through by m to produce

$$\ddot{\theta} + c\dot{\theta} + \omega \sin \theta = a \sin \Omega t, \quad (29.1)$$

where $c = \gamma/m$, $\omega = g/\ell$ and $a = A/m$.

This is a second order ODE because the second derivative with respect to time t is the highest derivative. It is nonlinear because it has the term $\sin \theta$ and which is a nonlinear function of the dependent variable θ . A solution of the equation would be a function $\theta(t)$. To get a specific solution we need side conditions. Because it is second order, 2 conditions are needed, and the usual conditions are initial conditions

$$\theta(0) = \theta_0 \quad \text{and} \quad \dot{\theta}(0) = v_0. \quad (29.2)$$

Converting a general higher order equation

All of the standard methods for solving ordinary differential equations are intended for first order equations. For this reason, it is inconvenient to solve higher order equations numerically. However, most higher-order differential equations that occur in applications can be converted to a *system* of first order equations and that is what is usually done in practice.

Suppose that an n -th order equation can be solved for the n -th derivative, i.e. it can be written in the form

$$x^{(n)} = f\left(t, x, \dot{x}, \ddot{x}, \dots, \frac{d^{n-1}x}{dt^{n-1}}\right).$$

Then it can be converted to a first-order system by this standard change of variables:

$$\begin{aligned} y_1 &= x \\ y_2 &= \dot{x} \\ &\vdots \\ y_n &= x^{(n-1)} = \frac{d^{n-1}x}{dt^{n-1}}. \end{aligned}$$

The resulting first-order system is

$$\begin{aligned} \dot{y}_1 &= \dot{x} = y_2 \\ \dot{y}_2 &= \ddot{x} = y_3 \\ &\vdots \\ \dot{y}_n &= x^{(n)} = f(t, y_1, y_2, \dots, y_n). \end{aligned}$$

In vector form this is simply $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ with $f_i(t, \mathbf{y}) = y_{i+1}$ for $i < n$ and $f_n(t, \mathbf{y}) = f(t, y_1, y_2, \dots, y_n)$.

For the example of the pendulum (29.1) the change of variables has the form

$$\begin{aligned} y_1 &= \theta \\ y_2 &= \dot{\theta}, \end{aligned}$$

and the resulting equations are

$$\begin{aligned} \dot{y}_1 &= y_2 \\ \dot{y}_2 &= -cy_2 - \omega \sin(y_1) + a \sin(\Omega t). \end{aligned} \tag{29.3}$$

In vector form this is

$$\dot{\mathbf{y}} = \begin{pmatrix} y_2 \\ -cy_2 - \omega \sin(y_1) + a \sin(\Omega t) \end{pmatrix}.$$

The initial conditions are converted to

$$\mathbf{y}(0) = \begin{pmatrix} y_1(0) \\ y_2(0) \end{pmatrix} = \begin{pmatrix} \theta_0 \\ v_0 \end{pmatrix}. \tag{29.4}$$

As stated above, the main reason we wish to change a higher order equation into a system of equations is that this form is convenient for solving the equation numerically. Most general software for solving ODEs

(including MATLAB) requires that the ODE be input in the form of a first-order system. In addition, there is a conceptual reason to make the change. In a system described by a higher order equation, knowing the position is not enough to know what the system is doing. In the case of a second order equation, such as the pendulum, one must know both the angle and the angular velocity to know what the pendulum is really doing. We call the pair $(\theta, \dot{\theta})$ the *state* of the system. Generally in applications the vector $\mathbf{y}$ is the state of the system described by the differential equation.

Using Matlab to solve a system of ODE's

In MATLAB there are several commands that can be used to solve an initial value problem for a system of differential equations. Each of these correspond to different solving methods. The standard one is `ode45`, which uses the algorithm “Runge-Kutta 4 5”. We will learn about this algorithm later.

To use `ode45` for a system, we have to input the vector function f that defines the system, the time span we want to consider and the initial value of the vector $\mathbf{y}$. Suppose we want to solve the pendulum system with $\omega = a = \Omega = 1$ and $c = .1$ for $t \in [0, 20]$ with initial condition $(\theta(0), \theta'(0)) = (1, -1.5)$. One way to use `ode45` is to enter

```
>> dy = @(t,y) [y(2); -.1*y(2) - sin(y(1)) + sin(t)]
>> [T Y] = ode45(dy, [0 20], [1; -1.5]);
```

Alternatively, we could create a function program

```
function dy = mypendulum(t,y)
    dy = [y(2); -.1*y(2) - sin(y(1)) + sin(t)]
end
```

and then enter

```
>> [T Y] = ode45(@mypendulum, [0 20], [1; -1.5]);
```

The output `T` contains times and `Y` contains values of the vector $\mathbf{y}$ at those times. Try

```
>> size(T)
>> T(1:10)
>> size(Y)
>> Y(1:10, :)
```

Since the first coordinate of the vector is the position (angle), we are mainly interested in its values:

```
>> theta = Y(:,1)
>> plot(T, theta)
```

In the next two sections we will learn enough about numerical methods for initial value problems to understand roughly how MATLAB produces this approximate solution.

Exercises

- 29.1 Consider the pendulum system but with no friction or forcing, i.e. $\gamma = A = 0$. What would equation (29.3) become in this case? Use the last example to solve the system with the initial condition $[\theta_0, 0]'$ for $\theta_0 = .1\pi$. Use the plot of the solution to find the frequency of the pendulum with this initial condition. Do the same for $\theta_0 = .5\pi$ and $.9\pi$. How does the frequency depend on the amplitude of a pendulum?

- 29.2 Transform the ODE

$$\ddot{x} + \ddot{x}^2 - 3\dot{x}^3 + \cos^2 x = e^{-t} \sin(3t)$$

into a first order system. Suppose the initial conditions for the ODE are $x(1) = 1$, $\dot{x}(1) = 2$, and $\ddot{x}(1) = 0$. Find a numerical solution of this IVP using `ode45` and plot the first coordinate (x). Try time intervals `[1 2]` and `[1 2.1]` and explain what you observe.

Lecture 30

Euler Methods

Numerical Solution of an IVP

Suppose we wish to numerically solve the initial value problem

$$\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(a) = \mathbf{y}_0, \quad (30.1)$$

on an interval of time $[a, b]$.

By a numerical solution, we must mean an approximation of the solution at a finite number of points, i.e.

$$(t_0, \mathbf{y}_0), (t_1, \mathbf{y}_1), (t_2, \mathbf{y}_2), \dots, (t_n, \mathbf{y}_n),$$

where $t_0 = a$ and $t_n = b$. The first of these points is exactly the initial value. If we take n steps as above, and the steps are evenly spaced, then the time change in each step is

$$h = \frac{b - a}{n}, \quad (30.2)$$

and the times t_i are given simply by $t_i = a + ih$. This leaves the most important part of finding a numerical solution: determining $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n$ in a way that is as consistent as possible with (30.1). To do this, first write the differential equation in the indexed notation

$$\dot{\mathbf{y}}_i \approx \mathbf{f}(t_i, \mathbf{y}_i), \quad (30.3)$$

and then replace the derivative $\dot{\mathbf{y}}$ by a difference. There are many ways we might carry this out and in the next section we study the simplest.

The Euler Method

The most straight forward approach is to replace $\dot{\mathbf{y}}_i$ in (30.3) by its forward difference approximation. This gives

$$\frac{\mathbf{y}_{i+1} - \mathbf{y}_i}{h} = \mathbf{f}(t_i, \mathbf{y}_i).$$

Rearranging this gives us a way to obtain $\mathbf{y}_{i+1}$ from $\mathbf{y}_i$ known as Euler's method:

$$\mathbf{y}_{i+1} = \mathbf{y}_i + h\mathbf{f}(t_i, \mathbf{y}_i). \quad (30.4)$$

With this formula, we can start from $(t_0, \mathbf{y}_0)$ and compute all the subsequent approximations $(t_i, \mathbf{y}_i)$. This is very easy to implement, as you can see from the following program (which can be downloaded as `myeuler.m`).

```

function [T , Y] = myeuler(f,tspan,y0,n)
% Solves dy/dt = f(t,y) with initial condition y(a) = y0
% on the interval [a,b] using n steps of Euler's method.
% Inputs: f -- a function f(t,y) that returns a column vector the
%           same length as y
%           tspan -- a vector [a,b] with the start and end times
%           y0 -- a column vector of the initial values, y(a) = y0
%           n -- number of steps to use
% Outputs: T -- a n+1 column vector containing the times
%           Y -- a (n+1) by d matrix where d is the length of y
%           Y(j,i) is the ith component of y at time T(j)
a = tspan(1); b = tspan(2); % parse starting and ending points
h = (b-a)/n; % step size
t = a; T = a; % t is the current time and T will record all times
y = y0; % y is the current variable values, as a column vector
Y = y0'; % Y will record the values at all steps, each in a row
for i = 1:n
    y = y + h*f(t,y); % Euler update of y.
    Y = [Y; y']; % y' becomes the next row in Y.
    t = t + h; % The next time.
    T = [T; t]; % Record t into T.
end
end

```

To use this program we need a function, such as the vector function for the pendulum:

```
>> dy = @(t,y)[y(2);-.1*y(2)-sin(y(1))+sin(t)]
```

Save this and then type

```
>> [T Y] = myeuler(dy,[0 20],[1;-1.5],5);
```

Here [0 20] is the time span you want to consider, [1;-1.5] is the initial value of the vector y and 5 is the number of steps. The output T contains times and Y contains values of the vector at the times. Try

```
>> size(T)
>> size(Y)
```

Since the first coordinate of the vector is the angle, we only plot its values:

```
>> theta = Y(:,1);
>> plot(T,theta)
```

In this plot it is clear that $n = 5$ is not adequate to represent the function. Type

```
>> hold on
```

then redo the above with 5 replaced by 10. Next try 20, 40, 80, and 200. As you can see the graph becomes increasingly better as n increases. We can compare these calculations with MATLAB's built-in function with the commands

```

>> [T Y] = ode45(dy,[0 20],[1;-1.5]);
>> theta = Y(:,1);
>> plot(T,theta,'r')

```

The problem with the Euler method

You can think of the Euler method as finding a linear approximate solution to the initial value problem on each time interval. An obvious shortcoming of the method is that it makes the approximation based on information at the beginning of the time interval only. This problem is illustrated well by the following IVP:

$$\ddot{x} + x = 0 \quad \text{with} \quad x(0) = 1 \quad \text{and} \quad \dot{x}(0) = 0. \quad (30.5)$$

You can easily check that the exact solution of this IVP is

$$x(t) = \cos(t).$$

If we make the standard change of variables

$$y_1 = x \quad \text{and} \quad y_2 = \dot{x},$$

then we get

$$\dot{y}_1 = y_2 \quad \text{and} \quad \dot{y}_2 = -y_1.$$

Then the solution should be $y_1(t) = \cos(t)$ and $y_2(t) = \sin(t)$. If we then plot the solution in the (y_1, y_2) plane, we should get exactly a unit circle. We can solve this IVP with Euler's method:

```

>> dy = @(t,y)[y(2);-y(1)]
>> [T Y] = myeuler(dy,[0 4*pi],[1;0],20)
>> y1 = Y(:,1);
>> y2 = Y(:,2);
>> plot(y1,y2)

```

As you can see the approximate solution goes far from the true solution. Even if you increase the number of steps, the Euler solution will eventually drift outward away from the circle because it does not take into account the curvature of the solution.

The Modified Euler Method

An idea which is similar to the idea behind the trapezoid method would be to consider f at both the beginning and end of the time step and take the average of the two. Doing this produces the Modified (or Improved) Euler method represented by the following equations:

$$\begin{aligned} \mathbf{k}_1 &= h\mathbf{f}(t_i, \mathbf{y}_i) \\ \mathbf{k}_2 &= h\mathbf{f}(t_i + h, \mathbf{y}_i + \mathbf{k}_1) \\ \mathbf{y}_{i+1} &= \mathbf{y}_i + \frac{1}{2}(\mathbf{k}_1 + \mathbf{k}_2). \end{aligned} \quad (30.6)$$

Here $\mathbf{k}_1$ captures the information at the beginning of the time step (same as Euler), while $\mathbf{k}_2$ is the information at the end of the time step.

A program that implements the Modified method can be downloaded as `mymodeuler.m`.

Test this program on the IVP above:

```
>> [T Ym] = mymodeuler(dy,[0 4*pi],[1;0],20)
>> ym1 = Ym(:,1);
>> ym2 = Ym(:,2);
>> plot(ym1,ym2)
```

You will find that the results are much better than for the plain Euler method.

Exercises

30.1 Download `myeuler.m` and `mymodeuler.m`.

(a) Type the following commands:

```
>> dy = @(t,y) sin(t)*cos(y);
>> hold on
>> [T Y] = myeuler(dy,[0,12],.1,20);
>> plot(T,Y)
```

Position the plot window so that it can always be seen and type

```
>> [T Y] = myeuler(dy,[0,12],.1,30);
>> plot(T,Y)
```

(You can use the up button to reduce typing.) Continue to increase the last number in the above until the graph stops changing (as far as you can see). Record this number and print the final graph. Type `hold off` and kill the plot window.

(b) Follow the same procedure using `mymodeuler.m`.

(c) Describe what you observed. In particular compare how fast the two methods converge as n is increased (h is decreased).

30.2 The equation of motion of a damped, unforced pendulum is

$$\ddot{\theta} + \frac{\gamma}{m}\dot{\theta} + \frac{g}{\ell}\sin\theta = 0.$$

The total energy of the pendulum is $E = m\ell^2\dot{\theta}^2/2 + mg\ell(1 - \cos(\theta))$. Suppose a pendulum has $m = 2$, $g = 9.81$, $\ell = 3$, and coefficient of friction $\gamma = 0.5$, all in SI units. Write a well-commented **script** program that uses the Modified Euler's method with $h = 0.01$ to simulate the movement of the pendulum from a starting position of $(\theta(0), \dot{\theta}(0)) = (.9\pi, 0)$ until the energy falls below 0.01 Joules (use a `while` loop). Record the steps in matrices `Y` and `T` and plot the motion. Turn in your program and plot.

Lecture 31

Higher Order Methods

The order of a method

For numerical solutions of an initial value problem there are two ways to measure the error. The first is the error of each step. This is called the Local Truncation Error or LTE. The other is the total error for the whole interval $[a, b]$. We call this the Global Truncation Error or GTE.

For the Euler method the LTE is of order $O(h^2)$, i.e. the error is comparable to h^2 . We can show this directly using Taylor's Theorem:

$$\mathbf{y}(t+h) = \mathbf{y}(t) + h\dot{\mathbf{y}}(t) + \frac{h^2}{2}\ddot{\mathbf{y}}(c)$$

for some c between t and $t+h$. In this equation we can replace $\dot{\mathbf{y}}(t)$ by $f(t, \mathbf{y}(t))$, which makes the first two terms of the right hand side be exactly the Euler method. The error is then $\frac{h^2}{2}\ddot{\mathbf{y}}(c)$ or $O(h^2)$. It would be slightly more difficult to show that the LTE of the modified Euler method is $O(h^3)$, an improvement of one power of h .

We can roughly get the GTE from the LTE by considering the number of steps times the LTE. For any method, if $[a, b]$ is the interval and h is the step size, then $n = (b-a)/h$ is the number of steps. Thus for any method, the GTE is one power lower in h than the LTE. Thus the GTE for Euler is $O(h)$ and for modified Euler it is $O(h^2)$.

By the **order** of a method, we mean the power of h in the GTE. Thus the Euler method is a 1st order method and modified Euler is a 2nd order method.

Fourth Order Runge-Kutta

The most famous of all IVP methods is the classic Runge-Kutta method of order 4:

$$\begin{aligned} \mathbf{k}_1 &= h\mathbf{f}(t_i, \mathbf{y}) \\ \mathbf{k}_2 &= h\mathbf{f}(t_i + h/2, \mathbf{y}_i + \mathbf{k}_1/2) \\ \mathbf{k}_3 &= h\mathbf{f}(t_i + h/2, \mathbf{y}_i + \mathbf{k}_2/2) \\ \mathbf{k}_4 &= h\mathbf{f}(t_i + h, \mathbf{y}_i + \mathbf{k}_3) \\ \mathbf{y}_{i+1} &= \mathbf{y}_i + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \end{aligned} \tag{31.1}$$

Notice that this method uses values of $\mathbf{f}(t, \mathbf{y})$ at 4 different points. In general a method needs n values of $\mathbf{f}$ to achieve order n . The constants used in this method and other methods are obtained from Taylor's Theorem. They are precisely the values needed to make all error terms cancel up to $h^{n+1}\mathbf{f}^{(n+1)}(c)/(n+1)!$.

Variable Step Size and RK45

If the order of a method is n , then the GTE is comparable to h^n , which means it is approximately Ch^n , where C is some constant. However, for different differential equations, the values of C may be very different. Thus it is not easy beforehand to tell how small h should be to get the error within a given tolerance. For instance, if the true solution oscillates very rapidly, we will obviously need a smaller step size than for a solution that is nearly constant.

How can a program then choose h small enough to produce the required accuracy? We also do not wish to make h much smaller than necessary, since that would increase the number of steps. To accomplish this a program tries an h and tests to see if that h is small enough. If not it tries again with a smaller h . If it is too small, it accepts that step, but on the next step it tries a larger h . This process is called **variable step size**.

Deciding if a single step is accurate enough could be accomplished in several ways, but the most common are called **embedded methods**. The Runge-Kutta 45 method, which is used in `ode45`, is an embedded method. In the RK45, the function f is evaluated at 5 different points. These are used to make a 5th order estimate $\mathbf{y}_{i+1}$. At the same time, 4 of the 5 values are used to also get a 4th order estimate. If the 4th order and 5th order estimates are close, then we can conclude that they are accurate. If there is a large discrepancy, then we can conclude that they are not accurate and a smaller h should be used.

To see variable step size in action, we will define and solve two different ODEs and solve them on the same interval. Create this script and run it:

```
% illustrates variable step size in RK45
dy1 = @(t,y) [-y(2);y(1)];           % create two ODE IVPs
dy2 = @(t,y) [-5*y(2);5*y(1)];
[T1 Y1] = ode45(dy1,[0 20],[1;0]);    % solve with ode45
[T2 Y2] = ode45(dy2,[0 20],[1;0]);
y1 = Y1(:,1);                         % extract position variables
y2 = Y2(:,1);
plot(T1,y1,'bx-')                     % plot both together
hold on
plot(T2,y2,'ro-')
size(T1)                              % print number of steps used
size(T2)
hold off
```

Why order matters

Many people would conclude on first encounter that the advantage of a higher order method would be that you can get a more accurate answer than for a lower order method. In reality, this is not quite how things work. In engineering problems, the accuracy needed is usually a given and it is usually not extremely high. Thus getting more and more accurate solutions is not very useful. So where is the advantage? Consider the following example.

Suppose that you need to solve an IVP with an error of less than 10^{-4} . If you use the Euler method, which has GTE of order $O(h)$, then you would need $h \approx 10^{-4}$. So you would need about $n \approx (b - a) \times 10^4$ steps to find the solution.

Suppose you use the second order, modified Euler method. In that case the GTE is $O(h^2)$, so you would need to use $h^2 \approx 10^{-4}$, or $h \approx 10^{-2}$. This would require about $n \approx (b - a) \times 10^2$ steps. That is a hundred times fewer steps than you would need to get the same accuracy with the Euler method.

If you use the RK4 method, then h^4 needs to be approximately 10^{-4} , and so $h \approx 10^{-1}$. This means you need only about $n \approx (b - a) \times 10$ steps to solve the problem, i.e. a thousand times fewer steps than for the Euler method.

Thus the real advantage of higher order methods is that they can run a lot faster at the same accuracy. This can be especially important in applications where one is trying to make real-time adjustments based on the calculations. Such is often the case in robots and other applications with dynamic controls.

Exercises

- 31.1 There is a Runge-Kutta 2 method, which is also known as the midpoint method. It is summarized by the following equations:

$$\begin{aligned} \mathbf{k}_1 &= h\mathbf{f}(t_i, \mathbf{y}_i) \\ \mathbf{k}_2 &= h\mathbf{f}(t_i + h/2, \mathbf{y}_i + \mathbf{k}_1/2) \\ \mathbf{y}_{i+1} &= \mathbf{y}_i + \mathbf{k}_2. \end{aligned} \tag{31.2}$$

- (a) Modify the program `mymodeuler` into a program `myRK2` that does the RK2 method.
- (b) Test `myRK2` and `mymodeuler` on the following IVP with time span $[0, 4\pi]$:

$$\ddot{x} + x = 0 \quad \text{with} \quad x(0) = 1 \quad \text{and} \quad \dot{x}(0) = 0.$$

Using `format long`, make a table with $\mathbf{y}_{n+1}$ for each of the two programs for $n = 10, 100$, and 1000 . Compute the difference between $\mathbf{y}_{n+1}$ and the true solution $\mathbf{y}(4\pi) = (1, 0)$.

Turn in the modified program and a summary of the results.

- 31.2 Consider the initial value problem

$$\frac{dy}{dt} = y(t^2 - 1), \quad y(0) = 1.$$

- (a) Find the exact solution by hand using Separation of Variables.
- (b) Solve it by hand using Euler's method with $h = 0.5$ on the interval $[0, 2]$.
- (c) Solve it using `ode45` on the interval $[0, 2]$.
- (d) Plot all three solutions on $[0, 2]$ in a single plot. Use a continuous curve for the exact solution and symbols at the data points for the approximate solutions (e.g. '*' for Euler, 'd' for `ode45`).
- (e) Rank steps (a) - (d) from easiest to hardest.

Turn in your hand work, the plot and your answer to (e).

Lecture 32

Multi-step Methods*

Exercises

32.1

Lecture 33

ODE Boundary Value Problems and Finite Differences

Steady State Heat and Diffusion

If we consider the movement of heat in a long thin object (like a metal bar), it is known that the temperature, $u(x, t)$, at a location x and time t satisfies the partial differential equation

$$u_t - u_{xx} = g(x, t), \quad (33.1)$$

where $g(x, t)$ is the effect of any external heat source. The same equation also describes the diffusion of a chemical in a one-dimensional environment. For example the environment might be a canal, and then $g(x, t)$ would represent how a chemical is introduced.

Sometimes we are interested only in the steady state of the system, supposing $g(x, t) = g(x)$ and $u(x, t) = u(x)$. In this case

$$u_{xx} = -g(x).$$

This is a linear second-order ordinary differential equation. We could find its solution exactly if $g(x)$ is not too complicated. If the environment or object we consider has length L , then typically one would have conditions on each end of the object, such as $u(0) = 0$, $u(L) = 0$. Thus instead of an initial value problem, we have a **boundary value problem** or **BVP**.

Beam With Tension

Consider a simply supported beam with modulus of elasticity E , moment of inertia I , a uniform load w , and end tension T (see Figure 33.1). If $y(x)$ denotes the deflection at each point x in the beam, then $y(x)$ satisfies the differential equation

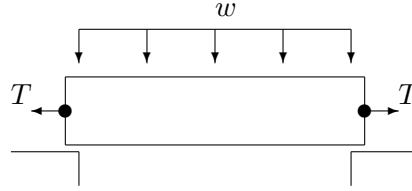
$$\frac{y''}{(1 + (y')^2)^{3/2}} - \frac{T}{EI}y = \frac{wx(L - x)}{2EI}, \quad (33.2)$$

with boundary conditions $y(0) = y(L) = 0$. This equation is nonlinear and there is no hope to solve it exactly. If the deflection is small then $(y')^2$ is negligible compared to 1 and the equation approximately simplifies to

$$y'' - \frac{T}{EI}y = \frac{wx(L - x)}{2EI}. \quad (33.3)$$

This is a linear equation and we can find the exact solution. We can rewrite the equation as

$$y'' - \alpha y = \beta x(L - x), \quad (33.4)$$

Figure 33.1: A simply supported beam with a uniform load w and end tension T .

where

$$\alpha = \frac{T}{EI} \quad \text{and} \quad \beta = \frac{w}{2EI}, \quad (33.5)$$

and then the exact solution is

$$y(x) = \frac{2\beta}{\alpha^2} \frac{e^{\sqrt{\alpha}L}}{e^{\sqrt{\alpha}L} + 1} e^{-\sqrt{\alpha}x} + \frac{2\beta}{\alpha^2} \frac{1}{e^{\sqrt{\alpha}L} + 1} e^{\sqrt{\alpha}x} + \frac{\beta}{\alpha} x^2 - \frac{\beta L}{\alpha} x + \frac{2\beta}{\alpha^2}. \quad (33.6)$$

Finite Difference Method – Linear ODE

A finite difference equation is an equation obtained from a differential equation by replacing the variables by their discrete versions and derivatives by difference formulas.

First we will consider equation (33.3). Suppose that the beam is a W12x22 structural steel I-beam. Then $L = 120$ in., $E = 29 \times 10^6$ lb./in.² and $I = 121$ in.⁴. Suppose that the beam is carrying a uniform load of 100,000 lb. so that $w = 100,000/120 = 10,000$ and a tension of $T = 10,000$ lb.. We calculate from (33.5) $\alpha = 2.850 \times 10^{-6}$ and $\beta = 1.425 \times 10^{-6}$. Thus we have the following BVP:

$$y'' = 2.850 \times 10^{-6}y + 1.425 \times 10^{-6}x(120 - x), \quad y(0) = y(120) = 0. \quad (33.7)$$

First subdivide the interval $[0, 120]$ into four equal subintervals. The nodes of this subdivision are $x_0 = 0$, $x_1 = 30$, $x_2 = 60$, $\dots$, $x_4 = 120$. We will then let $y_0, y_1, \dots, y_4$ denote the deflections at the nodes. From the boundary conditions we have immediately:

$$y_0 = y_4 = 0.$$

To determine the deflections at the interior points we will rely on the differential equation. Recall the central difference formula

$$y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}.$$

In this case we have $h = (b - a)/n = (120 - 0)/4 = 30$. Replacing all the variables in the equation (33.4) by their discrete versions we get

$$y_{i+1} - 2y_i + y_{i-1} = h^2 \alpha y_i + h^2 \beta x_i (L - x_i).$$

Substituting in for α , β and h we obtain:

$$\begin{aligned} y_{i+1} - 2y_i + y_{i-1} &= 900 \times 2.850 \times 10^{-6} y_i + 900 \times 1.425 \times 10^{-6} x_i (120 - x_i) \\ &= 2.565 \times 10^{-3} y_i + 1.2825 \times 10^{-3} x_i (120 - x_i). \end{aligned}$$

This equation makes sense for $i = 1, 2, 3$. At $x_1 = 30$, the equation becomes:

$$\begin{aligned} y_2 - 2y_1 + y_0 &= 2.565 \times 10^{-3}y_1 + 1.2825 \times 10^{-3} \times 30(90) \\ \Leftrightarrow y_2 - 2.002565y_1 &= 3.46275. \end{aligned} \quad (33.8)$$

Note that this equation is linear in the unknowns y_1 and y_2 . At $x_2 = 60$ we have:

$$\begin{aligned} y_3 - 2y_2 + y_1 &= .002565y_2 + 1.2825 \times 10^{-3} \times 60^2 \\ \Leftrightarrow y_3 - 2.002565y_2 + y_1 &= 4.617. \end{aligned} \quad (33.9)$$

At $x_3 = 90$ we have (since $y_4 = 0$)

$$-2.002565y_3 + y_2 = 3.46275. \quad (33.10)$$

Thus (y_1, y_2, y_3) is the solution of the linear system:

$$\left(\begin{array}{ccc|c} -2.002565 & 1 & 0 & 3.46275 \\ 1 & -2.002565 & 1 & 4.617 \\ 0 & 1 & -2.002565 & 3.46275 \end{array} \right).$$

We can easily find the solution of this system in MATLAB:

```
>> A = [ -2.002565  1  0 ; 1 -2.002565  1 ; 0 1 -2.002565]
>> b = [ 3.46275  4.617  3.46275 ]'
>> y = A\b
```

To graph the solution, we need define the x values and add on the values at the endpoints:

```
>> x = 0:30:120
>> y = [0 ; y ; 0]
>> plot(x,y,'d')
```

Adding a spline will result in an excellent graph.

The exact solution of this BVP is given in (33.6). That equation, with the parameter values for the W12x22 I-beam as in the example, is in the program `myexactbeam.m`. We can plot the true solution on the same graph:

```
>> hold on
>> myexactbeam
```

Thus our numerical solution is extremely good considering how few subintervals we used and how very large the deflection is.

An amusing exercise is to set $T = 0$ in the program `myexactbeam.m`; the program fails because the exact solution is no longer valid. Also try $T = .1$ for which you will observe loss of precision. On the other hand the finite difference method still works when we set $T = 0$.

Exercises

- 33.1 Derive the finite difference equations for the BVP (33.7) on the same domain $([0, 120])$, but with eight subintervals and solve (using MATLAB) as in the example. Plot your result, together on the same plot with the exact solution (33.6) from the program `myexactbeam.m`.
- 33.2 By replacing y'' and y' with central differences, derive the finite difference equation for the boundary value problem

$$y'' + y' - y = x \quad \text{on } [0, 1] \quad \text{with} \quad y(0) = y(1) = 0$$

using 5 subintervals. Solve them and plot the solution using MATLAB.

Lecture 34

Finite Difference Method – Nonlinear ODE

Heat conduction with radiation

If we again consider the heat in a metal bar of length L , but this time consider the effect of radiation as well as conduction, then the steady state equation has the form

$$u_{xx} - d(u^4 - u_b^4) = -g(x), \quad (34.1)$$

where u_b is the temperature of the background, d incorporates a coefficient of radiation and $g(x)$ is the heat source.

If we again replace the continuous problem by its discrete approximation then we get

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} - d(u_i^4 - u_b^4) = -g_i = -g(x_i). \quad (34.2)$$

This equation is nonlinear in the unknowns, thus we no longer have a system of linear equations to solve, but a system of nonlinear equations. One way to solve these equations would be by the multivariable Newton method in Lecture 13. Instead, we introduce another iterative method.

Relaxation Method for Nonlinear Finite Differences

We can rewrite equation (34.2) as

$$u_{i+1} - 2u_i + u_{i-1} = h^2 d(u_i^4 - u_b^4) - h^2 g_i.$$

From this we can solve for u_i in terms of the other quantities:

$$2u_i = u_{i+1} + u_{i-1} - h^2 d(u_i^4 - u_b^4) + h^2 g_i.$$

Next we add u_i to both sides of the equation to obtain

$$3u_i = u_{i+1} + u_i + u_{i-1} - h^2 d(u_i^4 - u_b^4) + h^2 g_i,$$

and then divide by 3 to get

$$u_i = \frac{1}{3} (u_{i+1} + u_i + u_{i-1}) - \frac{h^2}{3} (d(u_i^4 - u_b^4) - g_i).$$

Now for the main idea, we will begin with an initial guess for the value of u_i for each i , which we can represent as a vector $\mathbf{u}^0$. Then we will use the above equation to get better estimates, $\mathbf{u}^1, \mathbf{u}^2, \dots$, and hope that they converge to the correct answer.

If we let

$$\mathbf{u}^j = (u_0^j, u_1^j, u_2^j, \dots, u_{n-1}^j, u_n^j)$$

denote the j th approximation, then we can obtain that $j + 1$ st estimate from the formula

$$u_i^{j+1} = \frac{1}{3} \left(u_{i+1}^j + u_i^j + u_{i-1}^j \right) - \frac{h^2}{3} \left(d((u_i^j)^4 - u_b^4) - g_i \right).$$

Notice that g_i and u_b do not change. In the resulting equation, we have u_i at each successive step depending on its previous value and the equation itself.

Implementing the Relaxation Method

In the following program we solve the finite difference equations (34.2) with the boundary conditions $u(0) = 0$ and $u(L) = 0$. We let $L = 4$, $n = 4$, $d = 1$, and $g(x) = \sin(\pi x/4)$. Notice that the vector $\mathbf{u}$ always contains the current estimate of the values of $\mathbf{u}$.

```
% mynonlinheat (lacks comments)
% Purpose:
L = 4; %
n = 4; %
h = L/n; %
hh = h^2/3; %
u0 = 0; %
uL = 0; %
ub = .5; %
ub4 = ub^4; %
x = 0:h:L; %
g = sin(pi*x/4); %
u = zeros(1,n+1); %
steps = 4; %
u(1)=u0; %
u(n+1)=uL; %
for j = 1:steps
    %
    u(2:n) = (u(3:n+1)+u(2:n)+u(1:n-1))/3 + hh*(-u(2:n).^4+ub4+g(2:n));
end
plot(x,u)
```

If you run this program with the given $\mathbf{n}$ and $\mathbf{steps}$ the result will not seem reasonable.

We can plot the initial guess by adding the command `plot(x,u)` right before the `for` loop. We can also plot successive iterations by moving the last `plot(x,u)` before the `end`. Now we can experiment and see if the iteration is converging. Try various values of *steps* and *n* to produce a good plot. You will notice that this method converges quite slowly. In particular, as we increase *n*, we need to increase *steps* like n^2 , i.e. if *n* is large then *steps* needs to be *really* large.

Exercises

- 34.1 Modify the script program `mynonlinheat` to plot the initial guess and all intermediate approximations. Add complete comments to the program. Print the program and a plot using $n = 12$ and `steps` large enough to see convergence.
- 34.2 Modify your improved `mynonlinheat` to `mynonlinheattwo` that has the boundary conditions

$$u(0) = 5 \quad \text{and} \quad u(L) = 10.$$

Fix the comments to reflect the new boundary conditions. Print the program and a plot using $n = 50$ and large enough `steps` to see convergence.

Lecture 35

Parabolic PDEs - Explicit Method

Heat Flow and Diffusion

In the previous sections we studied PDE that represent *steady-state* heat problem. There was no time variable in the equation. In this section we begin to study how to solve equations that involve time, i.e. we calculate temperature profiles that are changing.

The conduction of heat and diffusion of a chemical happen to be modeled by the same differential equation. The reason for this is that they both involve similar processes. Heat conduction occurs when hot, fast moving molecules bump into slower molecules and transfer some of their energy. In a solid this involves moles of molecules all moving in different, nearly random ways, but the net effect is that the energy eventually spreads itself out over a larger region. The diffusion of a chemical in a gas or liquid similarly involves large numbers of molecules moving in different, nearly random ways. These molecules eventually spread out over a larger region.

In three dimensions, the equation that governs both of these processes is the heat/diffusion equation

$$u_t = c\Delta u,$$

where c is the coefficient of conduction or diffusion, and $\Delta u(x, y, z) = u_{xx} + u_{yy} + u_{zz}$. The symbol Δ in this context is called the *Laplacian*. If there is also a heat/chemical source, then it is incorporated a function $g(x, y, z, t)$ in the equation as

$$u_t = c\Delta u + g.$$

In some problems the z dimension is irrelevant, either because the object in question is very thin, or u does not change in the z direction. In this case the equation is

$$u_t = c\Delta u = c(u_{xx} + u_{yy}).$$

Finally, in some cases only the x direction matters. In this case the equation is just

$$u_t = cu_{xx}, \tag{35.1}$$

or

$$u_t = cu_{xx} + g(x, t). \tag{35.2}$$

In this lecture we will learn a straight-forward technique for solving (35.1) and (35.2). It is very similar to the finite difference method we used for nonlinear boundary value problems.

It is worth mentioning a related equation

$$u_t = c\Delta(u^\gamma) \quad \text{for } \gamma > 1,$$

which is called the porous-media equation. This equation models diffusion in a solid, but porous, material, such as sandstone or an earthen structure. We will not solve this equation numerically, but the methods introduced here would work. Many equations that involve 1 time derivative and 2 spatial derivatives are **parabolic** and the methods introduced here will work for most of them.

Explicit Method Finite Differences

The one dimensional heat/diffusion equation $u_t = cu_{xx}$, has two independent variables, t and x , and so we have to discretize both. Since we are considering $0 \leq x \leq L$, we subdivide $[0, L]$ into m equal subintervals, i.e. let

$$h = L/m$$

and

$$(x_0, x_1, x_2, \dots, x_{m-1}, x_m) = (0, h, 2h, \dots, L - h, L).$$

Similarly, if we are interested in solving the equation on an interval of time $[0, T]$, let

$$k = T/n$$

and

$$(t_0, t_1, t_2, \dots, t_{n-1}, t_n) = (0, k, 2k, \dots, T - k, T).$$

We will then denote the approximate solution at the grid points by

$$u_{ij} \approx u(x_i, t_j).$$

The equation $u_t = cu_{xx}$ can then be replaced by the difference equations

$$\frac{u_{i,j+1} - u_{i,j}}{k} = \frac{c}{h^2} (u_{i-1,j} - 2u_{i,j} + u_{i+1,j}). \quad (35.3)$$

Here we have used the forward difference for u_t and the central difference for u_{xx} . This equation can be solved for $u_{i,j+1}$ to produce

$$u_{i,j+1} = ru_{i-1,j} + (1 - 2r)u_{i,j} + ru_{i+1,j} \quad (35.4)$$

for $1 \leq i \leq m - 1$, $0 \leq j \leq n - 1$, where

$$r = \frac{ck}{h^2}. \quad (35.5)$$

The formula (35.4) allows us to calculate all the values of u at step $j + 1$ using the values at step j .

Notice that $u_{i,j+1}$ depends on $u_{i,j}$, $u_{i-1,j}$ and $u_{i+1,j}$. That is u at grid point i depends on its previous value and the values of its two nearest neighbors at the previous step (see Figure 35.1).

Initial Condition

To solve the partial differential equation (35.1) or (35.2) we need an initial condition. This represents the state of the system when we begin, i.e. the initial temperature distribution or initial concentration profile. This is represented by

$$u(x, 0) = f(x).$$

To implement this in a program we let

$$u_{i,0} = f(x_i).$$

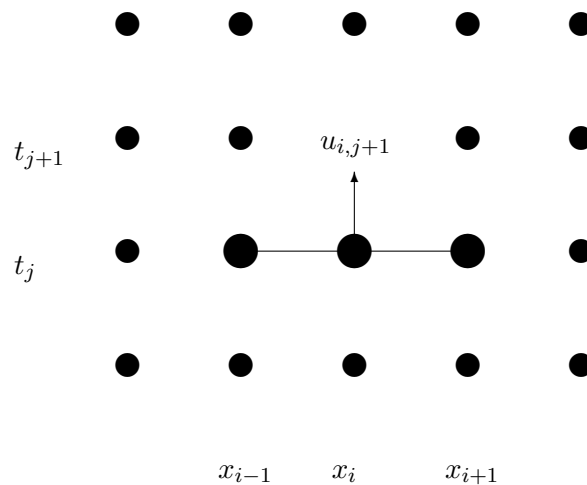


Figure 35.1: The value at grid point $(i, j + 1)$ depends on its previous value and the previous values of its nearest neighbors.

Boundary Conditions

To solve the partial differential equation (35.1) or (35.2) we also need boundary conditions. Just as in the previous section we will have to specify something about the ends of the domain, i.e. at $x = 0$ and $x = L$. One possibility is fixed boundary conditions, which we can implement just as we did for the ODE boundary value problem.

A second possibility is called **variable boundary conditions**. This is represented by time-dependent functions,

$$u(0, t) = g_1(t) \quad \text{and} \quad u(L, t) = g_2(t).$$

In a heat problem, g_1 and g_2 would represent heating or cooling applied to the ends. These are easily implemented in a program by letting $u_{0,j} = g_1(t_j)$ and $u_{m,j} = g_2(t_j)$.

Implementation

The following program (available to download as `myheat.m`) implements the explicit method. It incorporates variable boundary conditions at both ends. To run it you must define functions f , g_1 and g_2 . Notice that the main loop has only one line. The values of u are kept as a matrix. It is often convenient to define a matrix of the right dimension containing all zeros, and then fill in the calculated values as the program runs.

Run the following program using $L = 2$, $T = 20$, $f(x) = .5x$, $g_1(t) = 0$, and $g_2(t) = \cos(t)$. You will find that simply `g1 = @(t) 0` will not work, because it will only produce a single number 0. Instead use `g1 = @(t) 0*t`, which will produce the needed vector of zeros.

```

function [t x u] = myheat(f,g1,g2,L,T,m,n,c)
% function [t x u] = myheat(f,g1,g2,L,T,m,n,c)
% solve  $u_t = c u_{xx}$  for  $0 \leq x \leq L$ ,  $0 \leq t \leq T$ 
% BC:  $u(0, t) = g1(t)$ ;  $u(L, t) = g2(t)$ 
% IC:  $u(x, 0) = f(x)$ 
% Inputs:
%   f -- function for IC
%   g1,g2 -- functions for BC
%   L -- length of rod
%   T -- length of time interval
%   m -- number of subintervals for x
%   n -- number of subintervals for t
%   c -- rate constant in equation
% Outputs:
%   t -- vector of time points
%   x -- vector of x points
%   u -- matrix of the solution,  $u(i,j) \sim u(x(i), t(j))$ 
% Also plots.

h = L/m; k = T/n; % set space and time step sizes
r = c*k/h^2; rr = 1 - 2*r;
x = linspace(0,L,m+1); % set space discretization
t = linspace(0,T,n+1); % set time discretization
%Set up the matrix for u:
u = zeros(m+1,n+1);
% evaluate initial conditions
u(:,1) = f(x);
% evaluate boundary conditions
u(1,:) = g1(t); u(m+1,:) = g2(t);

% find solution at remaining time steps
for j = 1:n
    % explicit method update at next time
    u(2:m,j+1) = r*u(1:m-1,j) + rr*u(2:m,j) + r*u(3:m+1,j);
end

% plot the results
mesh(x,t,u')
end

```

Exercises

- 35.1 Run the program `myheat.m` with $L = 2\pi$, $T = 20$, $c = .5$, $g_1(t) = \sin(t)$, $g_2(t) = 0$ and $f(x) = -\sin(x/4)$. Set $m = 20$ and experiment with n . Get a plot when the program is stable and one when it isn't. Turn in the plots. Hint: The function $g_2(t)$ will need to be input using the formula `g2 = @(t) 0*t`.
- 35.2 Make a version of the program `myheat.m` that does not input n or T but instead has inputs:

```
%    k -- size of the time steps
%    temp -- keeps stepping in time until the maximum current
%            temperature in the bar is less than temp
```

For $L = 2\pi$, $c = .01$, $g_1(t) = 0$, $g_2(t) = 10$, $f(x) = 100$ and $m = 10$, set k so that the method will be stable. Run it with `temp = 20`. (Hint: see your Exercise 30.2 as a model.) *When* does the temperature in the bar drop below 20? Turn in your program and the plot.

Lecture 36

Solution Instability for the Explicit Method

As we saw in experiments using `myheat.m`, the solution can become unbounded unless the time steps are small. In this lecture we consider why.

Writing the Difference Equations in Matrix Form

If we use the boundary conditions $u(0) = u(L) = 0$ then the explicit method of the previous section has the form

$$u_{i,j+1} = ru_{i-1,j} + (1-2r)u_{i,j} + ru_{i+1,j} \quad \text{for } 1 \leq i \leq m-1 \quad \text{and} \quad 0 \leq j \leq n-1,$$

where $u_{0,j} = 0$ and $u_{m,j} = 0$. This is equivalent to the matrix equation

$$\mathbf{u}_{j+1} = A\mathbf{u}_j, \tag{36.1}$$

where $\mathbf{u}_j$ is the column vector $(u_{1,j}, u_{2,j}, \dots, u_{m,j})'$ representing the state at the j th time step and A is the matrix

$$A = \begin{pmatrix} 1-2r & r & 0 & \cdots & 0 \\ r & 1-2r & r & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & r & 1-2r & r \\ 0 & \cdots & 0 & r & 1-2r \end{pmatrix}. \tag{36.2}$$

Unfortunately, this matrix can have a property which is very bad in this context. Namely, it can cause exponential growth of error unless r is small. To see how this happens, suppose that $\mathbf{U}_j$ is the vector of correct values of u at time step t_j and $\mathbf{E}_j$ is the error of the approximation $\mathbf{u}_j$, then

$$\mathbf{u}_j = \mathbf{U}_j + \mathbf{E}_j.$$

From (36.1), the approximation at the next time step will be

$$\mathbf{u}_{j+1} = A\mathbf{U}_j + A\mathbf{E}_j,$$

and if we continue for k steps,

$$\mathbf{u}_{j+k} = A^k\mathbf{U}_j + A^k\mathbf{E}_j.$$

The problem with this is the term $A^k\mathbf{E}_j$. This term is exactly what we would do in the power method for finding the eigenvalue of A with the largest absolute value. If the matrix A has eigenvalues with absolute value greater than 1, then this term will grow exponentially. Figure 36.1 shows the largest absolute value of an eigenvalue of A as a function of the parameter r for various sizes of the matrix A . As you can see, for $r > 1/2$ the largest absolute eigenvalue grows rapidly for any m and quickly becomes greater than 1.

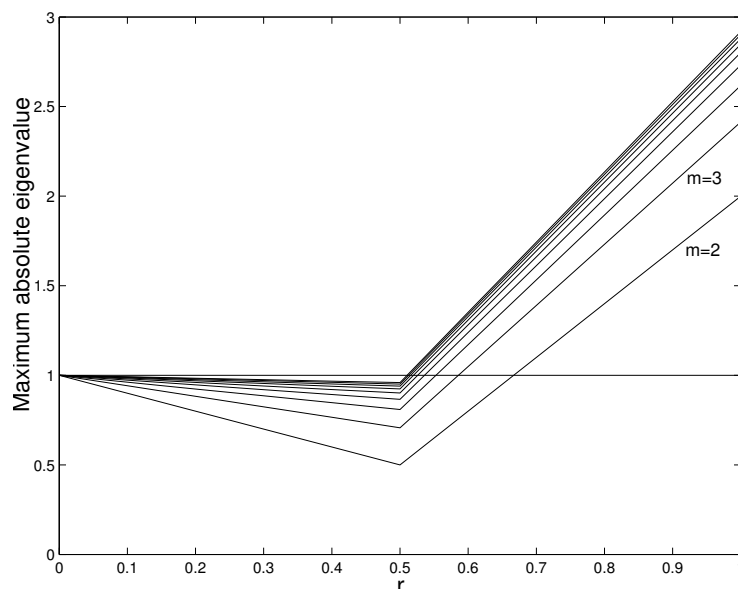


Figure 36.1: Maximum absolute eigenvalue as a function of r for the matrix A from the explicit method for the heat equation calculated for matrices A of sizes $m = 2 \dots 10$. Whenever the maximum absolute eigenvalue is greater than 1 the method is unstable, i.e. errors grow exponentially with each step. When using the explicit method $r < 1/2$ is a safe choice.

Consequences

Recall that $r = ck/h^2$. Since this must be less than $1/2$, we have

$$k < \frac{h^2}{2c}.$$

The first consequence is obvious: k must be relatively small. The second is that h cannot be too small. Since h^2 appears in the formula, making h small would force k to be extremely small! A third consequence is that we have a converse of this analysis. Suppose $r < .5$. Then all the eigenvalues will be less than one. Recall that the error terms satisfy

$$\mathbf{u}_{j+k} = A^k \mathbf{U}_j + A^k \mathbf{E}_j.$$

If all the eigenvalues of A are less than 1 in absolute value then $A^k \mathbf{E}_j$ grows smaller and smaller as k increases. This is really good. Rather than building up, the effect of any error diminishes as time passes! From this we arrive at the following principle: **If the explicit numerical solution for a parabolic equation does not blow up, then errors from previous steps fade away!**

Finally, we note that if we have non-zero boundary conditions then instead of equation (36.1) we have

$$\mathbf{u}_{j+1} = A\mathbf{u}_j + r\mathbf{b}_j, \quad (36.3)$$

where the first and last entries of $\mathbf{b}_j$ contain the boundary conditions and all the other entries are zero. In this case the errors behave just as before, if $r > 1/2$ then the errors grow and if $r < 1/2$ the errors fade away.

We can write a function program `myexpmatrix.m` that produces the matrix A in (36.2), for given inputs m and r . Without using loops we can use the `diag` command to set up the matrix:

```

function A = myexpmatrix(m,r)
    % produces the matrix for the explicit method for a parabolic equation
    % Inputs: m -- the size of the matrix
    %          r -- the main parameter, ck/h^2
    % Output: A -- an m by m matrix
    u = (1-2*r)*ones(m,1); % make a vector for the main diagonal
    v = r*ones(m-1,1);     % make a vector for the upper and lower diagonals
    A = diag(u) + diag(v,1) + diag(v,-1); % assemble
end

```

Test this using $m = 6$ and $r = .4, .6$. Check the eigenvalues and eigenvectors of the resulting matrices:

```

>> A = myexpmatrix(6,.6)
>> [v e] = eig(A)

```

What is the “mode” represented by the eigenvector with the largest absolute eigenvalue? How is that reflected in the unstable solutions?

Exercises

- 36.1 Let $L = \pi$, $T = 20$, $f(x) = .1 \sin(x)$, $g_1(t) = 0$, $g_2(t) = 0$, $c = .5$, and $m = 20$, as used in the program `myheat.m`. What value of n corresponds to $r = 1/2$? Try different n in `myheat.m` to find precisely when the method works and when it fails. Is $r = 1/2$ the boundary between failure and success? Hand in a plot of the last success and the first failure. Include the values of n and r in each.
- 36.2 Write a well-commented MATLAB **script** program that produces the graph in Figure 36.1 for $m = 4$. Your program should:
- define r values from 0 to 1,
 - for each r
 - create the matrix A by calling `myexpmatrix`,
 - calculate the eigenvalues of A ,
 - find the max of the absolute values, and
 - plot these numbers versus r .

Lecture 37

Implicit Methods

The Implicit Difference Equations

By approximating u_{xx} and u_t at t_{j+1} rather than t_j , and using a backwards difference for u_t , the equation $u_t = cu_{xx}$ is approximated by

$$\frac{u_{i,j+1} - u_{i,j}}{k} = \frac{c}{h^2}(u_{i-1,j+1} - 2u_{i,j+1} + u_{i+1,j+1}).$$

Note that all the terms have index $j+1$ except one and isolating this term leads to

$$u_{i,j} = -ru_{i-1,j+1} + (1 + 2r)u_{i,j+1} - ru_{i+1,j+1} \quad \text{for } 1 \leq i \leq m-1, \quad (37.1)$$

where $r = ck/h^2$ as before. The entries involved in (37.1) are illustrated in Figure 37.1.

Now we have $\mathbf{u}_j$ given in terms of $\mathbf{u}_{j+1}$. This seems like a problem, since $\mathbf{u}_{j+1}$ is the solution at a later time than $\mathbf{u}_j$, so we could never know $\mathbf{u}_{j+1}$ before we knew $\mathbf{u}_j$. However, the relationship between $\mathbf{u}_{j+1}$ and $\mathbf{u}_j$ is linear. Using matrix notation, we have

$$\mathbf{u}_j = B\mathbf{u}_{j+1} - r\mathbf{b}_{j+1},$$

where $\mathbf{b}_{j+1}$ represents the boundary conditions. Thus to find $\mathbf{u}_{j+1}$ from $\mathbf{u}_j$, we need only solve the linear system

$$B\mathbf{u}_{j+1} = \mathbf{u}_j + r\mathbf{b}_{j+1}, \quad (37.2)$$

where $\mathbf{u}_j$ and $\mathbf{b}_{j+1}$ are given and

$$B = \begin{pmatrix} 1+2r & -r & & & \\ -r & 1+2r & -r & & \\ & \ddots & \ddots & \ddots & \\ & & -r & 1+2r & -r \\ & & & -r & 1+2r \end{pmatrix}. \quad (37.3)$$

(This is an example of how a sparse matrix occurs in applications.) Using this scheme is called the **implicit method** since $\mathbf{u}_{j+1}$ is defined implicitly. Since we have to solve a linear system at each step, the implicit method requires more work per step than the explicit method.

Since we are solving (37.2), the most important quantity is the maximum absolute eigenvalue of B^{-1} , which is 1 divided by the smallest eigenvalue of B . Figure 37.2 shows the maximum absolute eigenvalues of B^{-1} as a function of r for various size matrices. Notice that this absolute maximum is always less than 1. Thus errors are always diminished over time and so this method is always stable. For the same reason it is also always as accurate as the individual steps.

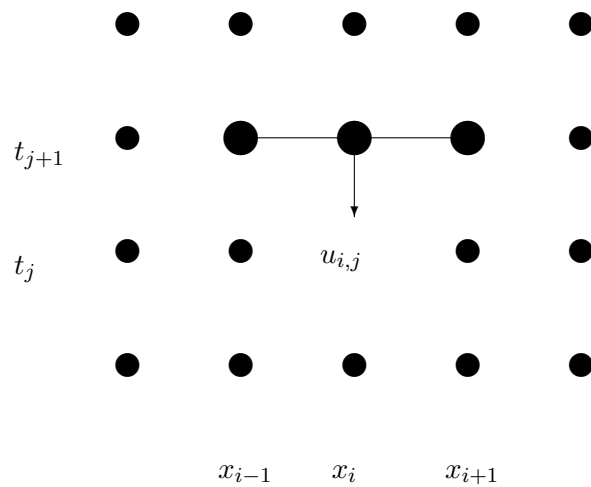


Figure 37.1: The value at grid point (i, j) depends on its future value and the future values of its nearest neighbors.

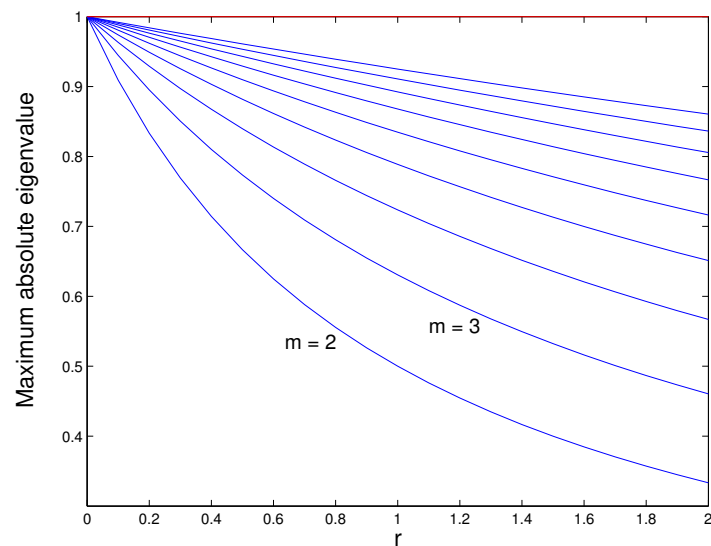


Figure 37.2: Maximum absolute eigenvalue as a function of r for the matrix B^{-1} from the implicit method for the heat equation calculated for matrices B of sizes $m = 2 \dots 10$. Whenever the maximum absolute eigenvalue is less than 1 the method is stable, i.e. it is always stable.

Both this implicit method and the explicit method in the previous lecture make $O(h^2)$ error in approximating u_{xx} and $O(k)$ error in approximating u_t , so they have total error $O(h^2 + k)$. Thus although the stability condition allows the implicit method to use arbitrarily large k , to maintain accuracy we still need $k \sim h^2$.

Crank-Nicholson Method

Now that we have two different methods for solving parabolic equation, it is natural to ask, “can we improve by taking an average of the two methods?” The answer is yes.

We implement a weighted average of the two methods by considering an average of the approximations of u_{xx} at j and $j + 1$. This leads to the equations

$$\frac{u_{i,j+1} - u_{i,j}}{k} = \frac{\lambda c}{h^2}(u_{i-1,j+1} - 2u_{i,j+1} + u_{i+1,j+1}) + \frac{(1-\lambda)c}{h^2}(u_{i-1,j} - 2u_{i,j} + u_{i+1,j}). \quad (37.4)$$

The implicit method contained in these equations is called the **Crank-Nicholson method**. Gathering terms yields the equations

$$-r\lambda u_{i-1,j+1} + (1 + 2r\lambda)u_{i,j+1} - r\lambda u_{i+1,j+1} = r(1-\lambda)u_{i-1,j} + (1 - 2r(1-\lambda))u_{i,j} + r(1-\lambda)u_{i+1,j}.$$

In matrix notation this is

$$B_\lambda \mathbf{u}_{j+1} = A_\lambda \mathbf{u}_j + r \mathbf{b}_{j+1},$$

where

$$A_\lambda = \begin{pmatrix} 1 - 2(1-\lambda)r & (1-\lambda)r & & & \\ (1-\lambda)r & 1 - 2(1-\lambda)r & (1-\lambda)r & & \\ & \ddots & \ddots & \ddots & \\ & & (1-\lambda)r & 1 - 2(1-\lambda)r & (1-\lambda)r \\ & & & (1-\lambda)r & 1 - 2(1-\lambda)r \end{pmatrix}$$

and

$$B_\lambda = \begin{pmatrix} 1 + 2r\lambda & -r\lambda & & & \\ -r\lambda & 1 + 2r\lambda & -r\lambda & & \\ & \ddots & \ddots & \ddots & \\ & & -r\lambda & 1 + 2r\lambda & -r\lambda \\ & & & -r\lambda & 1 + 2r\lambda \end{pmatrix}.$$

In this equation $\mathbf{u}_j$ and $\mathbf{b}_{j+1}$ are known, $A_\lambda \mathbf{u}_j$ can be calculated directly, and then the equation is solved for $\mathbf{u}_{j+1}$.

If we choose $\lambda = 1/2$, then we are in effect doing a central difference for u_t , which has error $O(k^2)$. Our total error is then $O(h^2 + k^2)$. With a bit of work, we can show that the method is always stable, and so we can use $k \sim h$ without a problem.

To get optimal accuracy with a weighted average, it is always necessary to use the right weights. For the Crank-Nicholson method with a given r , we need to choose

$$\lambda = \frac{r - 1/6}{2r}.$$

This choice will make the method have truncation error of order $O(h^4 + k^2)$, which is really good considering that the implicit and explicit methods each have truncation errors of order $O(h^2 + k)$. Surprisingly, we can

do even better if we also require

$$r = \frac{\sqrt{5}}{10} \approx 0.22361,$$

and, consequently,

$$\lambda = \frac{3 - \sqrt{5}}{6} \approx 0.12732.$$

With these choices, the method has truncation error of order $O(h^6)$, which is absolutely amazing.

To appreciate the implications, suppose that we need to solve a problem with 4 significant digits. If we use the explicit or implicit method alone then we will need $h^2 \approx k \approx 10^{-4}$. If $L = 1$ and $T \approx 1$, then we need $m \approx 100$ and $n \approx 10,000$. Thus we would have a total of 1,000,000 grid points, almost all in the interior. This is a lot.

Next suppose we solve the same problem using the optimal Crank-Nicholson method. We would need $h^6 \approx 10^{-4}$ which would require us to take $m \approx 4.64$, so we would take $m = 5$ and have $h = 1/5$. For k we need $k = (\sqrt{5}/10)h^2/c$. If $c = 1$, this gives $k = \sqrt{5}/250 \approx 0.0089442$ so we would need $n \approx 112$ to get $T \approx 1$. This gives us a total of 560 interior grid points, or, a factor of 1785 fewer than the explicit or implicit method alone.

Exercises

- 37.1 Modify the program `myexpmatrix.m` from exercise 36.2 into a function program `myimpmatrix.m` that produces the matrix B in (37.3) for given inputs m and r . Modify your script from exercise 36.2 to use B^{-1} and to plot for $r \in [0, 2]$; keep $m = 4$. It should produce a graph similar to that in Figure 37.2 for $m = 4$. Turn in the programs and the plot.
- 37.2 Modify the program `myheat` into a new program `myimplicitheat` that uses the implicit method to solve the boundary value problem

$$u_t = cu_{xx}, \quad u(t, 0) = u(t, L) = 0, \quad u(0, x) = f(x)$$

by repeatedly solving the system $B\mathbf{u}_{j+1} = \mathbf{u}_j$ for each time step.

(Hints: Delete `g1` and `g2` from the program since they are always zero. Call B using `myimpmatrix` before the loop and solve $B\mathbf{u}_{j+1} = \mathbf{u}_j$ inside the loop.)

Run the program with $L = 5$, $T = 50$, $c = .1$ and $f(x) = \sin(\pi x/5)$ and at least 10 pairs of values of m and n . Turn in the program and a list of the m, n you tried and whether the simulation was stable or unstable. Are you convinced that it is always stable?

Lecture 38

Insulated Boundary Conditions

Insulation

In many of the previous sections we have considered fixed boundary conditions, i.e. $u(0) = a$, $u(L) = b$. We implemented these simply by assigning $u_0^j = a$ and $u_n^j = b$ for all j .

We also considered variable boundary conditions, such as $u(0, t) = g_1(t)$. For example, we might have $u(0, t) = \sin(t)$ which could represent periodic heating and cooling of the end at $x = 0$.

A third important type of boundary condition is called the *insulated* boundary condition. It is so named because it mimics an insulator at the boundary. Physically, the effect of insulation is that no heat flows across the boundary. This means that the temperature gradient is zero, which implies that we should require the mathematical boundary condition $u'(L) = 0$.

To use it in a program, we must replace $u'(L) = 0$ by a discrete version. Recall that in our discrete equations we usually have $L = x_n$. Recall from the section on numerical derivatives, that there are three different ways to replace a derivative by a difference equation, left, right and central differences. The three of them at x_n would be

$$u'(x_n) \approx \frac{u_n - u_{n-1}}{h} \approx \frac{u_{n+1} - u_n}{h} \approx \frac{u_{n+1} - u_{n-1}}{2h}.$$

If x_n is the last node of our grid, then it is clear that we cannot use the right or central difference, but are stuck with the first of these. Setting that expression to zero implies

$$u_n = u_{n-1}.$$

This restriction can be easily implemented in a program simply by putting a statement `u(n+1)=u(n)` inside the loop that updates values of the profile. However, since this method replaces $u'(L) = 0$ by an expression that is only accurate to first order, it is not very accurate and is usually avoided.

Instead we want to use the most accurate version, the central difference. For that we should have

$$u'(L) = u'(x_n) = \frac{u_{n+1} - u_{n-1}}{2h} = 0.$$

or simply

$$u_{n+1} = u_{n-1}.$$

However, u_{n+1} would represent $u(x_{n+1})$ and x_{n+1} would be $L + h$, which is outside the domain. This, however, is not an obstacle in a program. We can simply extend the grid to one more node, x_{n+1} , and let u_{n+1} always equal u_{n-1} by copying u_{n-1} into u_{n+1} whenever u_{n-1} changes. The point x_{n+1} is “fictional”, but a computer does not know the difference between fiction and reality! This idea is carried out in the calculations of the next section and illustrated in Figure 38.1.

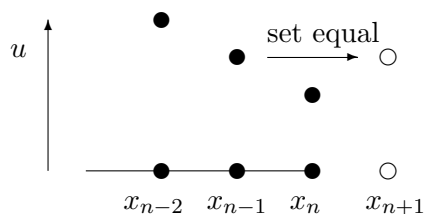


Figure 38.1: Illustration of an insulated boundary condition using a fictional point x_{n+1} with $u_{n+1} = u_{n-1}$.

A way to think of an insulated boundary that makes sense of the point $L + h$ is to think of two bars joined end to end, where you let the second bar be mirror image of the first bar. If you do this, then no heat will flow across the joint, which is exactly the same effect as insulating.

Another practical way to implement an insulated boundary is to let the grid points straddle the boundary. For example suppose we want to impose insulated boundary at the left end of a bar, i.e. $u'(0) = 0$, then you could let the first two grid points be at $x_0 = -h/2$ and $x_1 = h/2$. Then you can let

$$u_0 = u_1.$$

This will again force the central difference at $x = 0$ to be 0.

Implementation in a linear equation by elimination

Consider the BVP

$$u_{xx} = -1 \quad \text{with} \quad u(0) = 5 \quad \text{and} \quad u'(1) = 0. \quad (38.1)$$

This represents the steady state temperature of a bar with a uniformly applied heat source, with one end held at a fixed temperature and the other end insulated.

If we use 4 equally spaced intervals, then

$$m = 4 \quad \text{and} \quad L = 1 \quad \Rightarrow \quad h = \frac{L}{m} = \frac{1}{4},$$

and

$$x_0 = 0, x_1 = .25, x_2 = .5, x_3 = .75, x_4 = 1, \text{ and } x_5 = 1.25.$$

The point $x_5 = 1.25$ is outside the region and thus fictional. The boundary condition at $x_0 = 0$ is implemented as

$$u_0 = 5.$$

For the insulated condition, we will require

$$u_5 = u_3.$$

This makes the central difference for $u'(x_4)$ be zero. We can write the differential equation as a difference equation

$$\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} = -1$$

or

$$u_{i-1} - 2u_i + u_{i+1} = -0.0625, \quad i = 1, 2, 3, 4.$$

For $i = 1$, recalling that $u_0 = 5$, we have

$$5 - 2u_1 + u_2 = -.0625 \quad \text{or} \quad -2u_1 + u_2 = -5.0625.$$

For $i = 2$ and $i = 3$ we have

$$u_1 - 2u_2 + u_3 = -.0625 \quad \text{and} \quad u_2 - 2u_3 + u_4 = -.0625.$$

For $i = 4$ we have

$$u_3 - 2u_4 + u_5 = -.0625.$$

Note that we now have 5 unknowns in our problem: $u_1, \dots, u_5$. However, from the boundary condition $u_5 = u_3$ and so we can eliminate u_5 from our $i = 4$ equation and write

$$2u_3 - 2u_4 = -.0625.$$

Summarizing, we can put the unknown quantities in a vector $\mathbf{u} = (u_1, u_2, u_3, u_4)'$ and write the equations as a matrix equation $A\mathbf{u} = \mathbf{b}$ where

$$A = \begin{pmatrix} -2 & 1 & 0 & 0 \\ 1 & -2 & 1 & 0 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & 2 & -2 \end{pmatrix}$$

and $\mathbf{b} = (-5.0625, -.0625, -.0625, -.0625)'$. Solve this system and plot the results:

```
>> u = A\b
>> u = [5 ; u]
>> x = 0:.25:1
>> plot(x,u,'d')
```

Then interpolate with a spline.

The exact solution of this BVP is:

$$U(x) = 5 + x - .5x^2.$$

Use `hold on` and plot this function on the same graph to compare:

```
>> xx = 0:.01:1;
>> uu = 5 + xx - .5*xx.^2;
>> hold on
>> plot(xx,uu,'r')
```

You should see that our approximate solution is almost perfect!

Insulated boundary conditions in time-dependent problems

To implement the insulated boundary condition in an explicit difference equation with time, we need to copy values from inside the region to fictional points just outside the region. Note that you copy the *new* value from inside the region to the false point during each time step, i.e. inside the loop, but after the values are updated at the real points. See Figure 38.2 for an illustration.

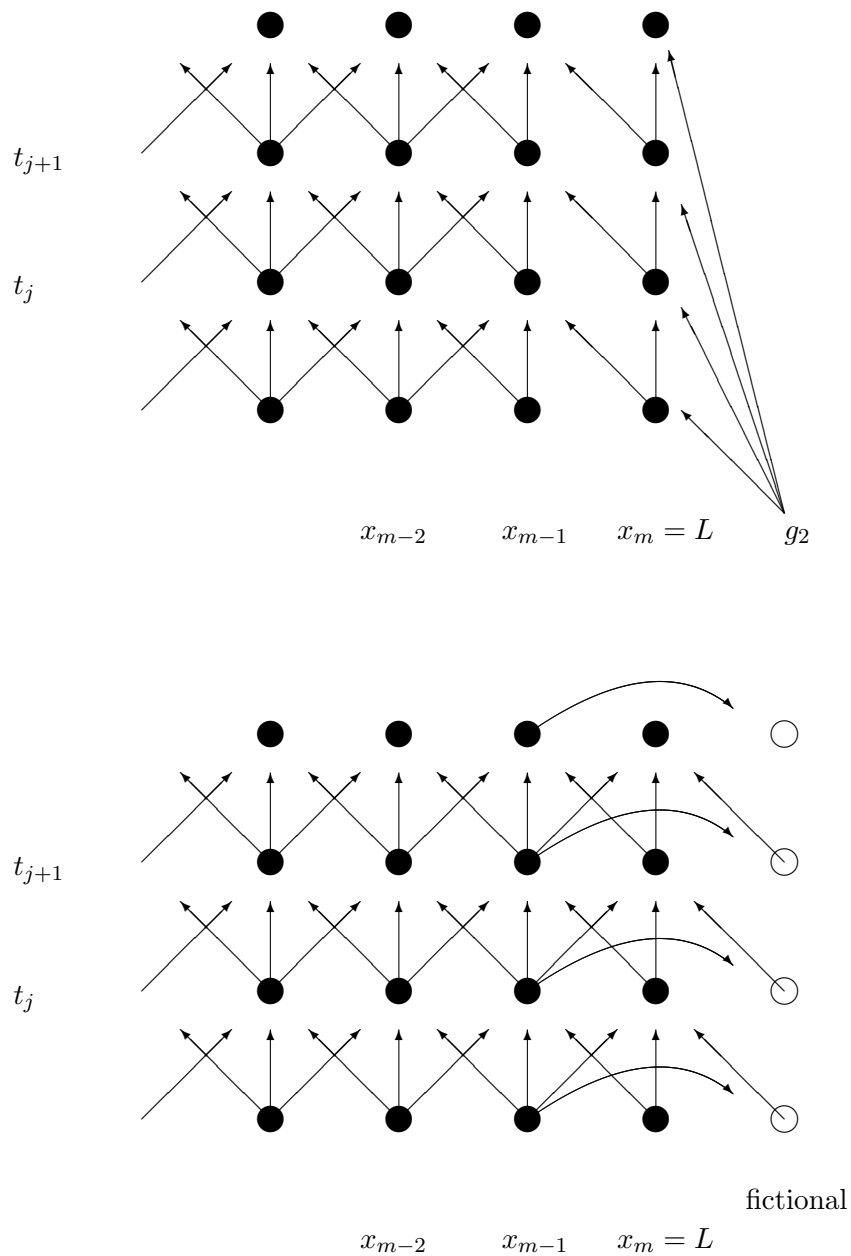


Figure 38.2: Illustration of information flow for the explicit method near the right boundary at $x = L$. The top figure shows a fixed boundary condition, where $u_{m,j}$ is set to be $g_2(t_j)$. The bottom figure shows an insulating boundary condition. Now $u_{m,j}$ is updated in the same way as the general $u_{i,j}$ and an additional entry $u_{m+1,j}$ is used with its value set by copying $u_{m-1,j}$.

An example

The steady state temperature $u(r)$ (given in polar coordinates) of a disk subjected to a radially symmetric heat load $g(r)$ and cooled by conduction to the rim of the disk and radiation to its environment is determined by the boundary value problem

$$\frac{\partial^2 u}{\partial r^2} + \frac{1}{r} \frac{\partial u}{\partial r} = d(u^4 - u_b^4) - g(r) \quad \text{with} \quad u(R) = u_R \quad \text{and} \quad u'(0) = 0. \quad (38.2)$$

Here u_b is the (fixed) background temperature and u_R is the (fixed) temperature at the rim of the disk.

The program `myheatdisk.m` implements these equations for parameter values $R = 5$, $d = .1$, $u_R = u_b = 10$ and $g(r) = (r - 5)^2$. Notice that the equations have a singularity (discontinuity) at $r = 0$. How does the program avoid this problem? How does the program implement $u_R = 10$ and $u'(0) = 0$? Run the program.

Exercises

- 38.1 Redo the calculations for the BVP (38.1) except do not include the fictional point x_5 . Instead, let x_4 be the last point and impose the insulated boundary by requiring $u_4 = u_3$. (Keep $m = 4$ and $h = 1/4$. Your system of equations should be 3×3 .) Compare this solution with the true solution and the better approximation in the lecture. Illustrate this comparison on a single plot.
- 38.2 Modify the program `myheat.m` to have an insulated boundary at $x = L$ (rather than $u(L, t) = g_2(t)$). You will need to change the domain to: `x = 0:h:L+h`, change the dimensions of all the other objects to fit this domain and implement the insulation (copy) step inside the loop (see the section ‘Insulated boundary conditions in time-dependent problems’ and the figure).
- Run the program with $L = 2\pi$, $T = 20$, $c = .6$, $g_1(t) = \sin(t)$ and $f(x) = -\sin(x/4)$. Set $m = 20$ and experiment with n . Get a plot when the program is stable. Turn in your program and plot.

Lecture 39

Finite Difference Method for Elliptic PDEs

Examples of Elliptic PDEs

Elliptic PDE's are equations with second derivatives in space and no time derivative. The most important examples are Laplace's equation

$$\Delta u = u_{xx} + u_{yy} + u_{zz} = 0$$

and the Poisson equation

$$\Delta u = f(x, y, z).$$

These equations are used in a large variety of steady-state physical situations such as: steady state heat problems, steady state chemical distributions, electrostatic potentials, elastic deformation and steady state fluid flows.

For the sake of clarity we will only consider the two dimensional problem. A good model problem in this dimension is the elastic deflection of a membrane. Suppose that a membrane such as a sheet of rubber is stretched across a rectangular frame. If some of the edges of the frame are bent, or if forces are applied to the sheet then it will deflect by an amount $u(x, y)$ at each point (x, y) . This u will satisfy the boundary value problem:

$$\begin{aligned} u_{xx} + u_{yy} &= f(x, y) \quad \text{for } (x, y) \text{ in } R, \\ u(x, y) &= g(x, y) \quad \text{for } (x, y) \text{ on } \partial R, \end{aligned} \tag{39.1}$$

where R is the rectangle, ∂R is the edge of the rectangle, $f(x, y)$ is the force density (pressure) applied at each point and $g(x, y)$ is the deflection at the edge.

The Finite Difference Equations

Suppose the rectangle is described by

$$R = \{a \leq x \leq b, c \leq y \leq d\}.$$

We will divide R in sub-rectangles. If we have m subdivisions in the x direction and n subdivisions in the y direction, then the step size in the x and y directions respectively are

$$h = \frac{b-a}{m} \quad \text{and} \quad k = \frac{d-c}{n}.$$

We obtain the finite difference equations for (39.1) by replacing u_{xx} and u_{yy} by their central differences to obtain

$$\frac{u_{i+1,j} - 2u_{ij} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{ij} + u_{i,j-1}}{k^2} = f(x_i, y_j) = f_{ij} \tag{39.2}$$

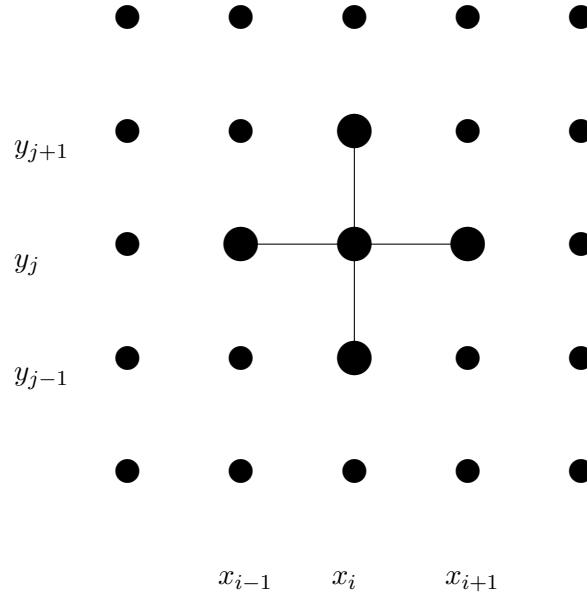


Figure 39.1: The finite difference equation relates five neighboring values in a + pattern.

for $1 \leq i \leq m-1$ and $1 \leq j \leq n-1$. See Figure 39.1 for an illustration. The boundary conditions are introduced by

$$u_{0,j} = g(a, y_j), \quad u_{m,j} = g(b, y_j), \quad u_{i,0} = g(x_i, c), \quad \text{and} \quad u_{i,n} = g(x_i, d). \quad (39.3)$$

Direct Solution of the Equations

Notice that since the edge values are prescribed, there are $(m-1) \times (n-1)$ grid points where we need to determine the solution. Note also that there are exactly $(m-1) \times (n-1)$ equations in (39.2). Finally, notice that the equations are all linear. Thus we could solve the equations exactly using matrix methods. To do this we would first need to express the u_{ij} 's as a vector, rather than a matrix. To do this there is a standard procedure: let $\mathbf{u}$ be the column vector we get by placing one column after another from the columns of (u_{ij}) . Thus we would list $u_{\cdot,1}$ first then $u_{\cdot,2}$, etc.. Next we would need to write the matrix A that contains the coefficients of the equations (39.2) and incorporate the boundary conditions in a vector $\mathbf{b}$. Then we could solve an equation of the form

$$A\mathbf{u} = \mathbf{b}. \quad (39.4)$$

Setting up and solving this equation is called the direct method.

An advantage of the direct method is that solving (39.4) can be done relatively quickly and accurately. The drawback of the direct method is that one must set up $\mathbf{u}$, A and $\mathbf{b}$, which is confusing. Further, the matrix A has dimensions $(m-1)(n-1) \times (m-1)(n-1)$, which can be rather large. Although A is large, many of its elements are zero. Such a matrix is called *sparse* and there are special methods intended for efficiently working with sparse matrices.

Iterative Solution

A usually preferred alternative to the direct method described above is to solve the finite difference equations iteratively. To do this, first solve (39.2) for u_{ij} , which yields

$$u_{ij} = \frac{1}{2(h^2 + k^2)} (k^2(u_{i+1,j} + u_{i-1,j}) + h^2(u_{i,j+1} + u_{i,j-1}) - h^2k^2f_{ij}). \quad (39.5)$$

This method is another example of a *relaxation method*. Using this formula, along with (39.3), we can update u_{ij} from its neighbors, just as we did in the relaxation method for the nonlinear boundary value problem. If this method converges, then the result is an approximate solution.

Download and read the script `mypoisson.m`, which implements the iterative solution. You will notice that `maxit` is set to 0. Thus the program will not do any iteration, but will plot the initial guess. The initial guess in this case consists of the proper boundary values at the edges, and zero everywhere in the interior. To see the solution evolve, gradually increase `maxit`.

Exercises

- 39.1 Modify the script `mypoisson.m` to change the force $f(x, y)$ to a negative constant $-p$. Obtain plots for $p = 5$ and $p = 50$. You will need to adjust `maxit` to obtain convergence. Tell how many iterations are needed for convergence in each. Turn in your plots and the modified program.
- 39.2 Further modify `mypoisson.m` to change the edge of the rectangle at $x = b$ ($= 10$) to be an insulated boundary, i.e. $u_x(b, y) = 0$. You will need to expand both the grid and the matrix `u` to include a row of fictional points. Then you will need to adjust a lot of indices and enforce insulation inside the loop. Run it with $p = 20$. If it is working correctly, the plot will be smooth and the derivative u_x on the edge $x = 10$ will appear to be zero. Turn in your plots and the modified program.

Lecture 40

Convection-Diffusion Equations*

Exercises

40.1

Lecture 41

Finite Elements

Triangulating a Region

A disadvantage of finite difference methods is that they require a very regular grid, and thus a very regular region, either rectangular or a regular part of a rectangle. Finite elements is a method that works for any shape region because it is not built on a grid, but on triangulation of the region, i.e. cutting the region up into triangles as we did in a previous lecture. The following figure shows a triangularization of a region.

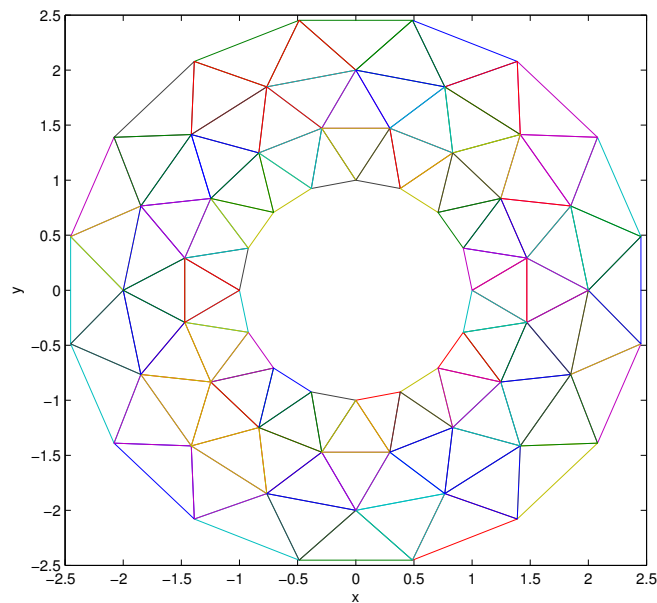


Figure 41.1: An annular region with a triangulation. Notice that the nodes and triangles are very evenly spaced.

This figure was produced by the script program `mywasher.m`. Notice that the nodes are evenly distributed. This is good for the finite element process where we will use it.

Open the program `mywasher.m`. This program defines a triangulation by defining the vertices in a matrix `V` in which each row contains the x and y coordinates of a vertex. Notice that we list the interior nodes first, then the boundary nodes.

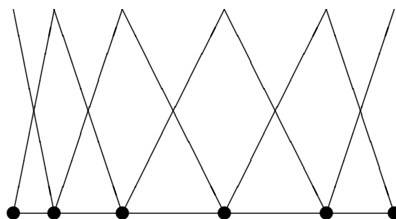


Figure 41.2: The finite elements for the “triangulation” of a one dimensional object.

Triangles are defined in the matrix T . Each row of T has three integer numbers indicating the indices of the nodes that form a triangle. For instance the first row is 43 42 25, so T_1 is the triangle with vertices $\mathbf{v}_{43}$, $\mathbf{v}_{42}$ and $\mathbf{v}_{25}$. The matrix T in this case was produced by the MATLAB command `delaunay`. The command produced more triangles than desired and the unwanted ones were deleted.

A three dimensional plot of the region and triangles is produced by having the last line be `trimesh(T,x,y,z)`.

What is a finite element?

The finite element method is a mathematically complicated process. However, a finite element is actually a very simple object. To each node $\mathbf{v}_j$ we associate a function Φ_j that has the properties $\Phi_j(\mathbf{v}_j) = 1$ and $\Phi_j(\mathbf{v}_i) = 0$ for $i \neq j$. Between nodes, Φ_j is a linear function. This function is the finite element. (There are fancier types of finite elements that we will not discuss.)

If we consider one dimension, then a triangulation is just a subdivision into subintervals. In Figure 41.2 we show an uneven subdivision of an interval and the finite elements corresponding to each node.

In two dimensions, Φ_j is composed of triangular piece of planes. Thus Φ_j is a function whose graph is a pyramid with its peak over node $\mathbf{v}_j$. For an example, try:

```
>> mywasher
>> z = 0*z;
>> z(1) = 1
>> trimesh(T,x,y,z)
```

What is a finite element solution?

A finite element (approximate) solution is a linear combination of the elements:

$$U(\mathbf{x}) = \sum_{j=1}^n C_j \Phi_j(\mathbf{x}). \quad (41.1)$$

Thus finding a finite element solution amounts to finding the best values for the constants $\{C_j\}_{j=1}^n$.

In the program `mywasher.m`, the vector $\mathbf{z}$ contains the node values C_j . These values give the height at each node in the graph. For instance if we set all equal to 0 except one equal to 1, then the function is a finite element. Do this for one boundary node, then for one interior node.

Notice that a sum of linear functions is a linear function. Thus the solution using linear elements is a piecewise linear function. Also notice that if we denote the j -th vertex by $\mathbf{v}_j$, then

$$U(\mathbf{v}_j) = C_j. \quad (41.2)$$

Thus we see that **the constants C_j are just the values at the nodes.**

Take the one-dimensional case. Since we know that the solution is linear on each subinterval, knowing the values at the endpoints of the subintervals, i.e. the nodes, gives us complete knowledge of the solution. Figure 41.3 could be a finite element solution since it is piecewise linear.

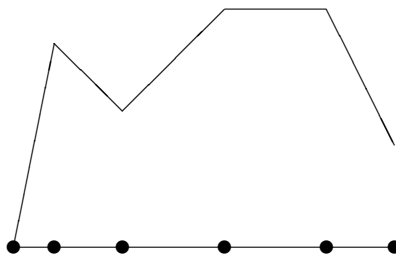


Figure 41.3: A possible finite element solution for a one dimensional object. Values are assigned at each node and a linear interpolant is used in between.

In the two-dimensional case, the solution is linear on each triangle, and so again, if we know the values $\{C_j\}_{j=1}^n$ at the nodes then we know everything.

Experiment with finite elements

By changing the values in $\mathbf{z}$ in the program we can produce different three dimensional shapes based on the triangles. The point then of a finite element solution is to find the values at the nodes that best approximate the true solution. This task can be subdivided into two parts: (1) assigning the values at the boundary nodes and (2) assigning the values at the interior nodes.

Values at boundary nodes

Once a triangulation and a set of finite elements is set up, the next step is to incorporate the boundary conditions of the problem. Suppose that we have fixed boundary conditions, i.e. of the form

$$u(\mathbf{x}) = g(\mathbf{x}) \quad \text{for } \mathbf{x} \in \partial D,$$

where D is the object (domain) and ∂D is its boundary. Then the boundary condition directly determines the values on the boundary nodes.

In particular, suppose that $\mathbf{v}_\ell$ is a boundary node. Since $\Phi_\ell(\mathbf{v}_\ell) = 1$ and all the other elements are zero at node $\mathbf{v}_\ell$, then to make

$$U(\mathbf{v}_\ell) = \sum_{j=1}^n C_j \Phi_j(\mathbf{v}_\ell) = g(\mathbf{v}_\ell),$$

we must choose

$$C_\ell = g(\mathbf{v}_\ell).$$

Thus **the constants C_j for the boundary nodes are set at exactly the value of the boundary condition at the nodes.**

Thus, if there are m interior nodes, then

$$C_j = g(\mathbf{v}_j), \quad \text{for all } m+1 \leq j \leq n.$$

In the program `mywasher.m` the first 32 vertices correspond to interior nodes and the last 32 correspond to boundary nodes. By setting the last 32 values of $\mathbf{z}$, we achieve the boundary conditions. We could do this by adding the following commands to the program:

```
z(33:64) = .5;
```

or more elaborately we might use functions:

```
z(33:48) = x(33:48).^2 - .5*y(33:48).^2;
z(49:64) = .2*cos(y(49:64));
```

Exercises

41.1 Generate an interesting or useful 2-d object and a well-distributed triangulation of it. Make it have at least 2 interior nodes and have it list the interior nodes before the boundary nodes.

- Plot the region.
- Plot one interior finite element.
- Plot one boundary finite element.
- Assign values to the boundary using a function (or functions) and plot the region with the boundary values.

Turn in your code and the four plots.

Lecture 42

Determining Internal Node Values

As seen in the previous section, a finite element solution of a boundary value problem boils down to finding the best values of the constants $\{C_j\}_{j=1}^n$, which are the values of the solution at the nodes. The interior nodes values are determined by *variational principles*. Variational principles usually amount to **minimizing internal energy**. It is a physical principle that systems seek to be in a state of minimal energy and this principle is used to find the internal node values.

Variational Principles

For the differential equations that describe many physical systems, the internal energy of the system is an integral. For instance, for the steady state heat equation

$$u_{xx}(x, y) + u_{yy}(x, y) = g(x, y) \quad (42.1)$$

the internal energy is the integral

$$I[u] = \iint_R u_x^2(x, y) + u_y^2(x, y) + 2g(x, y)u(x, y) dA, \quad (42.2)$$

where R is the region on which we are working. It can be shown that $u(x, y)$ is a solution of (42.1) if and only if it is minimizer of $I[u]$ in (42.2).

The finite element solution

Recall that a finite element solution is a linear combination of finite element functions:

$$U(x, y) = \sum_{j=1}^n C_j \Phi_j(x, y),$$

where n is the number of nodes. To obtain the values at the internal nodes, we will plug $U(x, y)$ into the energy integral and minimize. That is, we find the minimum of

$$I[U]$$

for all choices of $\{C_j\}_{j=1}^m$, where m is the number of internal nodes. In this as with any other minimization problem, the way to find a possible minimum is to differentiate the quantity with respect to the variables and set the results to zero. In this case the free variables are $\{C_j\}_{j=1}^m$. Thus to find the minimizer we should try to solve

$$\frac{\partial I[U]}{\partial C_j} = 0 \quad \text{for } 1 \leq j \leq m. \quad (42.3)$$

We call this set of equations the **internal node equations**. At this point we should ask whether the internal node equations can be solved, and if so, is the solution actually a minimizer (and not a maximizer). The following two facts answer these questions. These facts make the finite element method practical:

- For most applications the internal node equations are linear.
- For most applications the internal node equations give a minimizer.

We can demonstrate the first fact using an example.

Application to the steady state heat equation

If we plug the candidate finite element solution $U(x, y)$ into the energy integral for the heat equation (42.2), we obtain

$$I[U] = \iint_R U_x(x, y)^2 + U_y(x, y)^2 + 2g(x, y)U(x, y) dA. \quad (42.4)$$

Differentiating with respect to C_j we obtain the internal node equations

$$0 = \iint_R 2U_x \frac{\partial U_x}{\partial C_j} + 2U_y \frac{\partial U_y}{\partial C_j} + 2g(x, y) \frac{\partial U}{\partial C_j} dA \quad \text{for } 1 \leq j \leq m. \quad (42.5)$$

Now we have several simplifications. First note that since

$$U(x, y) = \sum_{j=1}^n C_j \Phi_j(x, y),$$

we have

$$\frac{\partial U}{\partial C_j} = \Phi_j(x, y).$$

Also note that

$$U_x(x, y) = \sum_{j=1}^n C_j \frac{\partial}{\partial x} \Phi_j(x, y),$$

and so

$$\frac{\partial U_x}{\partial C_j} = (\Phi_j)_x.$$

Similarly, $\frac{\partial U_y}{\partial C_j} = (\Phi_j)_y$. The integral (42.5) then becomes

$$0 = 2 \iint U_x(\Phi_j)_x + U_y(\Phi_j)_y + g(x, y)\Phi_j(x, y) dA \quad \text{for } 1 \leq j \leq m.$$

Next we use the fact that the region R is subdivided into triangles $\{T_i\}_{i=1}^p$ and the functions in question have different definitions on each triangle. The integral then is a sum of the integrals:

$$0 = 2 \sum_{i=1}^p \iint_{T_i} U_x(\Phi_j)_x + U_y(\Phi_j)_y + g(x, y)\Phi_j(x, y) dA \quad \text{for } 1 \leq j \leq m.$$

Now note that the function $\Phi_j(x, y)$ is linear on triangle T_i and so

$$\Phi_{ij}(x, y) = \Phi_j|_{T_i}(x, y) = a_{ij} + b_{ij}x + c_{ij}y.$$

This gives us the simplifications

$$(\Phi_{ij})_x(x, y) = b_{ij} \quad \text{and} \quad (\Phi_{ij})_y(x, y) = c_{ij}.$$

Also, U_x and U_y restricted to T_i have the form

$$U_x = \sum_{k=1}^n C_k b_{ik} \quad \text{and} \quad U_y = \sum_{k=1}^n C_k c_{ik}.$$

The internal node equations then reduce to

$$0 = \sum_{i=1}^p \iint_{T_i} \left(\sum_{k=1}^n C_k b_{ik} \right) b_{ij} + \left(\sum_{k=1}^n C_k c_{ik} \right) c_{ij} + g(x, y) \Phi_{ij}(x, y) dA \quad \text{for } 1 \leq j \leq m.$$

Now notice that $(\sum_{k=1}^n C_k b_{ik}) b_{ij}$ is just a constant on T_i , and, thus, we have

$$\iint_{T_i} \left(\sum_{k=1}^n C_k b_{ik} \right) b_{ij} + \left(\sum_{k=1}^n C_k c_{ik} \right) c_{ij} = \left[\left(\sum_{k=1}^n C_k b_{ik} \right) b_{ij} + \left(\sum_{k=1}^n C_k c_{ik} \right) c_{ij} \right] A_i,$$

where A_i is just the area of T_i . Finally, we apply the Three Corners rule to make an approximation to the integral

$$\iint_{T_i} g(x, y) \Phi_{ij}(x, y) dA.$$

Since $\Phi_{ij}(x_k, y_k) = 0$ if $k \neq j$ and even $\Phi_{ij}(x_j, y_j) = 0$ if T_i does not have a corner at (x_j, y_j) , we get the approximation

$$\Phi_{ij}(x_j, y_j) g(x_j, y_j) A_i / 3.$$

If T_i does have a corner at (x_j, y_j) then $\Phi_{ij}(x_j, y_j) = 1$.

Summarizing, the internal node equations are

$$0 = \sum_{i=1}^p \left[\left(\sum_{k=1}^n C_k b_{ik} \right) b_{ij} + \left(\sum_{k=1}^n C_k c_{ik} \right) c_{ij} + \frac{1}{3} g(x_j, y_j) \Phi_{ij}(x_j, y_j) \right] A_i \quad \text{for } 1 \leq j \leq m.$$

While not pretty, these equations are in fact linear in the unknowns $\{C_j\}$.

Experiment

Download the program `myfiniteelem.m`. This program produces a finite element solution for the steady state heat equation without source term:

$$u_{xx} + u_{yy} = 0.$$

To use it, you first need to set up the region and boundary values by running a script such as `mywasher.m` or `mywedge.m`. Try different settings for the boundary values `z`. You will see that the program works no matter what you choose.

Exercises

42.1 For this exercise, use the region that you made in Exercise 41.1.

- Set non-constant boundary values and plot.
- Then run `myfiniteelem.m` and plot.

Does the second plot look like it satisfies a variational principle? Why or why not? Turn in the two plots and your answers to the questions.

Review of Part IV

Methods and Formulas

Initial Value Problems

Reduction to First order system:

For an n -th order equation that can be solved for the n -th derivative

$$x^{(n)} = f\left(t, x, \dot{x}, \ddot{x}, \dots, \frac{d^{n-1}x}{dt^{n-1}}\right) \quad (42.6)$$

use the standard change of variables:

$$\begin{aligned} y_1 &= x \\ y_2 &= \dot{x} \\ &\vdots \\ y_n &= x^{(n-1)} = \frac{d^{n-1}x}{dt^{n-1}}. \end{aligned} \quad (42.7)$$

Differentiating results in a first-order system:

$$\begin{aligned} \dot{y}_1 &= \dot{x} = y_2 \\ \dot{y}_2 &= \ddot{x} = y_3 \\ &\vdots \\ \dot{y}_n &= x^{(n)} = f(t, y_1, y_2, \dots, y_n). \end{aligned} \quad (42.8)$$

Euler's method:

$$\mathbf{y}_{i+1} = \mathbf{y}_i + hf(t_i, \mathbf{y}_i).$$

Modified (or Improved) Euler method:

$$\begin{aligned} \mathbf{k}_1 &= hf(t_i, \mathbf{y}_i) \\ \mathbf{k}_2 &= hf(t_i + h, \mathbf{y}_i + \mathbf{k}_1) \\ \mathbf{y}_{i+1} &= \mathbf{y}_i + \frac{1}{2}(\mathbf{k}_1 + \mathbf{k}_2) \end{aligned}$$

Boundary Value Problems

Finite Differences:

Replace the Differential Equation by Difference Equations on a grid.
Review the lecture on Numerical Differentiation.

Explicit Method Finite Differences for Parabolic PDE (heat):

$$u_t \mapsto \frac{u_{i,j+1} - u_{ij}}{k} \quad \text{and} \quad u_{xx} \mapsto \frac{u_{i-1,j} - 2u_{ij} + u_{i+1,j}}{h^2} \quad (42.9)$$

leads to

$$u_{i,j+1} = ru_{i-1,j} + (1 - 2r)u_{ij} + ru_{i+1,j},$$

where $h = L/m$, $k = T/n$, and $r = ck/h^2$. The stability condition is $r < 1/2$.

Implicit Method Finite Differences for Parabolic PDE (heat):

$$u_t \mapsto \frac{u_{i,j+1} - u_{ij}}{k} \quad \text{and} \quad u_{xx} \mapsto \frac{u_{i+1,j+1} - 2u_{i,j+1} + u_{i-1,j+1}}{h^2} \quad (42.10)$$

leads to

$$u_{i,j} = -ru_{i-1,j+1} + (1 + 2r)u_{i,j+1} - ru_{i+1,j+1},$$

which is always stable and has truncation error $O(h^2 + k)$.

Crank-Nicholson Method Finite Differences for Parabolic PDE (heat):

$$-ru_{i-1,j+1} + 2(1 + r)u_{i,j+1} - ru_{i+1,j+1} = ru_{i-1,j} + 2(1 - r)u_{i,j} + ru_{i+1,j},$$

which is always stable and has truncation error $O(h^2 + k^2)$.

Finite Difference Method for Elliptic PDEs:

$$u_{xx} + u_{yy} = f(x, y) \mapsto \frac{u_{i+1,j} - 2u_{ij} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{ij} + u_{i,j-1}}{k^2} = f(x_i, y_j) = f_{ij},$$

Finite Elements:

Based on triangles instead of rectangles.

Can be used for irregularly shaped objects.

An element: Pyramid shaped function at a node.

A finite element solution is a linear combination of finite element functions:

$$U(x, y) = \sum_{j=1}^n C_j \Phi_j(x, y),$$

where n is the number of nodes, and where U is an approximation of the true solution.

C_j is the value of the solution at node j .

C_j at the boundary nodes are given by boundary conditions.

C_j at interior nodes are determined by variation principles.

The last step in determining C_j 's is solving a linear system of equations.

MATLAB

Initial value problem solver that uses the Runge-Kutta 45 method, which has error $O(h^5)$. The input $y0$ is the initial vector and **tspan** is the time span. You can either make f a vector valued anonymous function and do

```
>> df = @(t,y)[-y(2);y(1)]
>> [T Y] = ode45(dy,tspan,y0)
```

or make a function program that outputs a vector

```
function dy = myf(t,y)
    dy = [-y(2);y(1)];
end
```

and then do

```
>> [T Y] = ode45(@myf,tspan,y0)
```

The program **ode45** and other MATLAB IVP solvers use adaptive step size to achieve a desired local and global accuracy, with a default of **tol** = 10^{-6} for the global error.

The chief benefit of higher order methods and variable step size is that they allow a program to take only as few steps as necessary.

Part V

Appendices

©Copyright, Todd Young and Martin Mohlenkamp, Department of Mathematics, Ohio University, 2026

Appendix A

Glossary of Matlab Commands

Mathematical Operations

- `+` Addition. Type `help plus` for information.
- `-` Subtraction. Type `help minus` for information.
- `*` Scalar or matrix multiplication. Type `help mtimes` for information.
- `/` Scalar or right matrix division. Type `help slash` for information.
For matrices, the command `A/B` is equivalent to `A*inv(B)`.
- `^` Scalar or matrix powers. Type `help mpower` for information.
- `.*` Element by element multiplication. Type `help times` for information.
- `.^` Element by element exponentiation. Type `help power` for information.
- `./` Element by element division.

Built-in Mathematical Constants

- `eps` Machine epsilon, i.e. approximately the computer's floating point roundoff error.
- `i` $\sqrt{-1}$.
- `Inf` ∞ .
- `NaN` Not a number. Indicates an invalid operation such as `0/0`.
- `pi` $\pi = 3.14159\dots$

Built-in Mathematical Functions

- `abs(x)` Absolute value $|x|$.
- `acos(x)` Inverse cosine $\arccos x$.
- `asin(x)` Inverse sine $\arcsin x$.

`atan(x)` Inverse tangent $\arctan x$.
`cos(x)` Cosine $\cos x$.
`cosh(x)` Hyperbolic cosine $\cosh x$.
`cot(x)` Cotangent $\cot x$.
`exp(x)` Exponential function $e^x = \exp x$.
`log(x)` Natural logarithm $\ln x = \log_e x$.
`sec(x)` Secant $\sec x$.
`sin(x)` Sine $\sin x$.
`sinh(x)` Hyperbolic sine $\sinh x$.
`sqrt(x)` Square root $\sqrt{x}$.
`tan(x)` Tangent $\tan x$.
`tanh(x)` Hyperbolic tangent $\tanh x$.
`max` Computes maximum of the rows of a matrix.
`mean` Computes the average of the rows of a matrix.
`min` Computes the minimum of the rows of a matrix.

Built-in Numerical Mathematical Operations

`fzero` Tries to find a zero of the specified function near a starting point or on a specified interval.
`inline` Define a function in the command window.
`ode113` Numerical multiple step ODE solver.
`ode45` Runge-Kutta 45 numerical ODE solver.
`quad` Numerical integration using an adaptive Simpson's rule.
`dblquad` Double integration.
`triplequad` Triple integration.

Built-in Symbolic Mathematical Operations

`collect` Collects powers of the specified variable in a given symbolic expression.
`compose` Composition of symbolic functions.
`diff` Symbolic differentiation.
`double` Displays double-precision representation of a symbolic expression.
`dsolve` Symbolic ODE solver.
`expand` Expands an algebraic expression.
`factor` Factor a polynomial.

<code>int</code>	Symbolic integration; either definite or indefinite.
<code>limit</code>	Finds two-sided limit, if it exists.
<code>pretty</code>	Displays a symbolic expression in a nice format.
<code>simple</code>	Simplifies a symbolic expression.
<code>subs</code>	Substitutes for parts a a symbolic expression.
<code>sym</code> or <code>syms</code>	Create symbolic variables.
<code>symsum</code>	Performs a symbolic summation, possibly with infinitely many entries.
<code>taylor</code>	Gives a Taylor polynomial approximation of a given order at a specified point.

Graphics Commands

<code>contour</code>	Plots level curves of a function of two variables.
<code>contourf</code>	Filled contour plot.
<code>ezcontour</code>	Easy contour plot.
<code>loglog</code>	Creates a log-log plot.
<code>mesh</code>	Draws a mesh surface.
<code>meshgrid</code>	Creates arrays that can be used as inputs in graphics commands such as <code>contour</code> , <code>mesh</code> , <code>quiver</code> , and <code>surf</code> .
<code>ezmesh</code>	Easy mesh surface plot.
<code>plot</code>	Plots data vectors.
<code>ezplot</code>	Easy plot for symbolic functions.
<code>plot3</code>	Plots curves in 3-D.
<code>polar</code>	Plots in polar coordinates.
<code>quiver</code>	Plots a vector field.
<code>semilogy</code>	Semilog plot, with logarithmic scale along the vertical direction.
<code>surf</code>	Solid surface plot.
<code>trimesh</code>	Plot based on a triangulation
<code>trisurf</code>	Surface plot based on a triangulation

Special Matlab Commands

<code>:</code>	Range operator, used for defining vectors and in loops. Type <code>help colon</code> for information.
<code>;</code>	Suppresses output. Also separates rows of a matrix.
<code>=</code>	Assigns the variable on the left hand side the value of the right hand side.

ans The value of the most recent unassigned.

cd Change directory.

clear Clears all values and definitions of variables and functions. You may also use to clear only specified variables.

diary Writes a transcript of a MATLAB session to a file.

dir Lists the contents in the current working directory. Same as **ls**.

help

inline Define an inline function.

format Specifies output format, e.g. **> format long**.

load Load variables from a file.

save Saves workspace variables to a file.

Matlab Programming

== Is equal?

~= Is not equal?

< Less than?

> Greater than?

<= Less than or equal?

break Breaks out of a **for** or **while** loop.

end Terminates an **if**, **for** or **while** statement.

else Alternative in an **if** statement.

error Displays an error message and ends execution of a program.

for Repeats a block of commands a specified number of times.

function First word in a function program.

if Checks a condition before executing a block of statements.

return Terminates execution of a program.

warning Displays a warning message.

while Repeats a block of commands as long as a condition is true.

Commands for Matrices and Linear Algebra

Matrix arithmetic:

A = [1 3 -2 5 ; -1 -1 5 4 ; 0 1 -9 0] Manually enter a matrix.

```
u = [ 1  2  3  4]'
```

```
A*u
```

```
B = [3 2 1; 7 6 5; 4 3 2]
```

```
B*A .....multiply  $B$  times  $A$ .
```

```
2*A .....multiply a matrix by a scalar.
```

```
A + A .....add matrices.
```

```
A + 3 .....add a number to every entry of a matrix.
```

```
B.*B .....component-wise multiplication.
```

```
B.^3 .....component-wise exponentiation.
```

Special matrices:

```
I = eye(3) .....identity matrix
```

```
D = ones(5,5)
```

```
O = zeros(10,10)
```

```
C = rand(5,5) .....random matrix with uniform distribution in  $[0, 1]$ .
```

```
C = randn(5,5) .....random matrix with normal distribution.
```

```
hilb(6)
```

```
pascal(5)
```

General matrix commands:

```
size(C) .....gives the dimensions ( $m \times n$ ) of  $A$ .
```

```
norm(C) .....gives the norm of the matrix.
```

```
det(C) .....the determinant of the matrix.
```

```
max(C) .....the maximum of each row.
```

```
min(C) .....the minimum in each row.
```

```
sum(C) .....sums each row.
```

```
mean(C) .....the average of each row.
```

```
diag(C) .....just the diagonal elements.
```

```
inv(C) .....inverse of the matrix.
```

Matrix decompositions:

```
[L U P] = lu(C)
```

```
[Q R] = qr(C)
```

```
[U S V] = svd(C) .....singular value decomposition.
```